


```
class Demo{
    public static void main(String args[]){
        int[] ar=new int[3];
        ar[0]=100;
        ar[1]=200;
        ar[2]=300;
        //-----
        System.out.println("ar[0] : "+ar[0]); //prints 100
        System.out.println("ar[1] : "+ar[1]);
        System.out.println("ar[2] : "+ar[2]);
        //-----

        Student s1=new Student();
        s1.stId="S0001";
        s1.stName="Nimal";
        s1.prfMarks=85;
        s1.dbmsMarks=78;
        System.out.println("Student Id : "+s1.stId); //S001
        System.out.println("Student Name : "+s1.stName); //Nimal
        System.out.println("PRF Marks : "+s1.prfMarks);
        System.out.println("DBMS Marks : "+s1.dbmsMarks);
    }
}
```

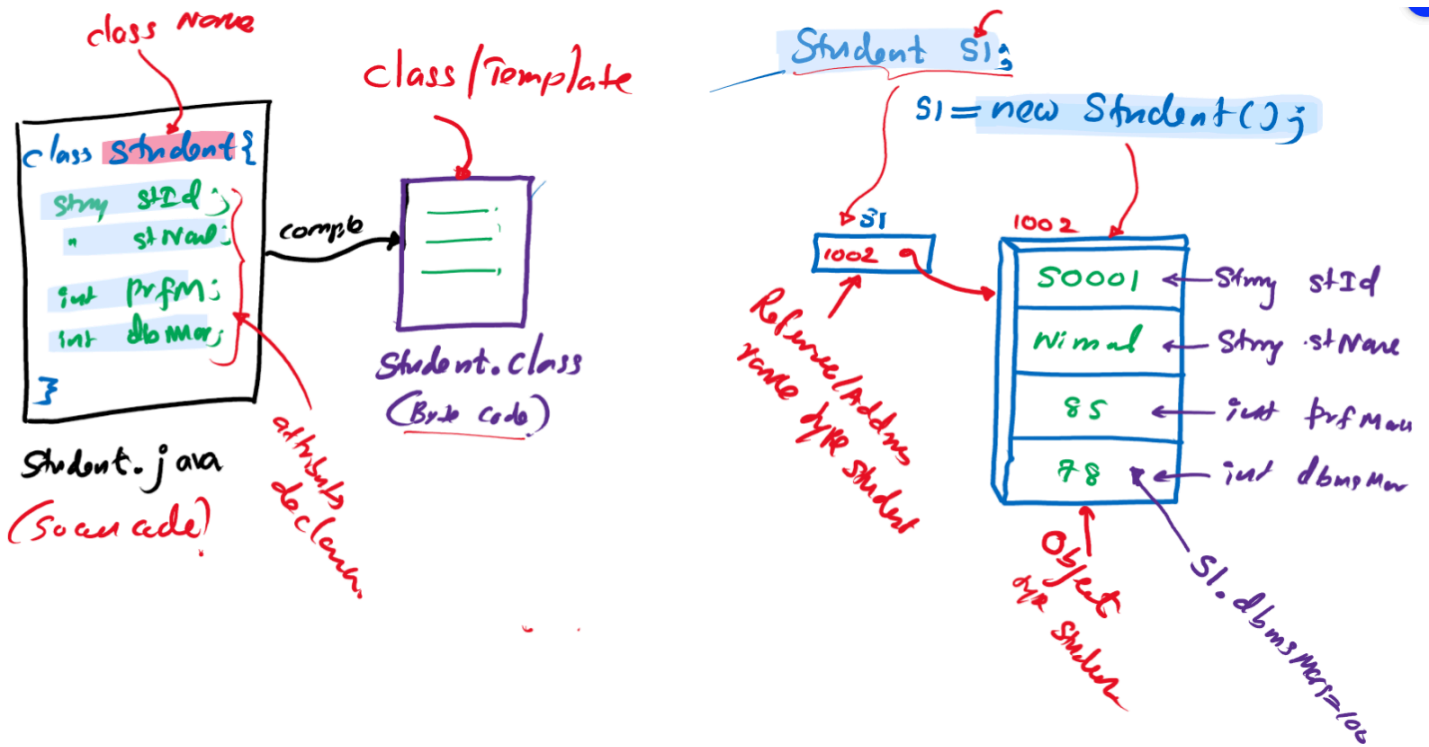
```
Q3
class Student{
    String stId;
    String stName;
    int prfMarks;
    int dbmsMarks;
}

class Demo{
    public static void main(String args[]){

        Student s1=new Student();
        s1.stId="S0001";
        s1.stName="Nimal";
        s1.prfMarks=85;
        s1.dbmsMarks=78;
        System.out.println("Student Id  : "+s1.stId); //S001
        System.out.println("Student Name : "+s1.stName); //Nimal
        System.out.println("PRF Marks   : "+s1.prfMarks);
        System.out.println("DBMS Marks  : "+s1.dbmsMarks);

    }
}
```

Q4



Q5

```
class Student{
    String stId; //attribute declaration
    String stName; //attribute declaration
    int prfMks; //attribute declaration
    int dbmsMks; //attribute declaration
}

class Demo{
    public static void main(String args[]){
        Student s1; //Create a reference/address
        s1=new Student(); //Create an Object type Student

        s1.stId="S0001"; //assign data
        s1.stName="Nimal";
        s1.prfMks=85;
        s1.dbmsMks=78;

        System.out.println("Student Id : "+s1.stId); //S001
        System.out.println("Student Name : "+s1.stName); //Nimal
        System.out.println("PRF Marks : "+s1.prfMks);
        System.out.println("DBMS Marks : "+s1.dbmsMks);
    }
}
```

```
////////////////////////////////////
Q6 Exercise
```

```
=====
```

```
class Demo{
    public static void main(String args[]){
        Box b1;
        b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;
        //-----

        System.out.println("Length : "+b1.length);
        System.out.println("Width : "+b1.width);
        System.out.println("Height : "+b1.height);

    }
}
```

```
////////////////////////////////////
Q7 From Q6
```

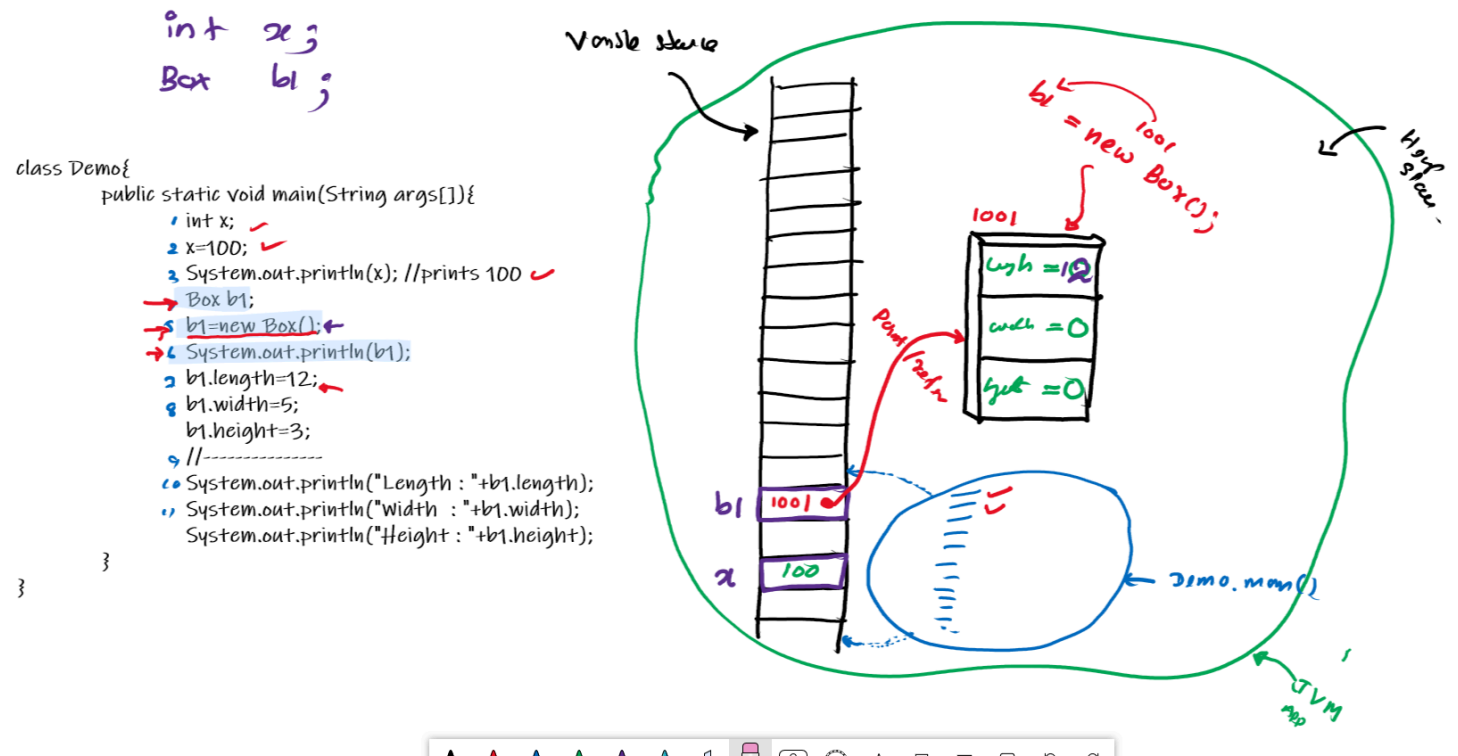
```
=====
```

```
class Box{
    int length;
    int width;
    int height;
}
class Demo{
    public static void main(String args[]){
        Box b1;
        b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;
        //-----

        System.out.println("Length : "+b1.length);
        System.out.println("Width : "+b1.width);
        System.out.println("Height : "+b1.height);

    }
}
```


Q8



Q9 Adding a method

```
class Box{  
    int length;  
    int width;  
    int height;  
}  
  
class Demo{  
    public static void main(String args[]){  
        Box b1=new Box();  
        b1.length=12;  
        b1.width=5;  
        b1.height=3;  
  
        int volume;  
        volume=b1.length*b1.width*b1.height;  
        System.out.println("Volume of the box : "+volume); //180  
    }  
}
```

Q10

```
class Box{  
    int length;  
    int width;  
    int height;  
}
```

```
class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;

        /*
        int volume;
        volume=b1.length*b1.width*b1.height;
        System.out.println("Volume of the box : "+volume); //180
        */
        b1.printVolume(); //180
    }
}
```

////////////////////////////////////

Q11 From Q10

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume); //180
    }
}
```

```
class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;

        /*
        int volume;
        volume=b1.length*b1.width*b1.height;
        System.out.println("Volume of the box : "+volume); //180
        */
        b1.printVolume(); //180
    }
}
```

```
////////////////////////////////////
```

Q12 Exercise

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume); //180
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;
        b1.printVolume(); //180
        b1.printArea(); //222
    }
}
```

```
////////////////////////////////////
```

Q13 From Q12

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void printArea(){
        int area;
        area=2*length*width+2*length*height+2*width*height;
        System.out.println("Area of the box : "+area);
    }
}
```

```

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;
        b1.printVolume(); //180
        b1.printArea(); //222
    }
}

```

Q14 Local Area, Local Variables

=====

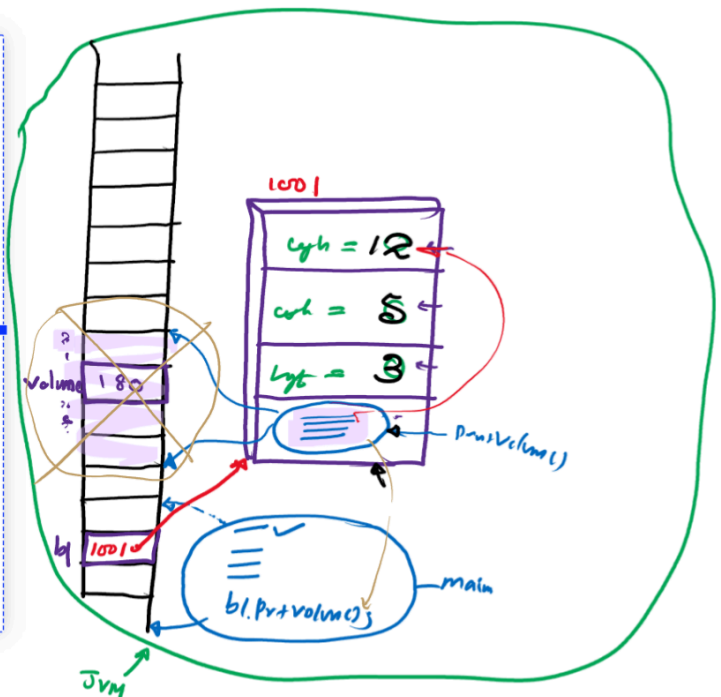
```

class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        1 int volume;
        2 volume=length*width*height;
        3 System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.length=12;
        b1.width=5;
        b1.height=3;
        b1.printVolume(); //180
    }
}

```



Q15 Adding a Parameterized method

=====

```

class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

```

////////////////////////////////////

Q16 From Q15

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

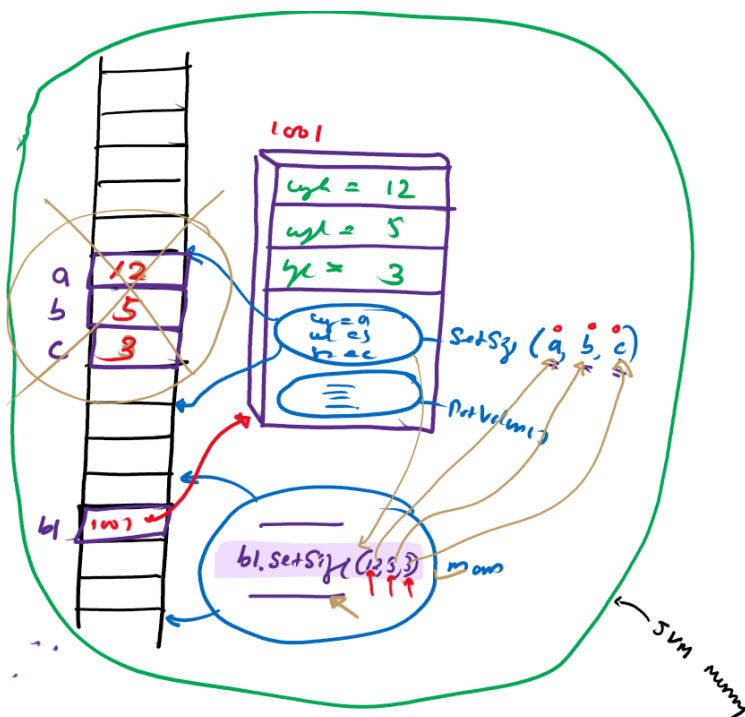
    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int a, int b, int c){
        length=a;
        width=b;
        height=c;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
    }
}
```

Q17

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int a, int b, int c){
        length=a;
        width=b;
        height=c;
    }
}
class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
    }
}
```



Q18 Exercise

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        length=length;
        width=width;
        height=height;
    }
}
class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //?
        System.out.println("Length : "+b1.length); //0
        System.out.println("Width : "+b1.width); //0
        System.out.println("Height : "+b1.height); //0
    }
}
```

////////////////////////////////////
Q19 Keyword "this"

=====

```
class Test{
    int a;

    public void print(){
        System.out.println(this);
    }
}
class Demo{
    public static void main(String args[]){
        Test t1=new Test();
        t1.print();
        System.out.println(t1);
        System.out.println("-----\n");

        Test t2=new Test();
        t2.print();
        System.out.println(t2);
    }
}
```

////////////////////////////////////

Q20 From Q18

=====

```
class Box{
    int length;
    int width;
    int height;

    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
}
class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.length); //12
        System.out.println("Width : "+b1.width); //5
        System.out.println("Height : "+b1.height); //3
    }
}
```

```

class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.length); //12
        System.out.println("Width : "+b1.width); //5
        System.out.println("Height : "+b1.height); //3

        b1.setLength(120);
        b1.setWidth(50);
        b1.setHeight(30);
        b1.printVolume(); //180000
        System.out.println("Length : "+b1.length); //120
        System.out.println("Width : "+b1.width); //50
        System.out.println("Height : "+b1.height); //30
    }
}

```


Q22 From Q21

=====

```
class Box{
    //-----attributes declarations-----
    int length;
    int width;
    int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void setLength(int length){
        this.length=length;
    }
    public void setHeight(int height){
        this.height=height;
    }
    public void setWidth(int width){
        this.width=width;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.length); //12
        System.out.println("Width : "+b1.width); //5
        System.out.println("Height : "+b1.height); //3

        b1.setLength(120);
        b1.setWidth(50);
        b1.setHeight(30);
        b1.printVolume(); //180000
        System.out.println("Length : "+b1.length); //120
        System.out.println("Width : "+b1.width); //50
        System.out.println("Height : "+b1.height); //30
    }
}
```

Q23 Encapsulated Fields

=====

```
class Box{
    //-----attributes declarations-----
    private int length;
    private int width;
    private int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void setLength(int length){
        this.length=length;
    }
    public void setHeight(int height){
        this.height=height;
    }
    public void setWidth(int width){
        this.width=width;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.length); //12
        System.out.println("Width : "+b1.width); //5
        System.out.println("Height : "+b1.height); //3
    }
}
```

////////////////////////////////////
Q24 From Q23

=====

```
class Box{
    //-----attributes declarations-----
    private int length;
    private int width;
    private int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void setLength(int length){
        this.length=length;
    }
    public void setHeight(int height){
        this.height=height;
    }
    public void setWidth(int width){
        this.width=width;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.getLength()); //12
        System.out.println("Width : "+b1.getWidth()); //5
        System.out.println("Height : "+b1.getHeight()); //3

        b1.setLength(180); //b1.length=180; //Illegal
    }
}
```

Q25 From Q24

=====

```
class Box{
    //-----attributes declarations-----
    private int length;
    private int width;
    private int height;

    //-----methods declarations-----
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
    public void setSize(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void setLength(int length){
        this.length=length;
    }
    public void setHeight(int height){
        this.height=height;
    }
    public void setWidth(int width){
        this.width=width;
    }
    public int getLength(){
        return this.length; //return length;
    }
    public int getWidth(){
        return width;
    }
    public int getHeight(){
        return height;
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.setSize(12,5,3);
        b1.printVolume(); //180
        System.out.println("Length : "+b1.getLength()); //12
        System.out.println("Width : "+b1.getWidth()); //5
        System.out.println("Height : "+b1.getHeight()); //3

        b1.setLength(180); //b1.length=180; //Illegal
    }
}
```

```
////////////////////////////////////
Q26 Exercise
```

```
=====
```

```
class Demo{
    public static void main(String args[]){
        Date d1=new Date();
        d1.setDate(2025,9,14);
        d1.printDate(); //2025-9-14
    }
}
```

```
////////////////////////////////////
Q27 From Q26
```

```
=====
```

```
class Date{
    private int year;
    private int month;
    private int day; //day of month

    public void setYear(int year){
        this.year=year;
    }
    public void setMonth(int month){
        this.month=month;
    }
    public void setDay(int day){
        this.day=day;
    }
    public int getYear(){
        return year;
    }
    public int getMonth(){
        return month;
    }
    public int getDay(){
        return day;
    }
    public void setDate(int year, int month, int day){
        setYear(year);//this.year=year;
        setMonth(month);
        setDay(day);
    }
    public void printDate(){
        System.out.println(year+"-"+month+"-"+day);
    }
}

class Demo{
    public static void main(String args[]){
        Date d1=new Date();
        d1.setDate(2026,6,7);
        d1.printDate(); //2026-6-7
    }
}
```

Object Initialization (Constructors)

Q28

```
class Box{
    int length;
    int width;
    int height;

    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.printVolume(); //?
        System.out.println("Length : "+b1.length);    // ?
        System.out.println("Width : "+b1.width);      // ?
        System.out.println("Height : "+b1.height);    // ?
    }
}
```

Q29

```
class Box{
    int length;
    int width;
    int height;
    /*Compiler inserts
    Box(){
        length=0;
        width=0;
        height=0;
    }
    */
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.printVolume(); //?
        System.out.println("Length : "+b1.length);    // ?
        System.out.println("Width : "+b1.width);      // ?
        System.out.println("Height : "+b1.height);    // ?
    }
}
```

Q30 Case I (Default Constructor)

=====

```
class Box{
    int length;
    int width;
    int height;

    Box(){ //Default Constructor
        length=12;
        width=5;
        height=3;
    }
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.printVolume(); //180
        System.out.println("Length : "+b1.length);    // 12
        System.out.println("Width : "+b1.width);      // 5
        System.out.println("Height : "+b1.height);    // 3
    }
}
```

Q31 Case II

=====

```
class Box{
    int length;
    int width;
    int height;

    Box(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box(120,50,30);
        b1.printVolume(); //180000
        System.out.println("Length : "+b1.length);    //120
    }
}
```

```
        System.out.println("Width : "+b1.width);           //50
        System.out.println("Height : "+b1.height);         //30
    }
}

//////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Q32 Case III
=====

```
class Box{
    int length;
    int width;
    int height;

    Box(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box(120,50,30);
        b1.printVolume(); //180000

        Box b2=new Box();//Illegal ?
    }
}
```

////////////////////////////////////

////////////////////////////////////
Q36 Case VII

=====

```
class Box{
    int length;
    int width;
    int height;

    Box(){
        /*length=1;
        width=1;
        height=1;*/
        this(1);
        System.out.println("Box()");
    }
    Box(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
        System.out.println("Box(int,int,int)");
    }
    Box(int length){
        /*this.length=length;
        this.width=length;
        this.height=length;*/
        //Box(length,length,length);
        this(length,length,length);
        System.out.println("Box(int)");
    }
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
    }
}
```

////////////////////////////////////
Q37 Case VIII Declaration Values

=====

```
class Box{
    int length=12;
    int width=5;
    int height=3;

    /*Compiler inserts
    Box(){
        length=12;
        width=5;
```


////////////////////////////////////
Q39

```
class Box{
    int length=12;
    int width=5;
    int height=3;

    Box(){
        /*compiler inserts
        length=12;
        width=5;
        height=3;
        */
    }
    Box(int length, int width, int height){
        /*compiler inserts
        length=12;
        width=5;
        height=3;
        */
    }
    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

class Demo{
    public static void main(String args[]){
        Box b1=new Box();
        b1.printVolume();

        Box b2=new Box(120,50,30);
        b2.printVolume();
    }
}
```

////////////////////////////////////
Q40

```
class Box{
    int length=12;
    int width=5;
    int height=3;

    Box(){

    }
    Box(int length, int width, int height){
        /*compiler inserts
        length=12;
        width=5;
        height=3;
        */
    }
}
```



```

class A {
    int a;
    {
        a = 10;
    }
    A() {
        //
    }
    A(int i) {
        //
    }
}

```

A.java

```

class A {
    int a;
    A() {
        a = 10;
    }
    A(int i) {
        a = 10;
    }
}

```

A.class

Q43

```

class A {
    int a = 100;
    A() {
        //
    }
    {
        a = 200;
    }
}

```

A.java

```

class A {
    int a;
    A() {
        a = 100;
        a = 200;
    }
}

```

A.class

Q44

```

class A {
    int a = 100;
    A() {
        //
    }
    {
        a = 200;
    }
}

```

////////////////////////////////////

////////////////////////////////////

////////////////////////////////////

```
class Demo{
    public static void main(String args[]){
        Test t1=new Test();
        t1.a=1;
        t1.b=2;
        Test t2=new Test();
        t2.a=10;
        t2.b=20;
        Test t3=new Test();
        t3.a=100;
        t3.b=200;

        System.out.println("t1 : "+t1.a+" "+t1.b); //
        System.out.println("t2 : "+t2.a+" "+t2.b); //
        System.out.println("t3 : "+t3.a+" "+t3.b); //
    }
}
```


Q47 Static Variables

=====

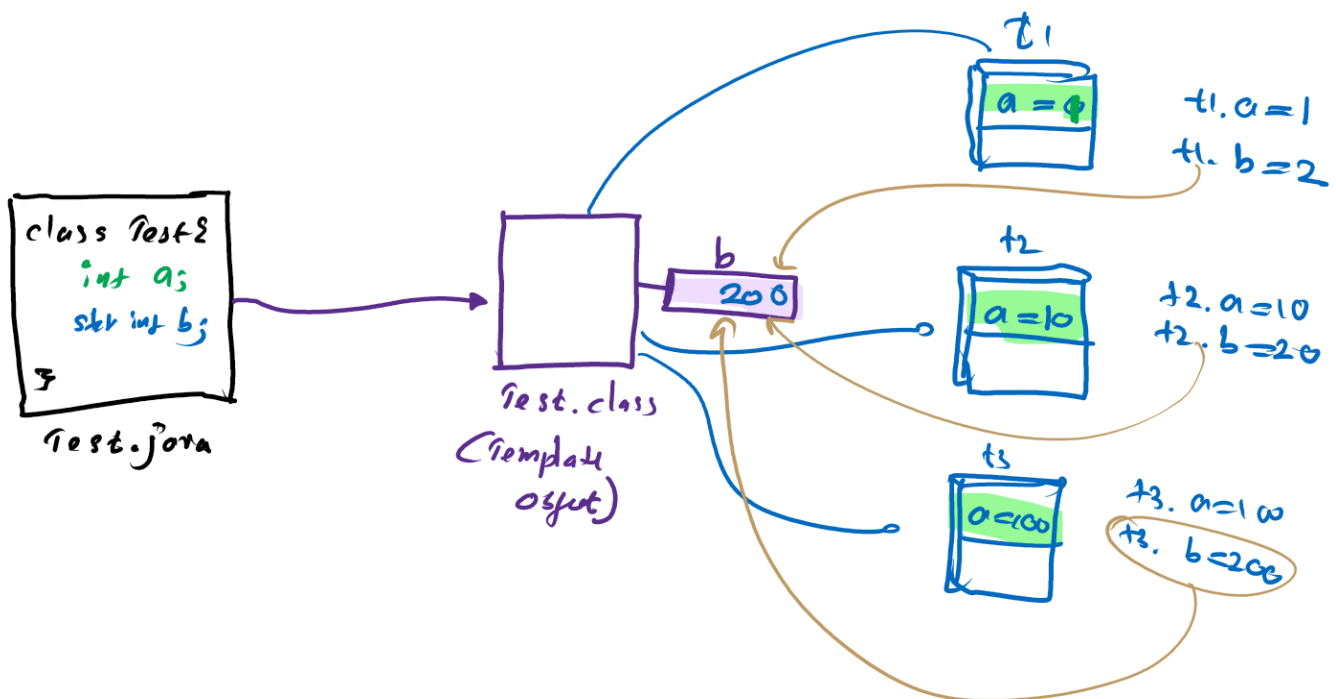
```
class Test{
    int a;
    static int b;
}

class Demo{
    public static void main(String args[]){
        Test t1=new Test();
        t1.a=1;
        t1.b=2;

        Test t2=new Test();
        t2.a=10;
        t2.b=20;

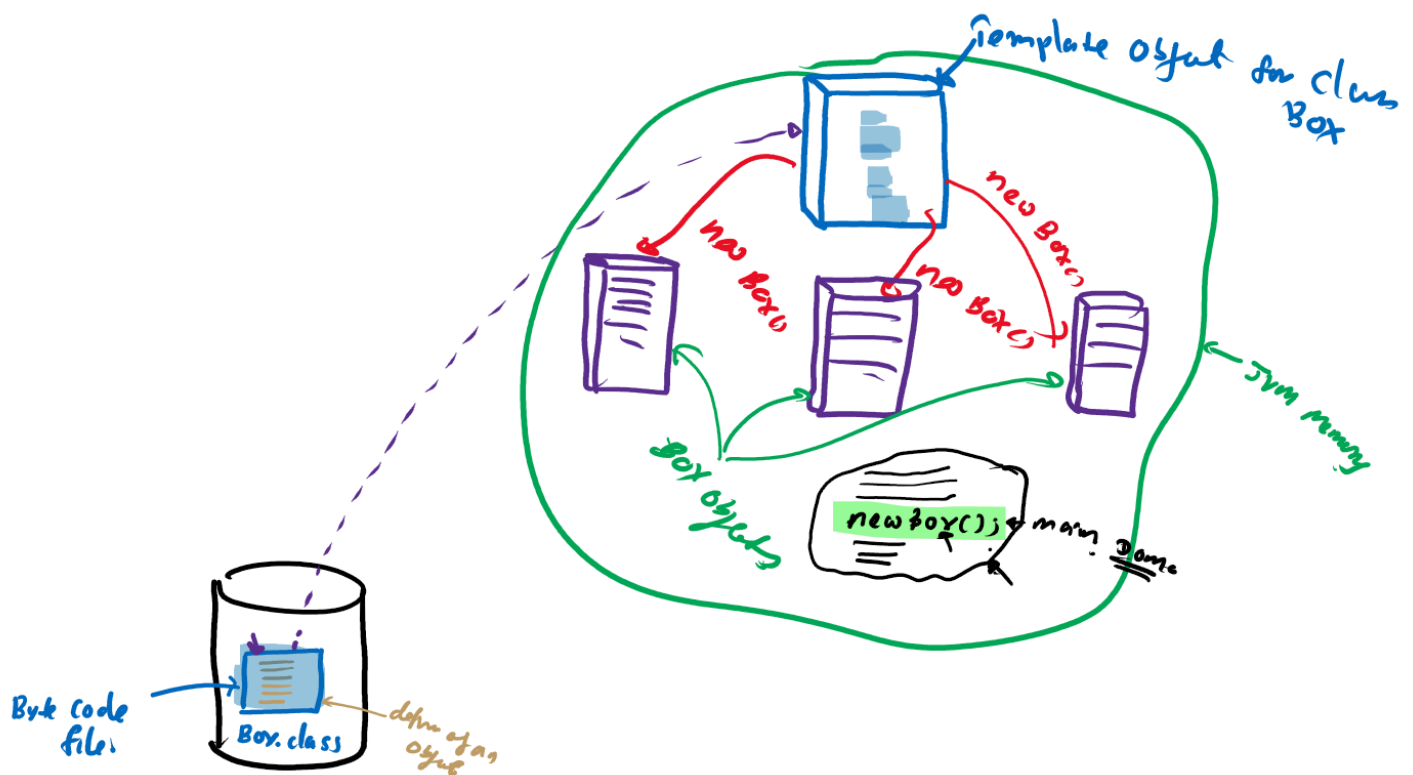
        Test t3=new Test();
        t3.a=100;
        t3.b=200;

        System.out.println("t1 : "+t1.a+" "+t1.b); //1 200
        System.out.println("t2 : "+t2.a+" "+t2.b); //10 200
        System.out.println("t3 : "+t3.a+" "+t3.b); //100 200
    }
}
```



Q48 Template Objects

=====



Q49 Static methods vs Instance Methods

=====

```
class Test{
    int a;
    static int b;
    Test(){
        a=200;
        System.out.println("Test()");
    }
    static{
        b=100;
        System.out.println("Static block");
    }
    public void mInstance(){
        System.out.println("mInstance");
    }
    public static void mStatic(){
        System.out.println("mStatic");
    }
}

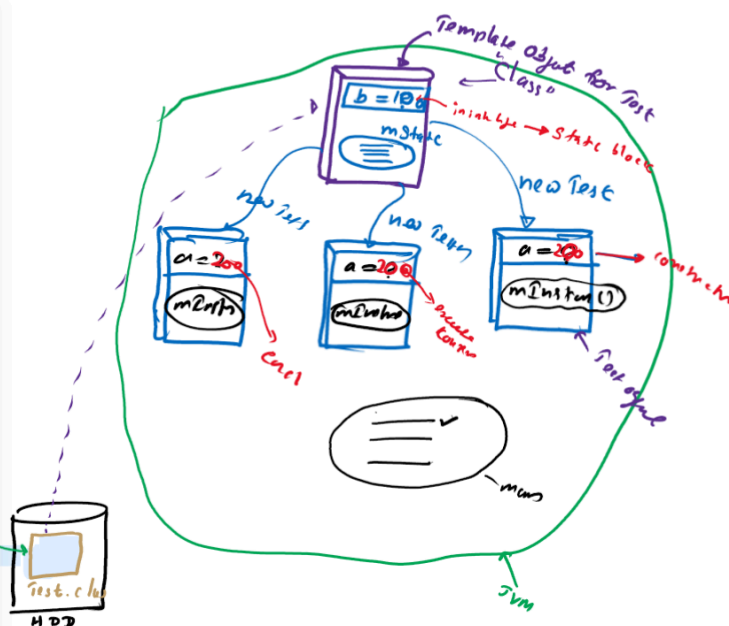
class Demo{
    public static void main(String args[]){
        Test t1=new Test();
        Test t2=new Test();
        Test t3=new Test();
    }
}
```

Q50 Static methods vs Instance Methods

```

class Test{
    int a;
    static int b;
    Test(){
        a=200;
        System.out.println("Test()");
    }
    static{
        b=100;
        System.out.println("Static block");
    }
    public void mInstance(){
        System.out.println("mInstance");
    }
    public void mStatic(){
        System.out.println("mStatic");
    }
}

class Demo{
    public static void main(String args[]){
        Test t1=new Test();
        Test t2=new Test();
        Test t3=new Test();
    }
}
    
```



Q51 Exercise

```

class Date{
    private static int year;
    private static int month;
    private static int day; //day of month

    public void setDate(int year, int month, int day){
        this.year=year;
        this.month=month;
        this.day=day;
    }
    public void printDate(){
        System.out.println(year+"-"+month+"-"+day);
    }
}

class Demo{
    public static void main(String args[]){
        Date d1=new Date();
        d1.setDate(2026,6,7);

        Date d2=new Date();
        d2.setDate(1999,12,31);

        Date d3=new Date();
        d3.setDate(2025,12,8);

        d1.printDate();
        d2.printDate();
        d3.printDate();
    }
}
    
```

Q52 Primitive vs Class Types (Reference/Objects)

```

class Order {
    int orderId;
    Date orderDate;
    String customerName;
    double delivery;
}
  
```

Primitive → int, double
 Class Type / Reference → Date, String

Java Primitive data type

- int 2g
- byte
- short
- long
- float
- double
- char
- boolean

32 bits

Order o1 = new Order();

o1 → { int, Date, String }

o1.orderDate = new Date(2026, 8, 14);
 o1.orderId = 1001

Q53 From Q36 of assignment (OOP)

```

class Date {
    public final static int YEAR=0;
    public final static int MONTH=1;
    public final static int DAY=2;

    private int year;
    private int month;
    private int day; //day of month

    public void setYear(int year){
        this.year=year;
    }
    public void setMonth(int month){
        this.month=month;
    }
    public void setDay(int day){
        this.day=day;
    }
    public int getYear(){
        return year;
    }
    public int getMonth(){
        return month;
    }
    public int getDay(){
        return day;
    }
}
  
```


Q55 From Q54

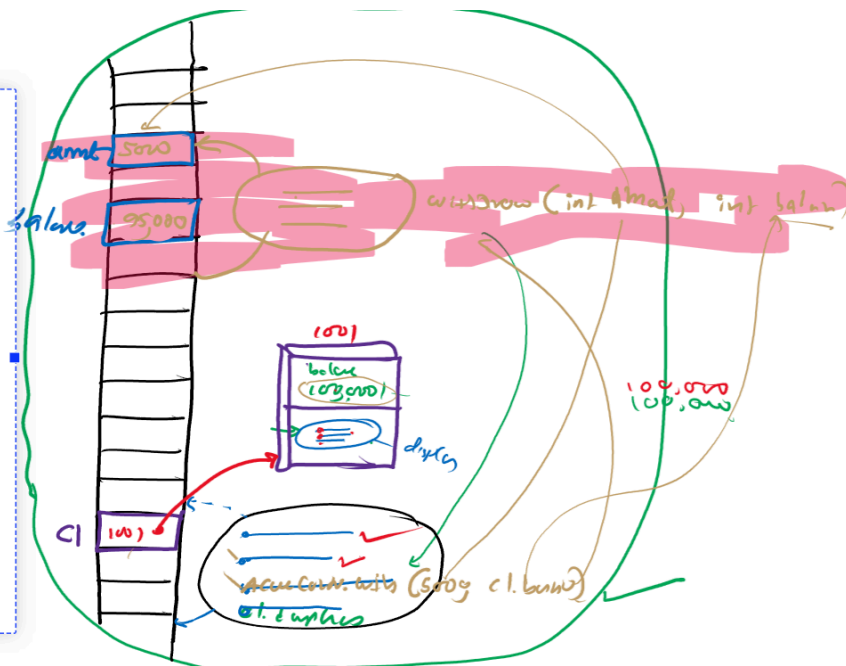
=====

```

class Account{
    int balance;
    Account(int balance){this.balance=balance;}
    public void displayBalance(){
        System.out.println("Balance : "+balance);
    }
}
class AccountController{
    public static void withdraw(int amount, int balance){
        balance-=amount; //balance=balance-amount
    }
}
class Demo{
    public static void main(String args[]){
        Account c1=new Account(100000); ✓
        c1.displayBalance(); ✓

        AccountController.withdraw(5000, c1.balance);
        c1.displayBalance();
    }
}

```



Q56 From Q55

=====

```

class Account{
    int balance;
    Account(int balance){this.balance=balance;}
    public void displayBalance(){
        System.out.println("Balance : "+balance);
    }
}
class AccountController{
    public static void withdraw(int amount, Account c1){
        c1.balance-=amount;
    }
}
class Demo{
    public static void main(String args[]){
        Account c1=new Account(100000);
        c1.displayBalance();

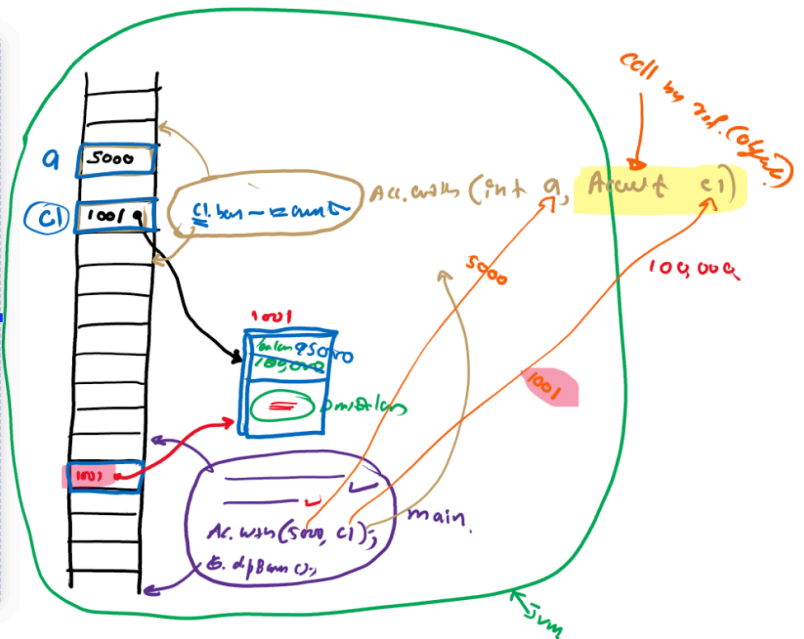
        AccountController.withdraw(5000, c1);
        c1.displayBalance();
    }
}

```

Q57 From Method Call by Reference (Object)

```
class Account{
    int balance;
    Account(int balance){this.balance=balance;}
    public void displayBalance(){
        System.out.println("Balance : "+balance);
    }
}
class AccountController{
    public static void withdraw(int amount, Account c1){
        c1.balance-=amount;
    }
}
class Demo{
    public static void main(String args[]){
        Account c1=new Account(100000);
        c1.displayBalance();

        AccountController.withdraw(5000, c1);
        c1.displayBalance();
    }
}
```



Q58 From Q50 of assignment 1 OOP

```
import java.time.*;

class Date{
    public final static int YEAR=0;
    public final static int MONTH=1;
    public final static int DAY=2;

    private int year;
    private int month;
    private int day; //day of month

    public Date(){
        this(1970,1,1);
    }
    public Date(int year, int month, int day){
        setDate(year,month,day);
    }
    public Date(Date date){
        this(date.year, date.month, date.day);
    }
    public void setYear(int year){
        this.year=year;
    }
    public void setMonth(int month){
        this.month=month;
    }
    public void setDay(int day){
        this.day=day;
    }
    public int getYear(){
        return year;
    }
}
```

```

    }
    public int getMonth(){
        return month;
    }
    public int getDay(){
        return day;
    }

    public void setDate(int year, int month, int day){
        setYear(year);//this.year=year;
        setMonth(month);
        setDay(day);
    }
    public void printDate(){
        System.out.println(year+"-"+month+"-"+day);
    }
    public void set(Date date){
        //setDate(date.year, date.month, date.day);
        set(YEAR,date.year);
        set(MONTH,date.month);
        set(DAY,date.day);
    }
    public void set(int field, int value){
        switch(field){
            case YEAR      :year=value;break;
            case MONTH     :month=value;break;
            case DAY        :day=value;break;
            default ://Error Handling
        }
    }
}

public static Date getDateInstance(){
    LocalDate dNow=LocalDate.now();
    Date date=new Date(dNow.getYear(),dNow.getMonth().getValue(),dNow.getDayOfMonth());
    return date;
}

}

class Demo{
    public static void main(String args[]){
        Date d1=new Date();
        d1.printDate(); //0-0-0

        d1.set(Date.YEAR,2016); //set(int field, int value)
        d1.set(Date.MONTH,05);
        d1.set(Date.DAY,30);
        d1.printDate(); //2016-5-30

        Date d2=new Date(2016,1,31);
        d2.printDate(); //2016-1-31

        Date d3=new Date(d2);
        d3.printDate(); //2016-1-31

        Date d4=new Date();
    }
}

```



```

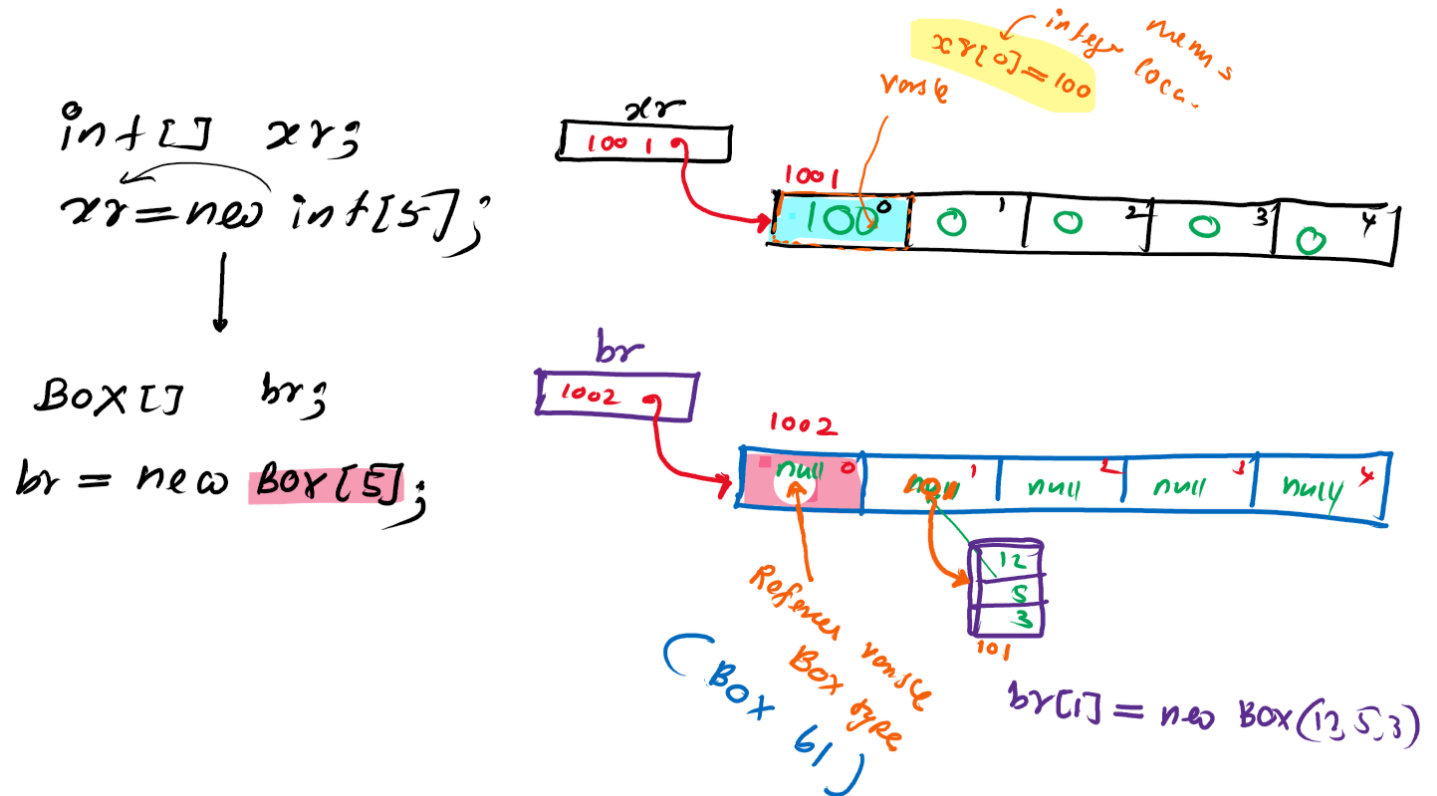
d4.set(d1);
d4.printDate();

Date d5=Date.getDateInstance();
d5.printDate(); //2026-6-14
}
}

```

Object Arrays

Q59



Q60

```

class Box{
    int length;
    int width;
    int height;

    Box(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }

    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}

```

```
class Demo{
    public static void main(String args[]){
        Box b1=new Box(12,5,3);
        b1.printVolume(); //180

        Box[] br=new Box[5];
        System.out.println(br[0]+" "+br[1]+" "+br[2]+" "+br[3]+" "+br[4]); //null null null....

        br[0]=new Box(1,2,3);
        br[1]=new Box(4,5,6);
        br[2]=new Box(2,3,4);
        br[3]=new Box(6,7,8);
        br[4]=new Box(8,9,2);

        br[2].printVolume(); //24
    }
}
```

```
class Box{
    int length;
    int width;
    int height;

    Box(int length, int width, int height){
        this.length=length;
        this.width=width;
        this.height=height;
    }

    public void printVolume(){
        int volume;
        volume=length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}
```


////////////////////////////////////

=====

//-----Demo-----

```
class Demo{
```

```

/*public static String[] idArray=new String[0];
public static String[] nameArray=new String[0];
public static int[] prfMarksArray=new int[0];
public static int[] dbmsMarksArray=new int[0];*/
public static Student[] studentArray=new Student[0];

public static void addStudent(){
    Scanner input=new Scanner(System.in);
    System.out.print("Input Student Id : ");
    String id=input.nextLine();
    System.out.print("Input Student Name : ");
    String name=input.nextLine();
    System.out.print("Input PRF Marks : ");
    int prfMarks=input.nextInt();
    System.out.print("Input DBMS Marks : ");
    int dbmsMarks=input.nextInt();

    incrementArrrays();
    Student s1=new Student(id,name,prfMarks,dbmsMarks);
    studentArray[studentArray.length-1]=s1;
}
private static void incrementArrrays(){
    Student[] tempStudentArray=new Student[studentArray.length+1];

    for(int i=0; i<studentArray.length; i++){
        tempStudentArray[i]=studentArray[i];
    }
    studentArray=tempStudentArray;
}
public static void printStudentsDetail(){
    System.out.println("ID\tName\tPRF\tDBMS");
    System.out.println("=====");
    for(int i=0; i<idArray.length; i++){
        System.out.println(idArray[i]+"\\t"+nameArray[i]+"\\t"+prfMarksArray[i]+"\\t"+dbmsMarksArray[i]);
    }
}
public static void main(String args[]){
    addStudent();
    addStudent();
    printStudentsDetail();
}
}

```

Q64 From Q63

=====

```
import java.util.*;
```

```
class Student{
```

```
String id;
```

String name;

```
int prfMarks;
```

```
int dbmsMarks;
```

```

Student(){

}

Student(String id, String name, int prfMarks, int dbmsMarks){
    this.id=id;
    this.name=name;
    this.prfMarks=prfMarks;
    this.dbmsMarks=dbmsMarks;
}

}

class Demo{
    public static Student[] studentArray=new Student[]{
        new Student("S001","Niroth",76,78),
        new Student("S002","Lakmal",96,28),
        new Student("S003","Nethuki",16,78),
    };

    public static void addStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student Id : ");
        String id=input.nextLine();
        System.out.print("Input Student Name : ");
        String name=input.nextLine();
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();

        incrementArrays();
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
        studentArray[studentArray.length-1]=s1;
    }

    private static void incrementArrays(){
        Student[] tempStudentArray=new Student[studentArray.length+1];

        for(int i=0; i<studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }

    public static void printStudentsDetail(){
        System.out.println("ID\tName\tPRF\tDBMS");
        System.out.println("=====");
        for(int i=0; i<studentArray.length; i++){

System.out.println(studentArray[i].id+"\t"+studentArray[i].name+"\t"+studentArray[i].prfMarks+"\t"+studentArray[i].db
msMarks);

        }

    }

    public static void removeStudent(){

}

```

```
public static void main(String args[]){
    //addStudent();
    //addStudent();
    printStudentsDetail();


    removeStudent();
    printStudentsDetail();
}
}

//////////////////////////////////////////////////////////////////////////////////////////////////////////////////
Q65 From Q64
=====
import java.util.*;
class Student{
    String id;
    String name;
    int prfMarks;
    int dbmsMarks;


    Student(){

    }

    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }

    public String toString(){
        return id+"\t"+name+"\t"+prfMarks+"\t"+dbmsMarks;
    }
}

class Demo{
    public static Student[] studentArray=new Student[]{
        new Student("S001","Niroth",76,78),
        new Student("S002","Lakmal",96,28),
        new Student("S003","Nethuki",16,78),

    };


    public static void addStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student Id : ");
        String id=input.nextLine();
        System.out.print("Input Student Name : ");
        String name=input.nextLine();
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();


        incrementArrays();
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
```



```

class Demo{
    public static Student[] studentArray=new Student[]{
        new Student("S001","Niroth",76,78),
        new Student("S002","Lakmal",96,28),
        new Student("S003","Nethuki",16,78),
    };

    public static void addStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student Id : ");
        String id=input.nextLine();
        System.out.print("Input Student Name : ");
        String name=input.nextLine();
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();

        incrementArrays();
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
        studentArray[studentArray.length-1]=s1;
    }
    private static void incrementArrays(){
        Student[] tempStudentArray=new Student[studentArray.length+1];

        for(int i=0; i<studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    public static void printStudentsDetail(){
        System.out.println("ID\tName\tPRF\tDBMS");
        System.out.println("=====");
        for(int i=0; i<studentArray.length; i++){
            System.out.println(studentArray[i].toString());
        }
    }
    public static void removeStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student ID : ");
        String id=input.nextLine();
        int index=searchStudent(id);
        if(index>=0){
            for(int i=index; i<studentArray.length-1; i++){
                studentArray[i]=studentArray[i+1];
            }
            Student[] tempStudentArray=new Student[studentArray.length-1];
            for (int i = 0; i < tempStudentArray.length; i++){
                tempStudentArray[i]=studentArray[i];
            }
            studentArray=tempStudentArray;
        }
    }
}

```



```

        Scanner input=new Scanner(System.in);
        System.out.print("Input Student Id : ");
        String id=input.nextLine();
        System.out.print("Input Student Name : ");
        String name=input.nextLine();
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();

        incrementArrays();
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
        studentArray[studentArray.length-1]=s1;
    }
    private static void incrementArrays(){
        Student[] tempStudentArray=new Student[studentArray.length+1];

        for(int i=0; i<studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    public static void printStudentsDetail(){
        System.out.println("ID\tName\tPRF\tDBMS");
        System.out.println("=====");
        for(int i=0; i<studentArray.length; i++){
            System.out.println(studentArray[i].toString());
        }
    }
    public static void removeStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student ID : ");
        String id=input.nextLine();
        int index=searchStudent(id);
        if(index>=0){
            for(int i=index; i<studentArray.length-1; i++){
                studentArray[i]=studentArray[i+1];
            }
            Student[] tempStudentArray=new Student[studentArray.length-1];
            for (int i = 0; i < tempStudentArray.length; i++){
                tempStudentArray[i]=studentArray[i];
            }
            studentArray=tempStudentArray;
        }
    }
    public static int searchStudent(String id){
        for (int i = 0; i < studentArray.length; i++){
            if(studentArray[i].equals(id)){
                return i;
            }
        }
        return -1;
    }
}

```



```

        return (prfMarks+dbmsMarks)/2.0; //
    }
}
class Demo{
    public static Student[] studentArray=new Student[]{
        new Student("S001","Niroth",76,78),
        new Student("S002","Lakmal",96,88),
        new Student("S003","Nethuki",80,78),
    };

    public static void addStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student Id : ");
        String id=input.nextLine();
        System.out.print("Input Student Name : ");
        String name=input.nextLine();
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();

        incrementArrrays();
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
        studentArray[studentArray.length-1]=s1;
    }
    private static void incrementArrrays(){
        Student[] tempStudentArray=new Student[studentArray.length+1];

        for(int i=0; i<studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    public static void printStudentsDetail(){
        System.out.println("ID\tName\tPRF\tDBMS");
        System.out.println("=====");
        for(int i=0; i<studentArray.length; i++){
            System.out.println(studentArray[i].toString());
        }
    }
    public static void removeStudent(){
        Scanner input=new Scanner(System.in);
        System.out.print("Input Student ID : ");
        String id=input.nextLine();
        int index=searchStudent(id);
        if(index>=0){
            for(int i=index; i<studentArray.length-1; i++){
                studentArray[i]=studentArray[i+1];
            }
            Student[] tempStudentArray=new Student[studentArray.length-1];
            for (int i = 0; i < tempStudentArray.length; i++){
                tempStudentArray[i]=studentArray[i];
            }
        }
    }
}

```

```

        studentArray=tempStudentArray;
    }
}
public static int searchStudent(String id){
    for (int i = 0; i < studentArray.length; i++){
        if(studentArray[i].equals(id)){
            return i;
        }
    }
    return -1;
}
public static void updateMarks(){
    Scanner input=new Scanner(System.in);
    System.out.print("Input Student ID : ");
    String id=input.next();
    int index=searchStudent(id);
    if(index>=0){
        System.out.print("Input PRF Marks : ");
        int prfMarks=input.nextInt();
        System.out.print("Input DBMS Marks : ");
        int dbmsMarks=input.nextInt();
        studentArray[index].setPrfMarks(prfMarks);
        studentArray[index].setDbmsMarks(dbmsMarks);
    }
}
public static void printBestStudent(){
    Student[] tempStudentArray=new Student[studentArray.length];
    for (int i = 0; i < studentArray.length; i++){
        tempStudentArray[i]=studentArray[i];
    }
    //Sorting
    for (int i = 0; i < tempStudentArray.length-1; i++){
        for(int j=0; j<tempStudentArray.length-1; j++){
            if(tempStudentArray[j].average()<tempStudentArray[j+1].average()){
                Student tempStudent=tempStudentArray[j];
                tempStudentArray[j]=tempStudentArray[j+1];
                tempStudentArray[j+1]=tempStudent;
            }
        }
    }
    System.out.println("ID\tName\tPRF\tDBMS");
    System.out.println("=====");
    for(int i=0; i<studentArray.length; i++){
        System.out.println(tempStudentArray[i].toString());
    }
}
public static void main(String args[]){
    //addStudent();
    //addStudent();
    printStudentsDetail();

    //removeStudent();

```

////////////////////////////////////

Q69 Exercise

=====

////////////////////////////////////

Q70 From Q69

=====

Java Swing Applications



(1) Containers (JFrame, JWindow, JPanel.....)

(2) Controller (JButton, JTextField, JLabel,.....)

(3) Layout (FlowLayout, GridLayout, BorderLayout.....)

(4) Event handling (KeyEvent, MouseEvent, ActionListener)

```
=====
import javax.swing.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame();

    }
}
```

```
=====
import javax.swing.*;

class Demo{

    public static void main(String args[]){
        JFrame f1=new JFrame();
        f1.setVisible(true);

    }

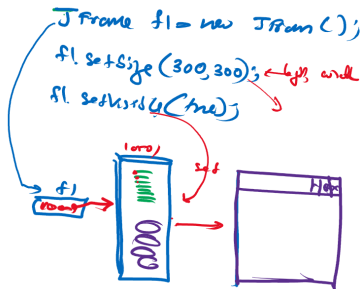
}
```

Diagram illustrating the visibility of a variable in a Java program:

- A variable `f` is declared with the value `100`.
- The variable `f` is assigned to a variable `f1` in a method.
- The variable `f1` is then passed to a function `f1.setVisible(true)`.
- The function `f1.setVisible(true)` is shown as a box with `f1` and `setVisible(true)` inside.
- The variable `f1` is also shown as a box with `f1` and `setVisible(true)` inside.
- The variable `f1` is then passed to a function `f1.setVisible(true)`.

Q73 setSize(300,300)

```
import javax.swing.*.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame();
        f1.setSize(300,300);
        f1.setVisible(true);
    }
}
```



Q74 setTitle("Calculator");

```
import javax.swing.*.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame();
        f1.setSize(300,300);
        f1.setTitle("Calculator");
        f1.setVisible(true);
    }
}
```

Q75

```
import javax.swing.*.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator"); //String->Title
        f1.setSize(300,300);
        //f1.setTitle("Calculator");
        f1.setVisible(true);
    }
}
```

Q76

```
import javax.swing.*.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator"); //String->Title
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setVisible(true);
    }
}
```

```

////////////////////////////////////
Q77
import javax.swing.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");    //String->Title
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        //f1.setDefaultCloseOperation();
        f1.setVisible(true);
        try{Thread.sleep(3000);}catch(Exception ex){}
        f1.setVisible(true);
    }
}

```

```

////////////////////////////////////
Q78 setDefaultCloseOperation(int)
=====
import javax.swing.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");    //String->Title
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f1.setVisible(true);
        try{Thread.sleep(3000);}catch(Exception ex){}
        f1.setVisible(true);
    }
}
//JFrame.DO_NOTHTNG_ON_CLOSE (Static Variables)->0
//JFrame.HIDE_ON_CLOSE (Static Variables) ->1
//JFrame.DISPOSE_ON_CLOSE (Static Variables)->2
//JFrame.EXIT_ON_CLOSE (Static Variables) ->3

```

```

////////////////////////////////////
Q79
import javax.swing.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator1");
        JFrame f2=new JFrame("Calculator2");
        f1.setSize(300,300);
        f2.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f2.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        f2.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        f1.setVisible(true);
        f2.setVisible(true);
    }
}

```

Q80 Adding controllers (JButton, JTextField, JLabel....)

JButton

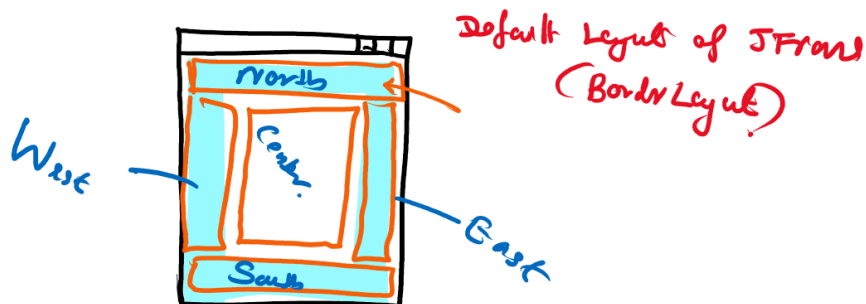
```
import javax.swing.*;
import java.awt.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JButton b1=new JButton();
        b1.setText("Hello");
        b1.setFont(new Font("",1,30));
        f1.add(b1);
        f1.setVisible(true);
    }
}
```

Q81

Layout manager



Q82

```
import javax.swing.*;
import java.awt.*;

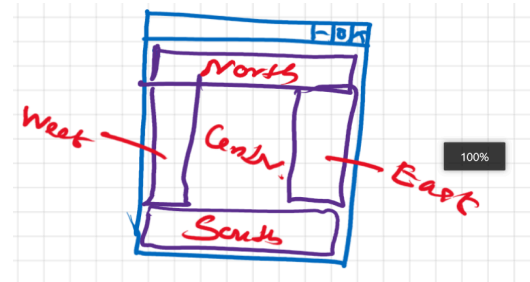
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JButton b1=new JButton();
        b1.setText("Hello");
        b1.setFont(new Font("",1,30));
        f1.add("South",b1);

        f1.setVisible(true);
    }
}
```

Q83 Exercise

=====



Q84 From Q83

=====

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JButton btNorth=new JButton("North"); //setText(String)
        JButton btSouth=new JButton("South");
        JButton btEast=new JButton("East");
        JButton btWest=new JButton("West");
        JButton btCenter=new JButton("Center");
        btNorth.setFont(new Font("",1,30));
        btSouth.setFont(new Font("",1,30));
        btEast.setFont(new Font("",1,30));
        btWest.setFont(new Font("",1,30));
        btCenter.setFont(new Font("",1,30));

        f1.add("South",btSouth);
        f1.add("North",btNorth);
        f1.add("East",btEast);
        f1.add("West",btWest);
        f1.add("Center",btCenter);
        f1.pack();
        f1.setVisible(true);
    }
}
```

Q85 From Q83 Using Button Arrays

```
import javax.swing.*;
import java.awt.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JButton[] btArray=new JButton[5];
        String[] buttonText={"North","South","East","West","Center"};
        for (int i = 0; i < 5; i++){
            btArray[i]=new JButton(buttonText[i]);
            btArray[i].setSize(300,400);
            btArray[i].setFont(new Font("",1,30));
            f1.add(buttonText[i],btArray[i]);
        }
        f1.pack();
        f1.setVisible(true);
    }
}
```

Q86 FlowLayout

```
import javax.swing.*;
import java.awt.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        f1.setLayout(new FlowLayout());

        JButton btAdd=new JButton("Add "); //setText(String)
        JButton btDelete=new JButton("Delete");
        JButton btUpdate=new JButton("Update");
        JButton btView=new JButton("View");
        btAdd.setFont(new Font("",1,30));
        btDelete.setFont(new Font("",1,30));
        btUpdate.setFont(new Font("",1,30));
        btView.setFont(new Font("",1,30));
        f1.add(btAdd);
        f1.add(btUpdate);
        f1.add(btDelete);
        f1.add(btView);

        f1.setVisible(true);
    }
}
```

////////////////////////////////////
Q87

```
import javax.swing.*;
import java.awt.*;
```

```
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        f1.setLayout(new FlowLayout(FlowLayout.RIGHT));
        //0->FlowLayout.CENTER
        //1->FlowLayout.RIGHT
        //2->FlowLayout.LEFT
        JButton btAdd=new JButton("Add "); //setText(String)
        JButton btDelete=new JButton("Delete");
        JButton btUpdate=new JButton("Update");
        JButton btView=new JButton("View");
        btAdd.setFont(new Font("",1,30));
        btDelete.setFont(new Font("",1,30));
        btUpdate.setFont(new Font("",1,30));
        btView.setFont(new Font("",1,30));

        f1.add(btAdd);
        f1.add(btUpdate);
        f1.add(btDelete);
        f1.add(btView);

        f1.setVisible(true);
    }
}
```

////////////////////////////////////
Q88 BorderLayout vs FlowLayout

=====

```
import javax.swing.*;
import java.awt.*;
```

```
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("BorderLayout");
        JFrame f2=new JFrame("FlowLayout");
        f1.setSize(300,300);
        f2.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f2.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        f2.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        f1.setLayout(new BorderLayout());//->Default BorderLayout
        f2.setLayout(new FlowLayout(FlowLayout.CENTER));
    }
}
```

////////////////////////////////////

=====



}

Q91 Exercise



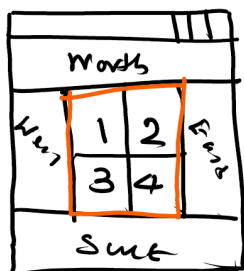
Q92 From Q91

```
import javax.swing.*;
import java.awt.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        f1.setLayout(new GridLayout(4,4,3,3));
        JButton[] btArray=new JButton[16];
        String[] buttonText={"7","8","9","*","4","5","6","/","1","2","3","+","0",".","=","-"};
        for (int i = 0; i < 16; i++){
            btArray[i]=new JButton(buttonText[i]);
            btArray[i].setFont(new Font("",1,25));
            f1.add(buttonText[i],btArray[i]);
        }
        f1.setVisible(true);
    }
}
```

Q93 Lightweight Container (JPanel)



```
Button bt1 = new JButton("1")
"      bt2 = new JButton("2")

JPanel container = new JPanel()
container.setLayout(new GridLayout(2,2));
container.add(bt1);
"      " (bt2)
"      " (bt3)
```


////////////////////////////////////
Q94 From Q93

=====

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
class Demo{
```

```
    public static void main(String args[]){
```

```
        JFrame f1=new JFrame("Calculator");
```

```
        f1.setSize(300,300);
```

```
        f1.setLocationRelativeTo(null);
```

```
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

```
        JButton btNorth=new JButton("North"); //setText(String)
```

```
        JButton btSouth=new JButton("South");
```

```
        JButton btEast=new JButton("East");
```

```
        JButton btWest=new JButton("West");
```

```
        JButton bt1=new JButton("1");
```

```
        JButton bt2=new JButton("2");
```

```
        JButton bt3=new JButton("3");
```

```
        JButton bt4=new JButton("4");
```

```
        bt1.setFont(new Font("",1,30));
```

```
        bt2.setFont(new Font("",1,30));
```

```
        bt3.setFont(new Font("",1,30));
```

```
        bt4.setFont(new Font("",1,30));
```

```
        JPanel centerPanel=new JPanel();
```

```
        centerPanel.setLayout(new GridLayout(2,2));
```

```
        centerPanel.add(bt1);
```

```
        centerPanel.add(bt2);
```

```
        centerPanel.add(bt3);
```

```
        centerPanel.add(bt4);
```

```
        btNorth.setFont(new Font("",1,30));
```

```
        btSouth.setFont(new Font("",1,30));
```

```
        btEast.setFont(new Font("",1,30));
```

```
        btWest.setFont(new Font("",1,30));
```

```
        f1.add("South",btSouth);
```

```
        f1.add("North",btNorth);
```

```
        f1.add("East",btEast);
```

```
        f1.add("West",btWest);
```

```
        f1.add(centerPanel);
```

```
        f1.pack();
```

```
        f1.setVisible(true);
```

```
    }
```

```
}
```

Q95 Exercise

=====



Q96 From Q95

=====

```
import javax.swing.*;
import java.awt.*;

class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("Calculator");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

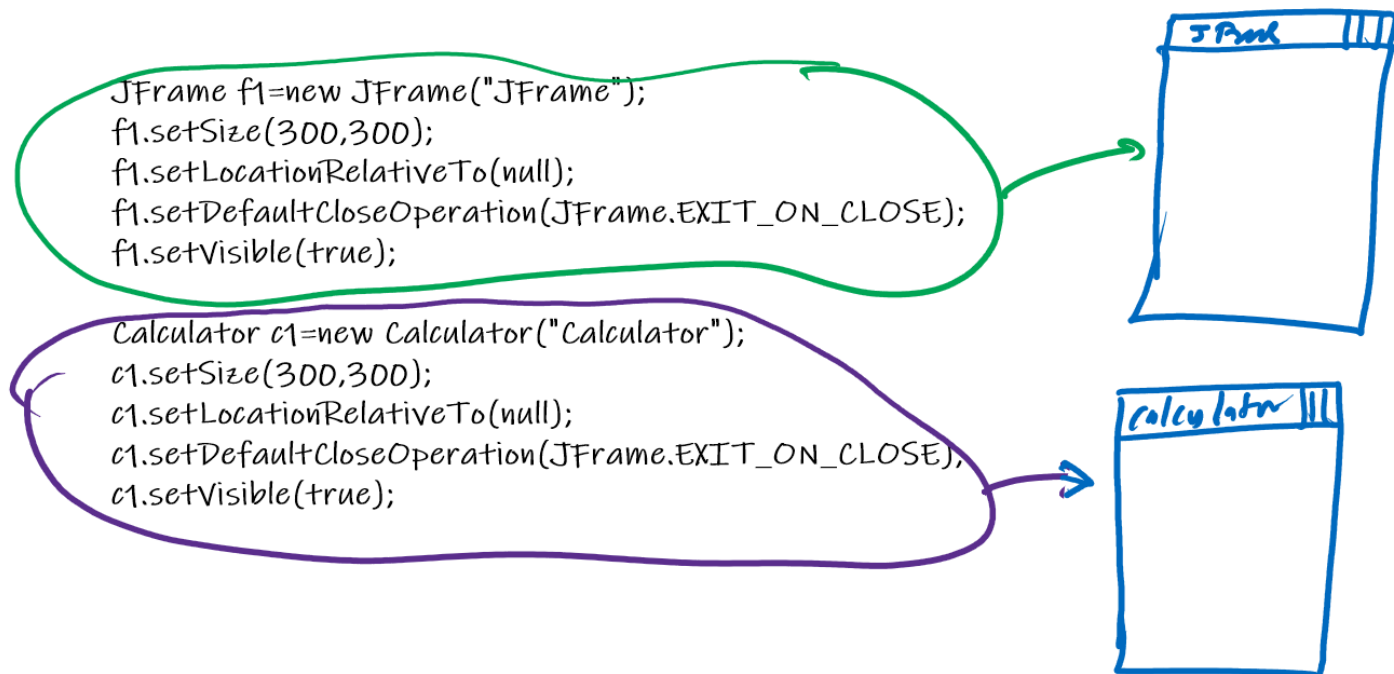
        JTextField txtDisplay=new JTextField();
        txtDisplay.setFont(new Font("",1,25));
        f1.add("North",txtDisplay);

        JPanel buttonPanel=new JPanel();
        buttonPanel.setLayout(new GridLayout(4,4,3,3));
        JButton[] btArray=new JButton[16];
        String[] buttonText={"7","8","9","*","4","5","6","/","1","2","3","+","0",".","=","-"};
        for (int i = 0; i < 16; i++){
            btArray[i]=new JButton(buttonText[i]);
            btArray[i].setFont(new Font("",1,25));
            buttonPanel.add(buttonText[i],btArray[i]);
        }
        f1.add("Center",buttonPanel);
        f1.setVisible(true);
    }
}
```

Q97

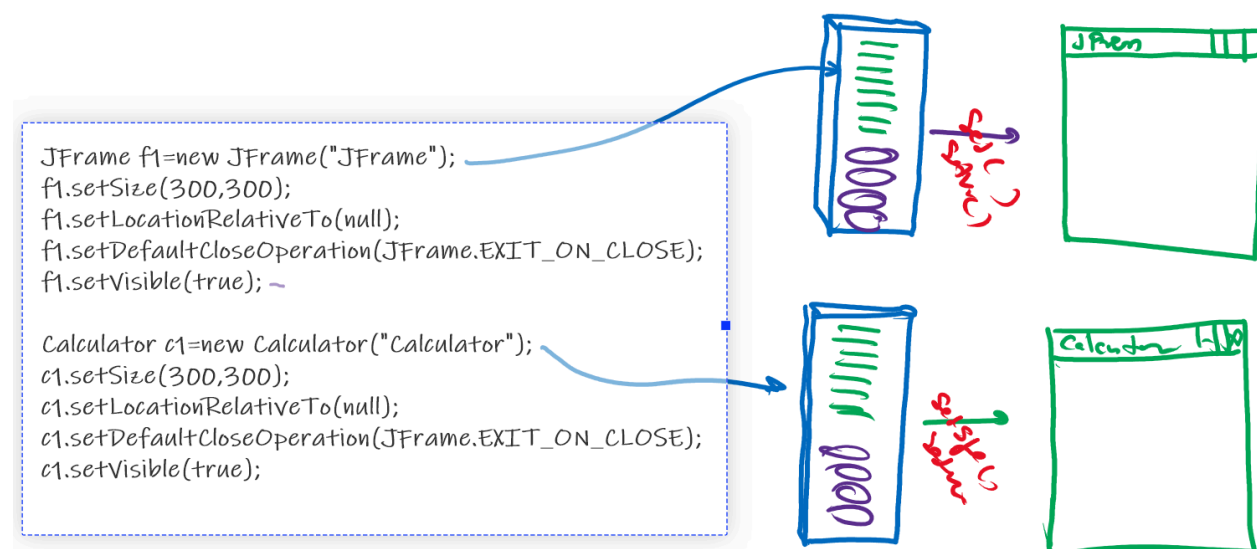
```
import javax.swing.*;
class Demo{
    public static void main(String args[]){
        JFrame f1=new JFrame("JFrame");
        f1.setSize(300,300);
        f1.setLocationRelativeTo(null);
        f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f1.setVisible(true);

        Calculator c1=new Calculator("Calculator");
        c1.setSize(300,300);
        c1.setLocationRelativeTo(null);
        c1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        c1.setVisible(true);
    }
}
```



Q98 From Q97

=====



```
////////////////////////////////////  
Q99 From Q98
```

```
=====
```

```
import javax.swing.*;  
class Calculator{  
    Calculator(String title){  
  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        JFrame f1=new JFrame("JFrame");  
        f1.setSize(300,300);  
        f1.setLocationRelativeTo(null);  
        f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        f1.setVisible(true);  
  
        Calculator c1=new Calculator("Calculator");  
        c1.setSize(300,300);  
        c1.setLocationRelativeTo(null);  
        c1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        c1.setVisible(true);  
  
    }  
}
```

```
////////////////////////////////////  
Q100 From Q99 Using Inheritance
```

```
=====
```

```
import javax.swing.*;  
class Calculator extends JFrame{  
    Calculator(String title){  
  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        JFrame f1=new JFrame("JFrame");  
        f1.setSize(300,300);  
        f1.setLocationRelativeTo(null);  
        f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        f1.setVisible(true);  
  
        Calculator c1=new Calculator("Calculator");  
        c1.setSize(300,300);  
        c1.setLocationRelativeTo(null);  
        c1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        c1.setVisible(true);  
  
    }  
}  
//Calculator ->Subclass  
//JFrame -->Super class
```



```
////////////////////////////////////  
Q103 From Q102
```

```
=====
```

```
import javax.swing.*;  
import java.awt.*;  
class Calculator extends JFrame{  
    private JButton btNorth;  
  
    Calculator(String title){  
        setTitle(title);  
        setSize(300,300);  
        setLocationRelativeTo(null);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        btNorth=new JButton("North");  
        btNorth.setFont(new Font("",1,25));  
        add("North",btNorth);  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        Calculator c1=new Calculator("Calculator");  
        c1.setVisible(true);  
    }  
}
```

```
////////////////////////////////////
```

Q104 Exercise From Q96 using Inheritance

```
=====
```

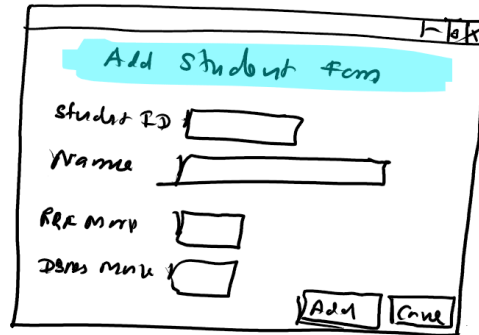
```
import javax.swing.*;  
import java.awt.*;  
class Calculator extends JFrame{  
    private JTextField txtDisplay;  
    private JButton[] btArray;  
    private String[] buttonText;  
  
    Calculator(String title){  
        setSize(300,300);  
        setLocationRelativeTo(null);  
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
        txtDisplay=new JTextField();  
        txtDisplay.setFont(new Font("",1,25));  
        add("North",txtDisplay);  
        JPanel buttonPanel=new JPanel();  
        buttonPanel.setLayout(new GridLayout(4,4,3,3));  
        btArray=new JButton[16];  
        buttonText=new String[]{"7","8","9","*","4","5","6","/",  
                                "1","2","3","+","0",".", "=", "-"};  
        for (int i = 0; i < 16; i++){  
            btArray[i]=new JButton(buttonText[i]);  
            btArray[i].setFont(new Font("",1,25));  
            buttonPanel.add(buttonText[i],btArray[i]);  
        }  
        add("Center",buttonPanel);  
    }  
}
```

```

class Demo{
    public static void main(String args[]){
        Calculator c1=new Calculator("Calculator");
        c1.setVisible(true);
    }
}

```

AddStudentForm



Q105 Step I

```

=====
import javax.swing.*;
import java.awt.*;

class AddStudentForm extends JFrame{
    private JTextField txtDisplay;

    AddStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    }
}

class Demo{
    public static void main(String args[]){
        new AddStudentForm().setVisible(true);
    }
}

```

Q106 Step II

```

=====
import javax.swing.*;
import java.awt.*;

class AddStudentForm extends JFrame{
    private JTextField txtDisplay;

    AddStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("Add Student Form");
    }
}

```

[illegible]

Q108 Step IV

=====

```
import javax.swing.*;
import java.awt.*;

class AddStudentForm extends JFrame{
    private JTextField txtDisplay;
    private JButton btCancel;
    private JButton btAdd;

    AddStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("Add Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btAdd=new JButton("Add Student");
        btCancel=new JButton("Cancel");
        btAdd.setFont(new Font("",1,20));
        btCancel.setFont(new Font("",1,20));

        southPanel.add(btAdd);
        southPanel.add(btCancel);
        add("South",southPanel);
        //-----

        JLabel lblStudentId=new JLabel("Student ID");
        JLabel lblName=new JLabel("Name");
        JLabel lblPrfMarks=new JLabel("Prf Marks");
        JLabel lblDbmsMarks=new JLabel("Dbms Marks");
        lblStudentId.setFont(new Font("",1,20));
        lblName.setFont(new Font("",1,20));
        lblPrfMarks.setFont(new Font("",1,20));
        lblDbmsMarks.setFont(new Font("",1,20));

        JPanel labelPanel=new JPanel(new GridLayout(4,1));
        labelPanel.add(lblStudentId);
        labelPanel.add(lblName);
        labelPanel.add(lblPrfMarks);
        labelPanel.add(lblDbmsMarks);
        add("West",labelPanel);
    }
}

class Demo{
    public static void main(String args[]){
        new AddStudentForm().setVisible(true);
    }
}
```

Q109 Step V

=====

```
import javax.swing.*;
import java.awt.*;

class AddStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btAdd;

    AddStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("Add Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btAdd=new JButton("Add Student");
        btCancel=new JButton("Cancel");
        btAdd.setFont(new Font("",1,20));
        btCancel.setFont(new Font("",1,20));

        southPanel.add(btAdd);
        southPanel.add(btCancel);
        add("South",southPanel);
        //-----

        JLabel lblStudentId=new JLabel("Student ID");
        JLabel lblName=new JLabel("Name");
        JLabel lblPrfMarks=new JLabel("Prf Marks");
        JLabel lblDbmsMarks=new JLabel("Dbms Marks");
        lblStudentId.setFont(new Font("",1,20));
        lblName.setFont(new Font("",1,20));
        lblPrfMarks.setFont(new Font("",1,20));
        lblDbmsMarks.setFont(new Font("",1,20));

        JPanel labelPanel=new JPanel(new GridLayout(4,1));
        labelPanel.add(lblStudentId);
        labelPanel.add(lblName);
        labelPanel.add(lblPrfMarks);
        labelPanel.add(lblDbmsMarks);
        add("West",labelPanel);
    }
}
```



```

JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
btAdd=new JButton("Add Student");
btCancel=new JButton("Cancel");
btAdd.setFont(new Font("",1,20));
btCancel.setFont(new Font("",1,20));

southPanel.add(btAdd);
southPanel.add(btCancel);
add("South",southPanel);
//-----

JLabel lblStudentId=new JLabel("Student ID");
JLabel lblName=new JLabel("Name");
JLabel lblPrfMarks=new JLabel("Prf Marks");
JLabel lblDbmsMarks=new JLabel("Dbms Marks");
lblStudentId.setFont(new Font("",1,20));
lblName.setFont(new Font("",1,20));
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));

JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);

//-----

txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanal.add(txtStudentId);
textPanel.add(idTextPanal);
textPanel.add(txtName);
textPanel.add(txtPrfMarks);
textPanel.add(txtDbmsMarks);
add("Center",textPanel);
}
}
class Demo{
    public static void main(String args[]){
        new AddStudentForm().setVisible(true);
    }
}

```

Q111 Step VII

=====

```
import javax.swing.*;
import java.awt.*;

class AddStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btAdd;

    AddStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("Add Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btAdd=new JButton("Add Student");
        btCancel=new JButton("Cancel");
        btAdd.setFont(new Font("",1,20));
        btCancel.setFont(new Font("",1,20));

        southPanel.add(btAdd);
        southPanel.add(btCancel);
        add("South",southPanel);
        //-----

        JLabel lblStudentId=new JLabel("Student ID");
        JLabel lblName=new JLabel("Name");
        JLabel lblPrfMarks=new JLabel("Prf Marks");
        JLabel lblDbmsMarks=new JLabel("Dbms Marks");
        lblStudentId.setFont(new Font("",1,20));
        lblName.setFont(new Font("",1,20));
        lblPrfMarks.setFont(new Font("",1,20));
        lblDbmsMarks.setFont(new Font("",1,20));

        JPanel labelPanel=new JPanel(new GridLayout(4,1));
        JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
        labelPanel.add(lblStudentId);
        labelPanel.add(lblName);
        labelPanel.add(lblPrfMarks);
        labelPanel.add(lblDbmsMarks);
        add("West",labelPanel);
```

```

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanel.add(txtStudentId);
textPanel.add(idTextPanel);

JPanel nameTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanel.add(txtName);
textPanel.add(nameTextPanel);

JPanel prfMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanel.add(txtPrfMarks);
textPanel.add(prfMarksTextPanel);

JPanel dbmsMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
dbmsMarksTextPanel.add(txtDbmsMarks);
textPanel.add(dbmsMarksTextPanel);
add("Center",textPanel);
}
}
class Demo{
    public static void main(String args[]){
        new AddStudentForm().setVisible(true);
    }
}

```

Event Handling

```

Q112
import javax.swing.*;
import java.awt.*;

class Calculator extends JFrame{
    private JButton btExit;

    Calculator(String title){
        setTitle(title);
        setSize(300,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        btExit=new JButton("Exit");
    }
}

```


Q114 ActionListener for JTextFields

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Calculator extends JFrame{
    private JButton btExit;
    private JTextField txtTitle;

    Calculator(String title){
        setTitle(title);
        setSize(300,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        btExit=new JButton("Exit");
        btExit.setFont(new Font("",1,25));
        btExit.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                System.out.println("Exited...");
                System.exit(0); //dispose();
            }
        });
        add(btExit);

        txtTitle=new JTextField(12);
        txtTitle.setFont(new Font("",1,25));
        txtTitle.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String title=txtTitle.getText();
                setTitle(title);
            }
        });
        add(txtTitle);
    }
}

class Demo{
    public static void main(String args[]){
        Calculator c1=new Calculator("Calculator");
        c1.setVisible(true);
    }
}
```

Q115 KeyListener(KeyAdapter) vs KeyListener

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Calculator extends JFrame{
    private JButton btExit;
    private JTextField txtTitle;
```


////////////////////////////////////

=====

[illegible]


```

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanel.add(txtStudentId);
textPanel.add(idTextPanel);

JPanel nameTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanel.add(txtName);
textPanel.add(nameTextPanel);

JPanel prfMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanel.add(txtPrfMarks);
textPanel.add(prfMarksTextPanel);

JPanel dbmsMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
dbmsMarksTextPanel.add(txtDbmsMarks);
textPanel.add(dbmsMarksTextPanel);
add("Center",textPanel);
}
}

```

```

//-----ViewStudentForm.java-----
import javax.swing.*;
import java.awt.*;

class ViewStudentDetailsForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btView;

    ViewStudentDetailsForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("View Student Details Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
    }
}

```

```
//-----
```

```
JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));  
btView=new JButton("Search Student");  
btCancel=new JButton("Cancel");  
btView.setFont(new Font("",1,20));  
btCancel.setFont(new Font("",1,20));
```

```
southPanel.add(btView);  
southPanel.add(btCancel);  
add("South",southPanel);
```

```
//-----
```

```
JLabel lblStudentId=new JLabel("Student ID");  
JLabel lblName=new JLabel("Name");  
JLabel lblPrfMarks=new JLabel("Prf Marks");  
JLabel lblDbmsMarks=new JLabel("Dbms Marks");  
lblStudentId.setFont(new Font("",1,20));  
lblName.setFont(new Font("",1,20));  
lblPrfMarks.setFont(new Font("",1,20));  
lblDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel labelPanel=new JPanel(new GridLayout(4,1));  
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
labelPanel.add(lblStudentId);  
labelPanel.add(lblName);  
labelPanel.add(lblPrfMarks);  
labelPanel.add(lblDbmsMarks);  
add("West",labelPanel);
```

```
//-----
```

```
txtStudentId=new JTextField(5);  
txtStudentId.setFont(new Font("",1,20));  
txtName=new JTextField(10);  
txtName.setFont(new Font("",1,20));  
txtPrfMarks=new JTextField(4);  
txtPrfMarks.setFont(new Font("",1,20));  
txtDbmsMarks=new JTextField(4);  
txtDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel textPanel=new JPanel(new GridLayout(4,1));
```

```
JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));  
idTextPanal.add(txtStudentId);  
textPanel.add(idTextPanel);
```

```
JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
nameTextPanal.add(txtName);  
textPanel.add(nameTextPanal);
```

```
JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
prfMarksTextPanal.add(txtPrfMarks);  
textPanel.add(prfMarksTextPanal);
```

```

        JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
        dbmsMarksTextPanal.add(txtDbmsMarks);
        textPanel.add(dbmsMarksTextPanal);
        add("Center",textPanel);
    }
}

```

//-----MainForm.java-----

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MainForm extends JFrame{
    private JButton btAddStudent;
    private JButton btViewStudent;
    private JButton btExit;

    MainForm(){
        setSize(500,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel titleLabel=new JLabel("Student' Marks Management System");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel buttonPanel=new JPanel(new GridLayout(3,1));
        btAddStudent=new JButton("Add Student Form");
        btAddStudent.setFont(new Font("",1,20));
        btAddStudent.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                new AddStudentForm().setVisible(true);
            }
        });
        buttonPanel.add(btAddStudent);

        btViewStudent=new JButton("View Student Form");
        btViewStudent.setFont(new Font("",1,20));
        buttonPanel.add(btViewStudent);

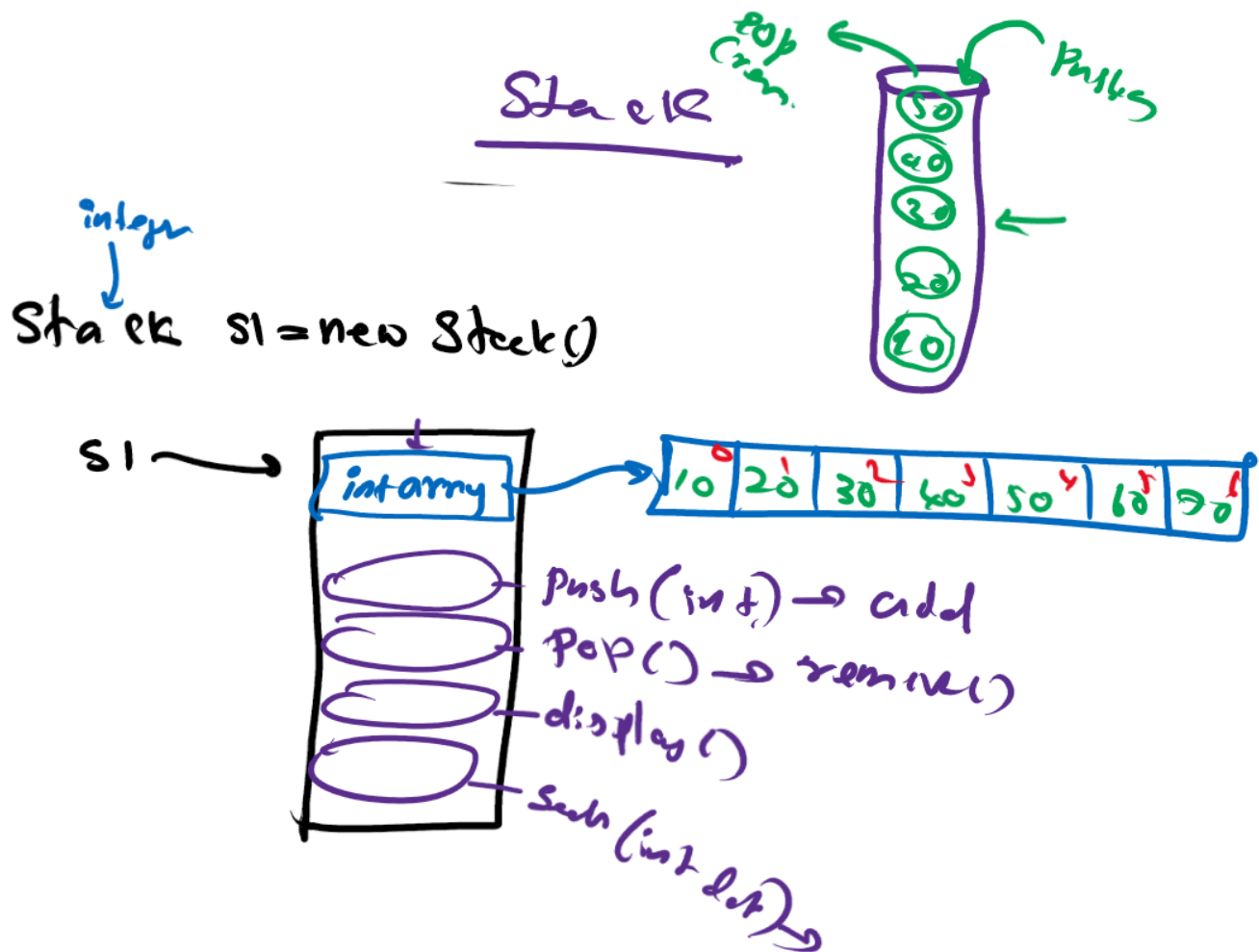
        btExit=new JButton("Exit");
        btExit.setFont(new Font("",1,20));
        buttonPanel.add(btExit);
        add("Center",buttonPanel);
    }

    public static void main(String args[]){
        new MainForm().setVisible(true);
    }
}

```

Data Structures and Algorithms 05-07-2026

Q120 class Stack (Last in First out)



Q121 Step I

```
class Stack{
    private int[] dataArray;

    public void push(int data){

    }
    public void pop(){

    }
    public int search(int data){
        return -1;
    }
    public void display(){

    }
}
```



```
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]
        s1.pop();
        s1.display(); //[40,30,20,10]
    }
}
```

Q122 Step II

```
class Stack{
    private int[] dataArray;
    Stack(){
        dataArray=new int[0];
    }
    private void extendsArray(){
        int[] tempDataArray=new int[dataArray.length+1];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public void push(int data){
        extendsArray();
        dataArray[dataArray.length-1]=data;
    }
    public void pop(){
    }
    public int search(int data){
        return -1;
    }
    public void display(){
    }
}
```

```
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]
        s1.pop();
        s1.display(); //[40,30,20,10]
    }
}
```

////////////////////////////////////
Q123 Step III - adding method display()
=====

```
class Stack{
    private int[] dataArray;

    Stack(){
        dataArray=new int[0];
    }
    private void extendsArray(){
        int[] tempDataArray=new int[dataArray.length+1];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public void push(int data){
        extendsArray();
        dataArray[dataArray.length-1]=data;
    }
    public void pop(){

    }
    public int search(int data){
        return -1;
    }
    public void display(){
        System.out.print("[");
        for(int i=dataArray.length-1; i>=0; i--){
            System.out.print(dataArray[i]+" ", "");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return dataArray.length<=0;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]

        s1.pop();
        s1.display(); //[40,30,20,10]
    }
}
```

////////////////////////////////////
Q124 Step IV(adding pop())

=====

```
class Stack{
    private int[] dataArray;

    Stack(){
        dataArray=new int[0];
    }
    private void extendsArray(){
        int[] tempDataArray=new int[dataArray.length+1];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public void push(int data){
        extendsArray();
        dataArray[dataArray.length-1]=data;
    }
    public void pop(){
        if(!isEmpty()){
            int[] tempDataArray=new int[dataArray.length-1];
            for (int i = 0; i < tempDataArray.length; i++){
                tempDataArray[i]=dataArray[i];
            }
            dataArray=tempDataArray;
        }
    }
    public int search(int data){
        return -1;
    }
    public void display(){
        System.out.print("[");
        for(int i=dataArray.length-1; i>=0; i--){
            System.out.print(dataArray[i]+" ", "");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return dataArray.length<=0;
    }
}

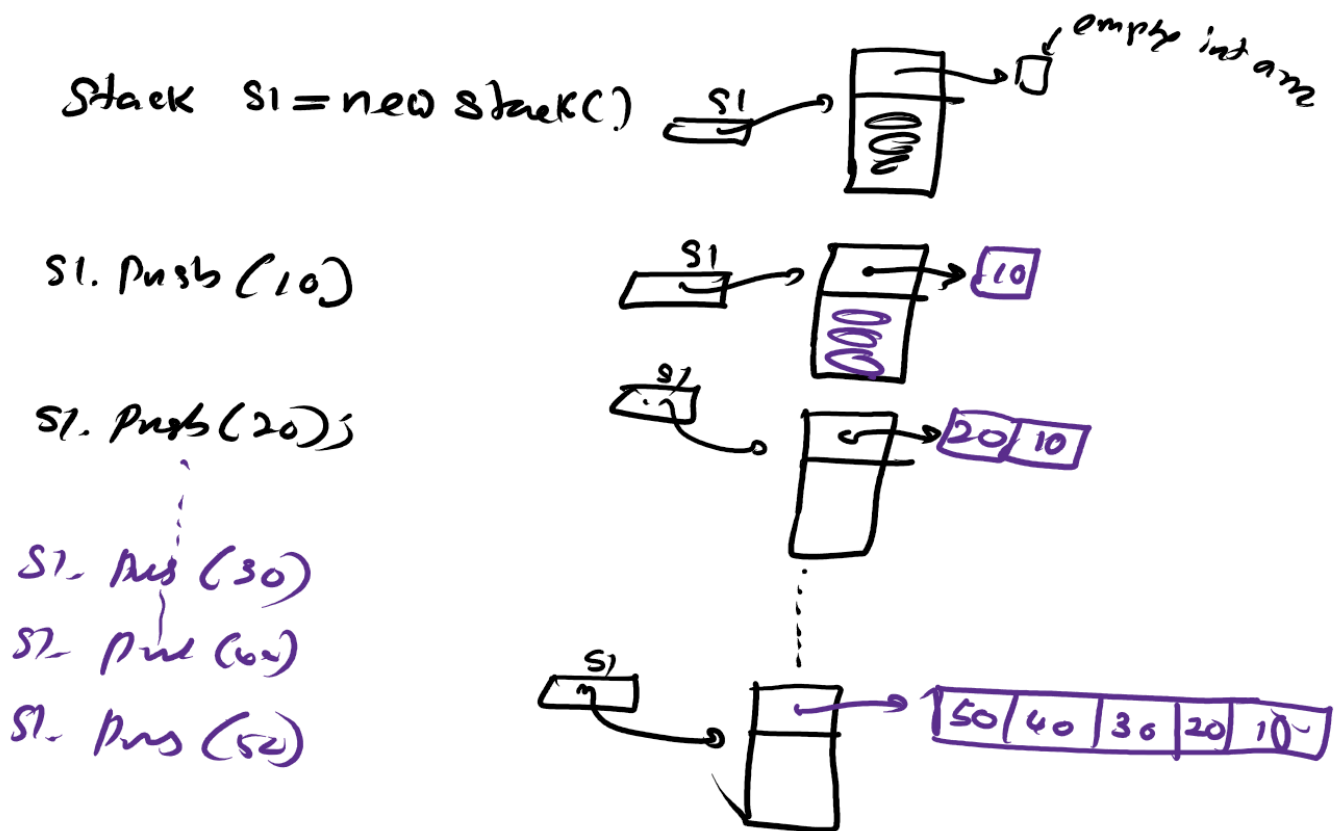
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]
```

```

        s1.pop();
        s1.display(); //[40,30,20,10]
    }
}

```

Q125 Exercise From Q125



Q126 From Q125

```

class Stack{
    private int[] dataArray;

    Stack(){
        dataArray=new int[0];
    }
    private void extendsArray(){
        int[] tempDataArray=new int[dataArray.length+1];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i+1]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public void push(int data){
        extendsArray();
        dataArray[0]=data;
    }
    public void pop(){
        if(!isEmpty()){
            int[] tempDataArray=new int[dataArray.length-1];
            for (int i = 0; i < tempDataArray.length; i++){

```

```
class Stack{
    private int[] dataArray;

    Stack(){
        dataArray=new int[0];
    }
    private void extendsArray(){
        int[] tempDataArray=new int[dataArray.length+1];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i+1]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public void push(int data){
        extendsArray();
        dataArray[0]=data;
    }
}
```

```

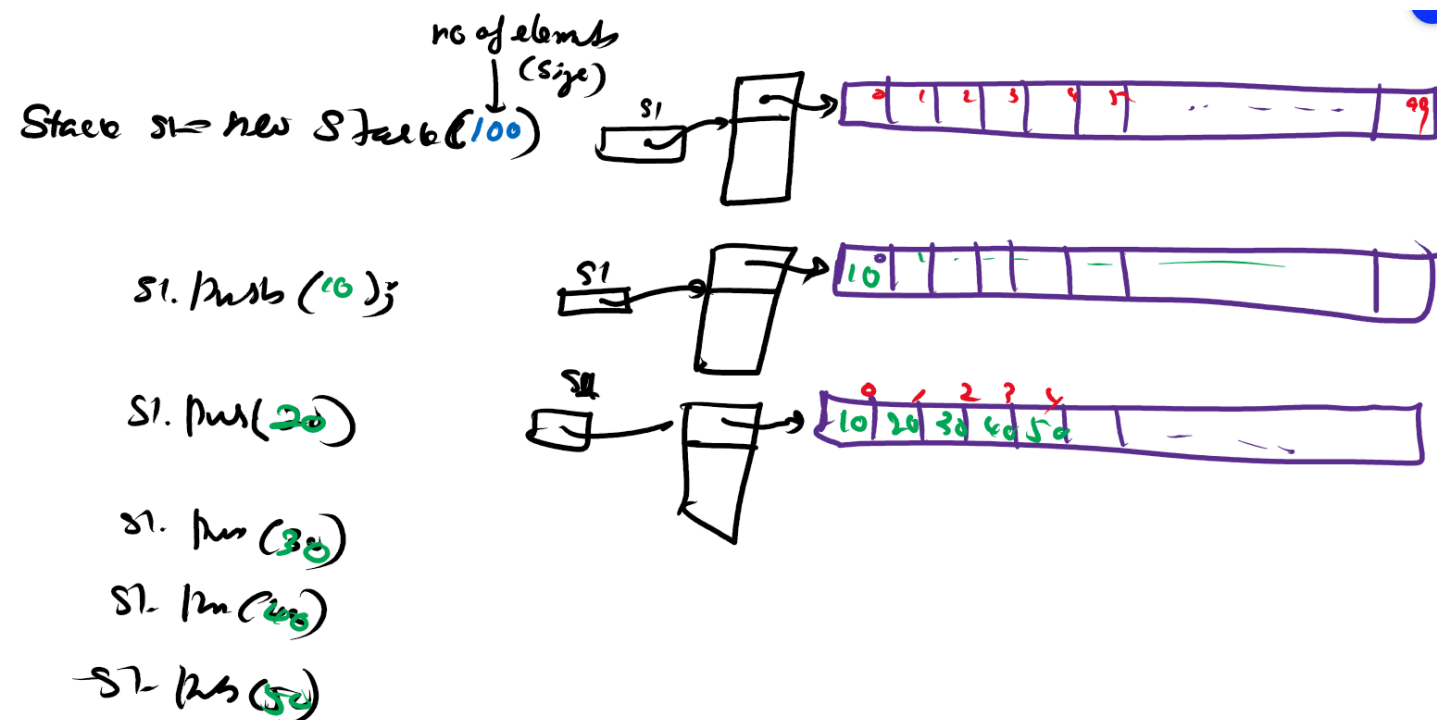
    }
    public void pop(){
        if(!isEmpty()){
            int[] tempDataArray=new int[dataArray.length-1];
            for (int i = 0; i < tempDataArray.length; i++){
                tempDataArray[i]=dataArray[i+1];
            }
            dataArray=tempDataArray;
        }
    }
    public int search(int data){
        for(int i=0; i<dataArray.length; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for(int i=0; i<dataArray.length;i++){
            System.out.print(dataArray[i]+" ", );
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return dataArray.length<=0;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]

        System.out.println("Index of 40 : "+s1.search(40));    //1
    }
}

```

=====



////////////////////////////////////

=====

```
class Stack{
    private int[] dataArray;
    private int nextIndex;
    Stack(int size){
        dataArray=new int[size];
        nextIndex=0;
    }
    private void extendsArray(){

    }
    public void push(int data){
        //
    }
    public void pop(){

    }
    public int search(int data){
        for(int i=0; i<dataArray.length; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        System.out.println(isEmpty()? "empty":"\b\b");
    }
}
```

```

        public boolean isEmpty(){
            return nextIndex<=0;
        }
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack(100);
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.display(); //[50,40,30,20,10]

        System.out.println("Index of 40 : "+s1.search(40));    //1

    }
}

```

////////////////////////////////////

Q130 From Q129

=====

```
class Stack{
    private int[] dataArray;
    private int nextIndex;
    Stack(int size){
        dataArray=new int[size];
        nextIndex=0;
    }
    private void extendsArray(){

    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void push(int data){
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void pop(){
        if(!isEmpty()){
            nextIndex--;
        }
    }
    public int search(int data){
        //Insert code here
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i =nextIndex-1; i>=0 ; i--){
```

```
private int[] dataArray;
```

```
private int nextIndex;
```

```
Stack(int size){
```

```
dataArray=new int[size];
```

```
nextIndex=0;
```

}

```
private void extendsArray(){
```

}

```
private boolean isFull(){
```

```
return nextIndex>=dataArray.length;
```

}

```
public void push(int data){
```

```
if(isFull()){
```

```
extendsArray();
```

}

```
dataArray[nextIndex++]=data;
```

}

```
public void pop(){
```

```
if(!isEmpty()){
```

```
nextIndex--;
```

}

}

```
public int search(int data){
```

```
//Insert code here
```

```
return -1;
```

}

```
public void display(){
```

```
System.out.print("[");
```

```
for (int i = nextIndex-1; i >= 0 ; i--){
```



```

        dataArray[nextIndex++]=data;
    }
    public void trimToSize(){
        int[] tempDataArray=new int[nextIndex];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
        nextIndex=0;
    }
    public void pop(){
        if(!isEmpty()){
            nextIndex--;
        }
    }
    public int search(int data){
        //Insert code here
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i =nextIndex-1; i>=0 ; i--){
            System.out.print(dataArray[i]+" ", "");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack(12,0.25);
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.push(60);
        s1.push(70);
    }
}

```

////////////////////////////////////

=====

```

    public void trimToSize(){
        int[] tempDataArray=new int[size()];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
        nextIndex=0;
    }
    public void pop(){
        if(!isEmpty()){
            nextIndex--;
        }
    }
    public int search(int data){
        //Insert code here
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i =nextIndex-1; i>=0 ; i--){
            System.out.print(dataArray[i]+" , ");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack(12,0.25);
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.push(60);
        s1.push(70);
        s1.push(80);
        s1.push(90);
    }
}

```

```
int topData;
topData=s1.peak();
System.out.println("Top Data : "+topData); //120
System.out.println("After calling peak()...");
s1.display(); //[120,110,100,90...50,40,30,20,10]

topData=s1.poll();
System.out.println("Top Data : "+topData); //120
System.out.println("After calling poll()...");
s1.display(); //[110,100,90...50,40,30,20,10]

}

}
```

Q133 From Q132

```
class Stack{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    Stack(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public int peek(){
        return isEmpty() ? -1: dataArray[nextIndex-1];
    }
    public int poll(){
        return isEmpty() ? -1: dataArray[--nextIndex];
    }
    public void push(int data){
        if(isFull()){
```

```

        extendsArray();
    }
    dataArray[nextIndex++]=data;
}
public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void pop(){
    if(!isEmpty()){
        nextIndex--;
    }
}
public int search(int data){
    //Insert code here
    return -1;
}
public void display(){
    System.out.print("[");
    for (int i =nextIndex-1; i>=0 ; i--){
        System.out.print(dataArray[i]+" , ");
    }
    System.out.println(isEmpty()? "empty":"\b\b");
}
public boolean isEmpty(){
    return nextIndex<=0;
}
public int capacity(){
    return dataArray.length;
}
}
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack(12,0.25);
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
    }
}

```


Q134 From Q133

=====

```
class Queue{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    Queue(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public int peek(){
        return isEmpty() ? -1: dataArray[nextIndex-1];
    }
    public int poll(){
        return isEmpty() ? -1: dataArray[--nextIndex];
    }
    public void add(int data){
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void trimToSize(){
        int[] tempDataArray=new int[size()];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
    }
}
```



```

        nextIndex=0;
    }
    public void remove(){
        if(!isEmpty()){

        }
    }
    public int search(int data){
        //Insert code here
        return -1;
    }
    public void display(){
        System.out.print("[");

        System.out.println(isEmpty()? "empty]":"\b\b]");
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}
class Demo{
    public static void main(String args[]){
        Queue q1=new Queue(12,0.25);
        q1.add(10);
        q1.add(20);
        q1.add(30);
        q1.add(40);
        q1.add(50);
        q1.add(60);
        q1.add(70);
        q1.add(80);
        q1.add(90);
        q1.add(100);
        q1.add(110);
        q1.add(120);
        q1.display(); //[10,20,30,40,50 .....120]
        System.out.println("Size    : "+q1.size()); //12

        q1.remove();
        q1.display(); //[20,30,40,50 .....120]
        System.out.println("Size    : "+q1.size()); //11

        System.out.println("Index of 50 : "+q1.search(50)); //3
    }
}

```

////////////////////////////////////
Q135 From Q134

=====

```
class Queue{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    Queue(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public int peek(){
        return isEmpty() ? -1: dataArray[0];
    }
    public int poll(){
        if(!isEmpty()){
            int firstData=dataArray[0];
            remove();
            return firstData;
        }
        return -1;
    }
    public void add(int data){
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void trimToSize(){
        int[] tempDataArray=new int[size()];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
```

```

        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
        nextIndex=0;
    }
    public void remove(){
        if(!isEmpty()){
            for (int i = 0; i <nextIndex-1; i++){
                dataArray[i]=dataArray[i+1];
            }
            nextIndex--;
        }
    }
    public int search(int data){
        for (int i = 0; i < nextIndex; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(dataArray[i]+" ", );
        }

        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        Queue q1=new Queue(12,0.25);
        q1.add(10);
        q1.add(20);
        q1.add(30);
        q1.add(40);
        q1.add(50);
        q1.add(60);
        q1.add(70);
        q1.add(80);
        q1.add(90);
        q1.add(100);
        q1.add(110);
        q1.add(120);
    }
}

```

```
private void extendsArray(){
    int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
    for (int i = 0; i < dataArray.length; i++){
```

```

        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
private boolean isFull(){
    return nextIndex>=dataArray.length;
}
public void add(int data){
    if(isFull()){
        extendsArray();
    }
    dataArray[nextIndex++]=data;
}
public void add(int index,int data){
    if(isFull()){
        extendsArray();
    }
    dataArray[nextIndex++]=data;
}
public void addFirst(int data){
    if(isFull()){
        extendsArray();
    }
}
public void addLast(int data){
    if(isFull()){
        extendsArray();
    }
}
public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        //Insert code here
        nextIndex--;
    }
}
}

```

```

public void removeFirst(){
    if(!isEmpty()){
        //Insert code here
        nextIndex--;
    }
}
public void removeLast(){
    if(!isEmpty()){
        //Insert code here
        nextIndex--;
    }
}
public int search(int data){
    for (int i = 0; i < nextIndex; i++){
        if(dataArray[i]==data){
            return i;
        }
    }
    return -1;
}
public void display(){
    System.out.print("[");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(dataArray[i]+", ");
    }
    System.out.println(isEmpty()? "empty":"\b\b");
}
public void display(int start){

}
public void display(int startIndex, int endIndex){

}
public int get(int index){
    return -1;
}
public int getFirst(){
    return -1;
}
public int getLast(){
    return -1;
}
public boolean isEmpty(){
    return nextIndex<=0;
}
public int capacity(){
    return dataArray.length;
}
}

```

////////////////////////////////////

Q137 Adding Method (public void add(index, data))

=====

```
class List{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    List(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
    }
}
```

```

    }
    dataArray[nextIndex++]=data;
}
public void add(int index,int data){
    if(index>=0 && index<=nextIndex){
        for (int i = nextIndex-1; i >=index; i--){
            dataArray[i+1]=dataArray[i];
        }
        dataArray[index]=data;
        nextIndex++;
    }
}
public void addFirst(int data){
    if(isFull()){
        extendsArray();
    }
}
public void addLast(int data){
    if(isFull()){
        extendsArray();
    }
}
public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        //Insert code here
        nextIndex--;
    }
}
public void removeFirst(){
    if(!isEmpty()){
        //Insert code here
        nextIndex--;
    }
}
public void removeLast(){

```



```

        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public int search(int data){
        for (int i = 0; i < nextIndex; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(dataArray[i]+" ", "");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public void display(int start){

    }
    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
    }
}

```

```
list1.add(90);
list1.display(); //[10,20,30,40,50 .....90]
```

```
list1.add(3,35);
list1.display(); //[10,20,30,35,40 .....90]
```

```
    }
}
```

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Q138 From Q137

```
=====
```

```
class List{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    List(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void add(int index,int data){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                dataArray[i+1]=dataArray[i];
            }
            dataArray[index]=data;
            nextIndex++;
        }
    }
    public void addFirst(int data){
```

```

        add(0,data);
    }
    public void addLast(int data){
        add(nextIndex,data);
    }
    public void trimToSize(){
        int[] tempDataArray=new int[size()];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
        nextIndex=0;
    }
    public void remove(int index){
        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public void removeFirst(){
        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public void removeLast(){
        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public int search(int data){
        for (int i = 0; i < nextIndex; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(dataArray[i]+" , ");
        }
    }

```

```

        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public void display(int start){

    }
    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
        list1.add(90);
        list1.display(); //[10,20,30,40,50 .....90]

        list1.addFirst(9);
        list1.addLast(100);
        list1.display(); //[9,10,20,30,35,40 .....90,100]

    }
}

```

////////////////////////////////////
Q139 Adding method (remove(index), removeFirst(), removeLast())
=====

```
class List{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    List(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void add(int index,int data){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                dataArray[i+1]=dataArray[i];
            }
            dataArray[index]=data;
            nextIndex++;
        }
    }
    public void addFirst(int data){
        add(0,data);
    }
    public void addLast(int data){
        add(nextIndex,data);
    }
    public void trimToSize(){
        int[] tempDataArray=new int[size()];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
    }
}
```

```

        }
        dataArray=tempDataArray;
    }
    public int size(){
        return nextIndex;
    }
    public void trim(){
        //
        //
    }
    public void clear(){
        dataArray=new int[initSize];
        nextIndex=0;
    }
    public void remove(int index){
        if(!isEmpty()){
            if(index>=0 && index<nextIndex){
                for(int i=index; i<nextIndex-1; i++){
                    dataArray[i]=dataArray[i+1];
                }
            }
            nextIndex--;
        }
    }
    public void removeFirst(){
        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public void removeLast(){
        if(!isEmpty()){
            //Insert code here
            nextIndex--;
        }
    }
    public int search(int data){
        for (int i = 0; i < nextIndex; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(dataArray[i]+" ", "");
        }
        System.out.println(isEmpty()? "empty]":"\b\b]");
    }
    public void display(int start){

```

```

    }
    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}
class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
        list1.add(90);
        list1.display(); //[10,20,30,40,50 .....90]

        list1.remove(3);
        list1.display(); //[10,20,30, .....90,100]

        list1.remove(0);
        list1.display(); //[10,20,30, .....90,100]

    }
}

```

////////////////////////////////////
Q140 removeFirst() and removeLast()
=====

```
class List{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    List(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void add(int index,int data){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                dataArray[i+1]=dataArray[i];
            }
            dataArray[index]=data;
            nextIndex++;
        }
    }
    public void addFirst(int data){
        add(0,data);
    }
    public void addLast(int data){
        add(nextIndex,data);
    }
}
```



```

public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        if(index>=0 && index<nextIndex){
            for(int i=index; i<nextIndex-1; i++){
                dataArray[i]=dataArray[i+1];
            }
        }
        nextIndex--;
    }
}
public void removeFirst(){
    remove(0);
}
public void removeLast(){
    remove(size()-1);
}
public int search(int data){
    for (int i = 0; i < nextIndex; i++){
        if(dataArray[i]==data){
            return i;
        }
    }
    return -1;
}
public void display(){
    System.out.print("[");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(dataArray[i]+" ", "");
    }
    System.out.println(isEmpty()? "empty]":"\b\b]");
}
public void display(int start){
}

```

```

    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
        list1.add(90);
        list1.display(); //[10,20,30,40,50 .....90]

        list1.removeFirst();
        list1.removeLast();
        list1.display(); //[20,30, .....90]

    }
}

```

Q141

```
class List{
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    List(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void add(int index,int data){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                dataArray[i+1]=dataArray[i];
            }
            dataArray[index]=data;
            nextIndex++;
        }
    }
    public void addFirst(int data){
        add(0,data);
    }
    public void addLast(int data){
        add(nextIndex,data);
    }
    public void add(int...intData){
    }
    public void add(int index, int...intData){
        //
    }
}
```

```

public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        if(index>=0 && index<nextIndex){
            for(int i=index; i<nextIndex-1; i++){
                dataArray[i]=dataArray[i+1];
            }
        }
        nextIndex--;
    }
}
public void removeFirst(){
    remove(0);
}
public void removeLast(){
    remove(size()-1);
}
public void remove(int startIndex, int endIndex){
    //
}
public int search(int data){
    for (int i = 0; i < nextIndex; i++){
        if(dataArray[i]==data){
            return i;
        }
    }
    return -1;
}
public void display(){
    System.out.print("[");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(dataArray[i]+" ", );
    }
    System.out.println(isEmpty()? "empty]":"\b\b]");
}
public void display(int start){

```

```

    }
    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
        list1.add(90);
        list1.display(); //[10,20,30,40,50 .....90]

        list1.removeFirst();
        list1.removeLast();
        list1.display(); //[20,30, .....90]

    }
}

```

Q142 StudentList

=====

```
class Student{
    String id;
    String name;
    int prfMarks;
    int dbmsMarks;

    Student(){

    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
}

class StudentList{ //Collection of student objects
    private int[] dataArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        dataArray=new int[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        int tempDataArray[]=new int[dataArray.length+(int)(loadFact*dataArray.length)];
        for (int i = 0; i < dataArray.length; i++){
            tempDataArray[i]=dataArray[i];
        }
        dataArray=tempDataArray;
    }
    private boolean isFull(){
        return nextIndex>=dataArray.length;
    }
    public void add(int data){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        dataArray[nextIndex++]=data;
    }
    public void add(int index,int data){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
```

```

        dataArray[i+1]=dataArray[i];
    }
    dataArray[index]=data;
    nextIndex++;
}
}
public void addFirst(int data){
    add(0,data);
}
public void addLast(int data){
    add(nextIndex,data);
}
public void add(int...intData){
    //
}
public void add(int index, int...intData){
    //
}
public void trimToSize(){
    int[] tempDataArray=new int[size()];
    for (int i = 0; i < dataArray.length; i++){
        tempDataArray[i]=dataArray[i];
    }
    dataArray=tempDataArray;
}
public int size(){
    return nextIndex;
}
public void trim(){
    //
    //
}
public void clear(){
    dataArray=new int[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        if(index>=0 && index<nextIndex){
            for(int i=index; i<nextIndex-1; i++){
                dataArray[i]=dataArray[i+1];
            }
        }
        nextIndex--;
    }
}
}
public void removeFirst(){
    remove(0);
}
public void removeLast(){
    remove(size()-1);
}
public void remove(int startIndex, int endIndex){

```

```

        //
    }
    public int search(int data){
        for (int i = 0; i < nextIndex; i++){
            if(dataArray[i]==data){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("[");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(dataArray[i]+", ");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public void display(int start){

    }
    public void display(int startIndex, int endIndex){

    }
    public int get(int index){
        return -1;
    }
    public int getFirst(){
        return -1;
    }
    public int getLast(){
        return -1;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return dataArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List(12,0.25);
        list1.add(10); //Insertion order
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.add(60);
        list1.add(70);
        list1.add(80);
        list1.add(90);
        list1.display(); //[10,20,30,40,50 .....90]
    }
}

```



```

        list1.removeFirst();
        list1.removeLast();
        list1.display(); //[20,30, .....90]

    }

}

////////////////////////////////////
Q143 From Q142
=====
class Student{
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    Student(){
    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
}

class StudentList{ //Collection of student objects
    private Student[] studentArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        studentArray=new Student[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        Student tempStudentArray[]=new Student[studentArray.length+(int)(loadFact*studentArray.length)];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    private boolean isFull(){
        return nextIndex>=studentArray.length;
    }
    public void add(Student student){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        studentArray[nextIndex++]=student;
    }
}

```

```

public void add(int index,Student student){
    if(isFull()){
        extendsArray();
    }
    if(index>=0 && index<=nextIndex){
        for (int i = nextIndex-1; i >=index; i--){
            studentArray[i+1]=studentArray[i];
        }
        studentArray[index]=student;
        nextIndex++;
    }
}

public void addFirst(Student student){
    add(0,student);
}

public void addLast(Student student){
    add(nextIndex,student);
}

public void trimToSize(){
    Student[] tempStudentArray=new Student[size()];
    for (int i = 0; i < studentArray.length; i++){
        tempStudentArray[i]=studentArray[i];
    }
    studentArray=tempStudentArray;
}

public int size(){
    return nextIndex;
}

public void trim(){
    //
}

public void clear(){
    studentArray=new Student[initSize];
    nextIndex=0;
}

public void remove(int index){
    if(!isEmpty()){
        if(index>=0 && index<nextIndex){
            for(int i=index; i<nextIndex-1; i++){
                studentArray[i]=studentArray[i+1];
            }
        }
        nextIndex--;
    }
}

public void removeFirst(){
    remove(0);
}

public void removeLast(){
    remove(size()-1);
}

```

```

public int search(Student student){
    for (int i = 0; i < nextIndex; i++){
        if(studentArray[i]==student){
            return i;
        }
    }
    return -1;
}

public void display(){
    System.out.print("[");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(studentArray[i]+" ");
    }
    System.out.println(isEmpty()? "empty":"\b\b");
}

public void display(int start){

}

public void display(int startIndex, int endIndex){

}

public Student get(int index){
    return null;
}

public Student getFirst(){
    return null;
}

public Student getLast(){
    return null;
}

public boolean isEmpty(){
    return nextIndex<=0;
}

public int capacity(){
    return studentArray.length;
}

}

class Demo{
    public static void main(String args[]){
        StudentList stList=new StudentList(12,0.25);
        stList.add(new Student("S0001","Nimal",65,67));
        stList.add(new Student("S0002","Amal",85,70));
        stList.add(new Student("S0003","Bimal",35,30));
        stList.add(new Student("S0004","Ramal",55,70));
        stList.add(new Student("S0005","Anil",95,90));
        stList.display(); //{[S0001,Nimal,65,76], [S0002,"Amal",85,70]...}
    }
}

```

////////////////////////////////////
Q144 From Q143 (search(Student)

=====

```
class Student{
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    Student(){

    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
    public String toString(){
        return "["+id+", "+name+", "+prfMarks+", "+dbmsMarks+"]";
    }
}

class StudentList{ //Collection of student objects
    private Student[] studentArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        studentArray=new Student[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        Student tempStudentArray[]=new Student[studentArray.length+(int)(loadFact*studentArray.length)];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    private boolean isFull(){
        return nextIndex>=studentArray.length;
    }
    public void add(Student student){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        studentArray[nextIndex++]=student;
    }
    public void add(int index,Student student){
        if(isFull()){
            extendsArray();
        }
    }
}
```

```

        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                studentArray[i+1]=studentArray[i];
            }
            studentArray[index]=student;
            nextIndex++;
        }
    }

    public void addFirst(Student student){
        add(0,student);
    }

    public void addLast(Student student){
        add(nextIndex,student);
    }

    public void trimToSize(){
        Student[] tempStudentArray=new Student[size()];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }

    public int size(){
        return nextIndex;
    }

    public void trim(){
        //
    }

    public void clear(){
        studentArray=new Student[initSize];
        nextIndex=0;
    }

    public void remove(int index){
        if(!isEmpty()){
            if(index>=0 && index<nextIndex){
                for(int i=index; i<nextIndex-1; i++){
                    studentArray[i]=studentArray[i+1];
                }
            }
            nextIndex--;
        }
    }

    public void removeFirst(){
        remove(0);
    }

    public void removeLast(){
        remove(size()-1);
    }

    public int search(Student student){
        for (int i = 0; i < nextIndex; i++){
            if(studentArray[i]==student){
                return i;
            }
        }
    }

```

```

    }
    return -1;
}
public void display(){
    System.out.print("{");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(studentArray[i].toString()+" ", );
    }
    System.out.println(isEmpty()? "empty":"\b\b");
}
public void display(int start){

}
public void display(int startIndex, int endIndex){

}
public Student get(int index){
    return null;
}
public Student getFirst(){
    return null;
}
public Student getLast(){
    return null;
}
public boolean isEmpty(){
    return nextIndex<=0;
}
public int capacity(){
    return studentArray.length;
}
}
class Demo{
    public static void main(String args[]){
        StudentList stList=new StudentList(12,0.25);
        stList.add(new Student("S0001","Nimal",65,67));
        stList.add(new Student("S0002","Amal",85,70));
        stList.add(new Student("S0003","Bimal",35,30));
        stList.add(new Student("S0004","Ramal",55,70));
        stList.add(new Student("S0005","Anil",95,90));
        stList.display(); //{S0001,Nimal,65,76}, [S0002,"Amal",85,70]...}

        System.out.println("Index of S003-Bimal : "+stList.search(new Student("S0003","Bimal",35,30))); //2
    }
}

```

////////////////////////////////////
Q145 From Q144

=====

```
class Student{
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    Student(){

    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
    public String getId(){
        return id;
    }
    public boolean equals(Student student){
        return id.equalsIgnoreCase(student.id);
    }
    public String toString(){
        return "["+id+", "+name+", "+prfMarks+", "+dbmsMarks+"]";
    }
}

class StudentList{ //Collection of student objects
    private Student[] studentArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        studentArray=new Student[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        Student tempStudentArray[]=new Student[studentArray.length+(int)(loadFact*studentArray.length)];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    private boolean isFull(){
        return nextIndex>=studentArray.length;
    }
}
```

```

public void add(Student student){ //Insertion order
    if(isFull()){
        extendsArray();
    }
    studentArray[nextIndex++]=student;
}
public void add(int index,Student student){
    if(isFull()){
        extendsArray();
    }
    if(index>=0 && index<=nextIndex){
        for (int i = nextIndex-1; i >=index; i--){
            studentArray[i+1]=studentArray[i];
        }
        studentArray[index]=student;
        nextIndex++;
    }
}
public void addFirst(Student student){
    add(0,student);
}
public void addLast(Student student){
    add(nextIndex,student);
}
public void trimToSize(){
    Student[] tempStudentArray=new Student[size()];
    for (int i = 0; i < studentArray.length; i++){
        tempStudentArray[i]=studentArray[i];
    }
    studentArray=tempStudentArray;
}
public int size(){
    return nextIndex;
}

public void clear(){
    studentArray=new Student[initSize];
    nextIndex=0;
}
public void remove(int index){
    if(!isEmpty()){
        if(index>=0 && index<nextIndex){
            for(int i=index; i<nextIndex-1; i++){
                studentArray[i]=studentArray[i+1];
            }
        }
        nextIndex--;
    }
}
public void removeFirst(){
    remove(0);
}

```



```

    public void removeLast(){
        remove(size()-1);
    }
    public int search(Student student){
        for (int i = 0; i < nextIndex; i++){
            if(student.equals(studentArray[i])){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("{}");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(studentArray[i].toString()+" ");
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public void display(int start){

    }
    public void display(int startIndex, int endIndex){

    }
    public Student get(int index){
        return null;
    }
    public Student getFirst(){
        return null;
    }
    public Student getLast(){
        return null;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public int capacity(){
        return studentArray.length;
    }
}

class Demo{
    public static void main(String args[]){
        StudentList stList=new StudentList(12,0.25);
        stList.add(new Student("S0001","Nimal",65,67));
        stList.add(new Student("S0002","Amal",85,70));
        stList.add(new Student("S0003","Bimal",35,30));
        stList.add(new Student("S0004","Ramal",55,70));
        stList.add(new Student("S0005","Anil",95,90));
        stList.display(); //{[S0001,Nimal,65,76], [S0002,"Amal",85,70]...}

        System.out.println("Index of S003-Bimal : "+stList.search(new Student("S0003","Bimal",35,30))); //{2

    }
}

```

////////////////////////////////////
Q146 Add a new method "public boolean remove(Student student)"
=====

```
class Student{
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    Student(){

    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
    public String getId(){
        return id;
    }
    public boolean equals(Student student){
        return id.equalsIgnoreCase(student.id);
    }
    public String toString(){
        return "["+id+", "+name+", "+prfMarks+", "+dbmsMarks+"]";
    }
}

class StudentList{ //Collection of student objects
    private Student[] studentArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        studentArray=new Student[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
        this.initSize=initSize;
    }
    private void extendsArray(){
        Student tempStudentArray[]=new Student[studentArray.length+(int)(loadFact*studentArray.length)];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    private boolean isFull(){
        return nextIndex>=studentArray.length;
    }
    public void add(Student student){ //Insertion order
        if(isFull()){
            extendsArray();
        }
    }
}
```

```

        }
        studentArray[nextIndex++]=student;
    }
    public void add(int index,Student student){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                studentArray[i+1]=studentArray[i];
            }
            studentArray[index]=student;
            nextIndex++;
        }
    }
    public void addFirst(Student student){
        add(0,student);
    }
    public void addLast(Student student){
        add(nextIndex,student);
    }
    public void trimToSize(){
        Student[] tempStudentArray=new Student[size()];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    public int size(){
        return nextIndex;
    }
    public void clear(){
        studentArray=new Student[initSize];
        nextIndex=0;
    }
    public void remove(int index){
        if(!isEmpty()){
            if(index>=0 && index<nextIndex){
                for(int i=index; i<nextIndex-1; i++){
                    studentArray[i]=studentArray[i+1];
                }
            }
            nextIndex--;
        }
    }
    public void removeFirst(){
        remove(0);
    }
    public void removeLast(){
        remove(size()-1);
    }
}

```

```

public int search(Student student){
    for (int i = 0; i < nextIndex; i++){
        if(student.equals(studentArray[i])){
            return i;
        }
    }
    return -1;
}

public void display(){
    System.out.print("{}");
    for (int i = 0; i < nextIndex; i++){
        System.out.print(studentArray[i].toString()+" ", "");
    }
    System.out.println(isEmpty()? "empty":"\b\b");
}

public void display(int start){

}

public void display(int startIndex, int endIndex){

}

public Student get(int index){
    return null;
}

public Student getFirst(){
    return null;
}

public Student getLast(){
    return null;
}

public boolean isEmpty(){
    return nextIndex<=0;
}

public int capacity(){
    return studentArray.length;
}

public boolean remove(Student student){
    int index=search(student);
    remove(index);
    return index!=-1;
}

}

class Demo{
    public static void main(String args[]){
        StudentList stList=new StudentList(12,0.25);
        stList.add(new Student("S0001","Nimal",65,67));
        stList.add(new Student("S0002","Amal",85,70));
        stList.add(new Student("S0003","Bimal",35,30));
        stList.add(new Student("S0004","Ramal",55,70));
        stList.add(new Student("S0005","Anil",95,90));
        stList.display(); //{["S0001,Nimal,65,76], [S0002", "Amal",85,70]}
        stList.remove(new Student("S0003","Bimal",35,30));
        stList.display(); //{["S0001,Nimal,65,76], [S0002", "Amal",85,70]}
    }
}

```

$$\left. \begin{array}{l} \{ \\ \} \end{array} \right\}$$

Student Marks Management System

Q147

```
//-----Student.java-----
```

```
class Student{
    private String id;
    private String name;
    private int prfMarks;
    private int dbmsMarks;

    Student(){

    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
    public String getId(){
        return id;
    }
    public String getName(){
        return name;
    }
    public int getPrfMarks(){
        return prfMarks;
    }
    public int getDbmsMarks(){
        return dbmsMarks;
    }
    public boolean equals(Student student){
        return id.equalsIgnoreCase(student.id);
    }
    public String toString(){
        return "["+id+", "+name+", "+prfMarks+", "+dbmsMarks+"]";
    }
}
```

```
//-----StudentList.java-----
```

```
class StudentList{ //Collection of student objects
    private Student[] studentArray;
    private int nextIndex;
    private double loadFact;
    private int initSize;

    StudentList(int initSize, double loadFact){
        studentArray=new Student[initSize];
        nextIndex=0;
        this.loadFact=loadFact;
    }
}
```

```

        this.initSize=initSize;
    }
    private void extendsArray(){
        Student tempStudentArray[]=new Student[studentArray.length+(int)(loadFact*studentArray.length)];
        for (int i = 0; i < studentArray.length; i++){
            tempStudentArray[i]=studentArray[i];
        }
        studentArray=tempStudentArray;
    }
    private boolean isFull(){
        return nextIndex>=studentArray.length;
    }
    public void add(Student student){ //Insertion order
        if(isFull()){
            extendsArray();
        }
        studentArray[nextIndex++]=student;
    }
    public void add(int index,Student student){
        if(isFull()){
            extendsArray();
        }
        if(index>=0 && index<=nextIndex){
            for (int i = nextIndex-1; i >=index; i--){
                studentArray[i+1]=studentArray[i];
            }
            studentArray[index]=student;
            nextIndex++;
        }
    }
    public int size(){
        return nextIndex;
    }
    public void clear(){
        studentArray=new Student[initSize];
        nextIndex=0;
    }
    public void remove(int index){
        if(!isEmpty()){
            if(index>=0 && index<nextIndex){
                for(int i=index; i<nextIndex-1; i++){
                    studentArray[i]=studentArray[i+1];
                }
            }
            nextIndex--;
        }
    }
    public Student search(String id){
        for (int i = 0; i < nextIndex; i++){
            if(studentArray[i].getId().equalsIgnoreCase(id)){
                return studentArray[i];
            }
        }
    }

```

```

        return null;
    }
    public int search(Student student){
        for (int i = 0; i < nextIndex; i++){
            if(student.equals(studentArray[i])){
                return i;
            }
        }
        return -1;
    }
    public void display(){
        System.out.print("{}");
        for (int i = 0; i < nextIndex; i++){
            System.out.print(studentArray[i].toString()+" ", );
        }
        System.out.println(isEmpty()? "empty":"\b\b");
    }
    public Student get(int index){
        return null;
    }
    public boolean isEmpty(){
        return nextIndex<=0;
    }
    public boolean remove(Student student){
        int index=search(student);
        remove(index);
        return index!=-1;
    }
}

```

//-----AddStudent.java-----

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class AddStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btAdd;

    private StudentList studentList;
    AddStudentForm(StudentList studentList){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.studentList=studentList;
        JLabel titleLabel=new JLabel("Add Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
    }
    //-----

```

```

JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
btAdd=new JButton("Add Student");
btCancel=new JButton("Cancel");
btAdd.setFont(new Font("",1,20));
btCancel.setFont(new Font("",1,20));

southPanel.add(btAdd);
btAdd.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent evt){
        String id=txtStudentId.getText();
        String name=txtName.getText();
        int prfMarks=Integer.parseInt(txtPrfMarks.getText());
        int dbmsMarks=Integer.parseInt(txtDbmsMarks.getText());
        Student s1=new Student(id,name,prfMarks,dbmsMarks);
        studentList.add(s1);
    }
});

southPanel.add(btCancel);
add("South",southPanel);
//-----

JLabel lblStudentId=new JLabel("Student ID");
JLabel lblName=new JLabel("Name");
JLabel lblPrfMarks=new JLabel("Prf Marks");
JLabel lblDbmsMarks=new JLabel("Dbms Marks");
lblStudentId.setFont(new Font("",1,20));
lblName.setFont(new Font("",1,20));
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));

JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));

```



```

        idTextPanal.add(txtStudentId);
        textPanel.add(idTextPanal);

        JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
        nameTextPanal.add(txtName);
        textPanel.add(nameTextPanal);

        JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
        prfMarksTextPanal.add(txtPrfMarks);
        textPanel.add(prfMarksTextPanal);

        JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
        dbmsMarksTextPanal.add(txtDbmsMarks);
        textPanel.add(dbmsMarksTextPanal);
        add("Center",textPanel);
    }
}

```

//-----SearchStudent.java-----

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class SearchStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btSearch;
    private StudentList studentList;
    SearchStudentForm(StudentList studentList){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.studentList=studentList;

        JLabel titleLabel=new JLabel("Search Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btSearch=new JButton("Search Student");
        btCancel=new JButton("Cancel");
        btSearch.setFont(new Font("",1,20));
        btSearch.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String id=txtStudentId.getText();
                Student s1=studentList.search(id);
                if(s1!=null){
                    txtName.setText(s1.getName());

```

```

        txtPrfMarks.setText(""+s1.getPrfMarks());
        txtDbmsMarks.setText(s1.getDbmsMarks()+"");
    }
}

});
btCancel.setFont(new Font("",1,20));

southPanel.add(btSearch);
southPanel.add(btCancel);
add("South",southPanel);
//-----

JLabel lblStudentId=new JLabel("Student ID");
JLabel lblName=new JLabel("Name");
JLabel lblPrfMarks=new JLabel("Prf Marks");
JLabel lblDbmsMarks=new JLabel("Dbms Marks");
lblStudentId.setFont(new Font("",1,20));
lblName.setFont(new Font("",1,20));
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));

JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanal.add(txtStudentId);
textPanel.add(idTextPanel);

JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanal.add(txtName);
textPanel.add(nameTextPanal);

JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanal.add(txtPrfMarks);
textPanel.add(prfMarksTextPanal);

```

```

        JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
        dbmsMarksTextPanal.add(txtDbmsMarks);
        textPanel.add(dbmsMarksTextPanal);
        add("Center",textPanel);
    }
}
//-----MainForm.java-----
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MainForm extends JFrame{
    private JButton btAddStudent;
    private JButton btSearchStudent;
    private JButton btViewStudent;
    private JButton btExit;
    private StudentList studentList;
    MainForm(){
        setSize(500,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        studentList=new StudentList(100,0.5);
        JLabel titleLabel=new JLabel("Student' Marks Management System");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel buttonPanel=new JPanel(new GridLayout(3,1));
        btAddStudent=new JButton("Add Student Form");
        btAddStudent.setFont(new Font("",1,20));
        btAddStudent.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                new AddStudentForm(studentList).setVisible(true);
            }
        });
        buttonPanel.add(btAddStudent);

        btViewStudent=new JButton("View Student Form");
        btViewStudent.setFont(new Font("",1,20));
        buttonPanel.add(btViewStudent);

        btSearchStudent=new JButton("Search Student Form");
        btSearchStudent.setFont(new Font("",1,20));
        buttonPanel.add(btSearchStudent);
        btSearchStudent.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                new SearchStudentForm(studentList).setVisible(true);
            }
        });
        btExit=new JButton("Exit");
        btExit.setFont(new Font("",1,20));
        buttonPanel.add(btExit);
        add("Center",buttonPanel);
    }
}

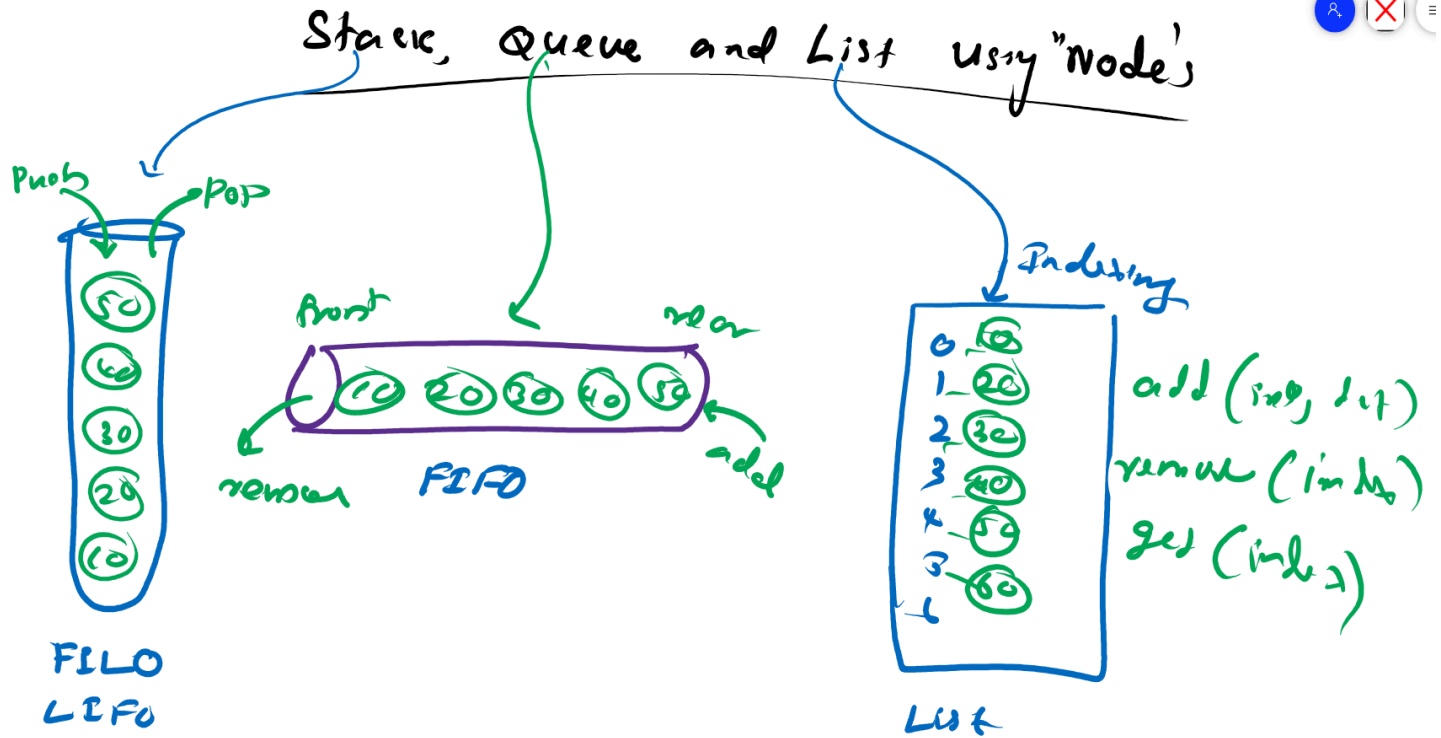
```

```

}
public static void main(String args[]){
    new MainForm().setVisible(true);
}
}

```

Data Structures using Linked List (Nodes) 12-07-2026



Class Stack using "Nodes"

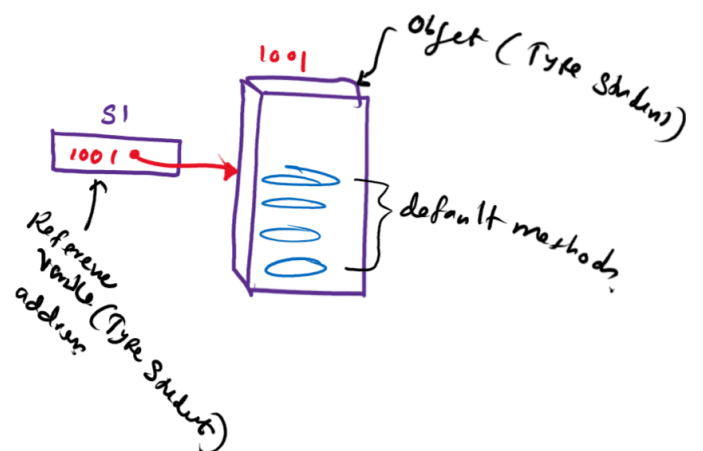
Q148 Step I

=====

```

class Student{
}
class Demo{
    public static void main(String args[]){
        Student s1;
        s1=new Student();
    }
}

```



Q149 Step II

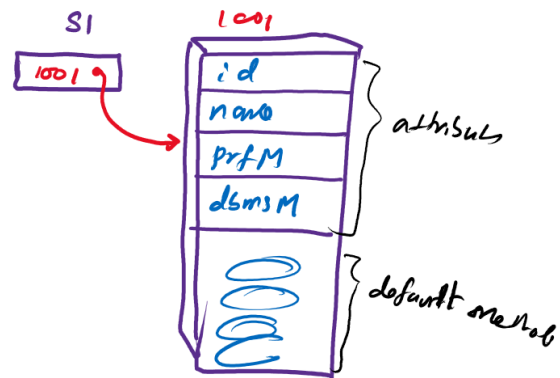
=====

```
class Student{
    String id;
    String name;
    int prfMarks;
    int dbmsMarks;
}

class Demo{
    public static void main(String args[]){
        Student s1;
        s1=new Student();
    }
}
```

```
class Student{
    String id;
    String name;
    int prfMarks;
    int dbmsMarks;
}

class Demo{
    public static void main(String args[]){
        Student s1;
        s1=new Student();
    }
}
```



Q150 Step III

=====

```
class Student{
    String id;
    String name;
    int prfMarks;
    int dbmsMarks;

    Student(){
    }
    Student(String id, String name, int prfMarks, int dbmsMarks){
        this.id=id;
        this.name=name;
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
}

class Demo{
    public static void main(String args[]){
        Student s1=new Student("S0001","Nimal",76,78);
    }
}
```



```
Student(){  
    //  
}  
Student(String id, String name, int prfMarks, int dbmsMarks){  
    this.id=id;  
    this.name=name;  
    this.prfMarks=prfMarks;  
    this.dbmsMarks=dbmsMarks;  
}
```

=====

}

```
Student(){  
    //
```

}

}

}

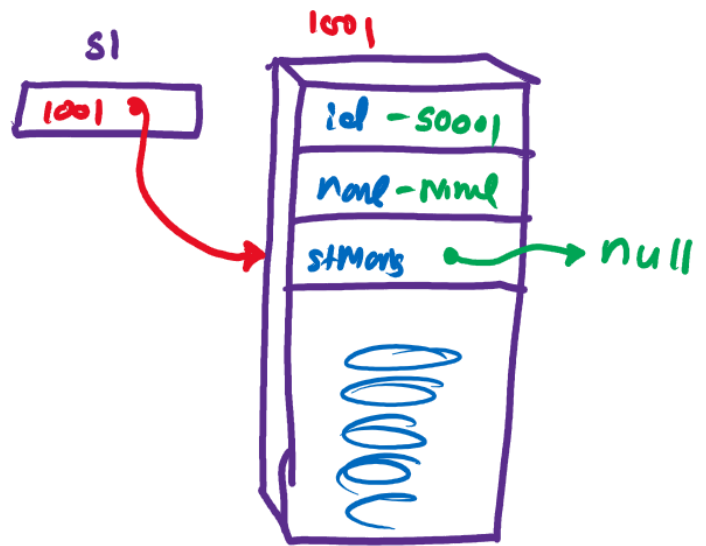
```
class Demo{
    public static void main(String args[]){
        Student s1=new Student("S0001","Nimal");
    }
}
```

```

class Marks{
    int prfMarks;
    int dbmsMarks;
    Marks(int prfMarks, int dbmsMarks){
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
}
class Student{
    String id;
    String name;
    Marks studentMarks;

    Student(){
        //
    }
    Student(String id, String name){
        this.id=id;
        this.name=name;
    }
}
class Demo{
    public static void main(String args[]){
        Student s1=new Student("S0001","Nimal");
    }
}

```



Q152 Step V

```

class Marks{
    int prfMarks;
    int dbmsMarks;
    Marks(int prfMarks, int dbmsMarks){
        this.prfMarks=prfMarks;
        this.dbmsMarks=dbmsMarks;
    }
}
class Student{
    String id;
    String name;
    Marks studentMarks;

    Student(){
    }
    Student(String id, String name){
        this.id=id;
        this.name=name;
    }
}
class Demo{
    public static void main(String args[]){
        Student s1=new Student("S0001","Nimal");
        s1.studentMarks=new Marks(67,78);
    }
}

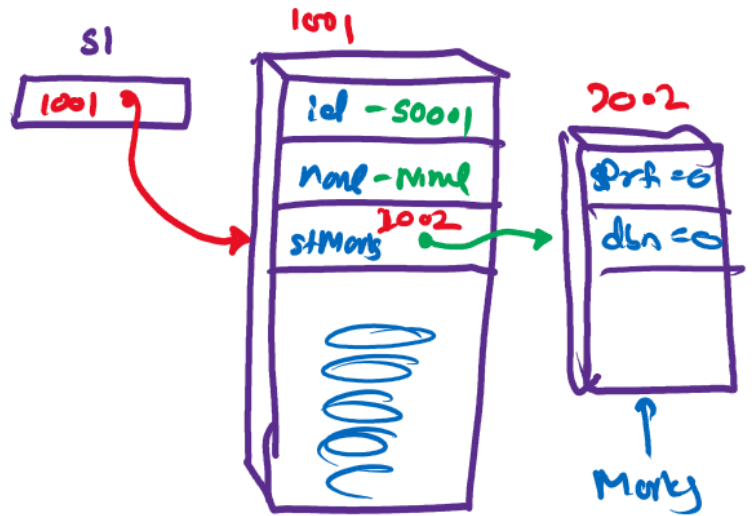
```

```

    }
}

class Demo{
    public static void main(String args[]){
        Student s1=new Student("S0001","Nimal");
        s1.studentMarks=new Marks(67,78);
    }
}

```



}

Diagram illustrating a linked list structure:

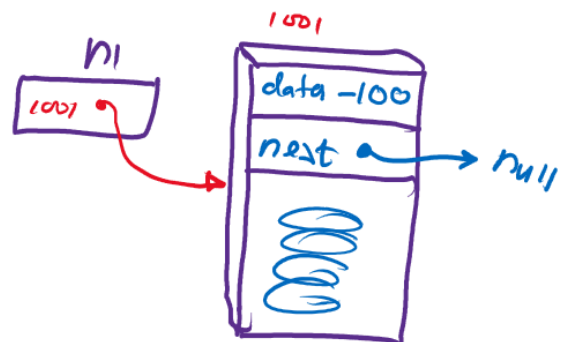
- Node 1:**
 - id - 50001
 - name - Mune
 - st Marks 1002
- Node 2:**
 - Prof 67
 - dtn 78

The diagram shows a pointer **S1** pointing to the first node. The first node's **st Marks** field (1002) points to the second node. A separate pointer labeled **Marks** also points to the second node.

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

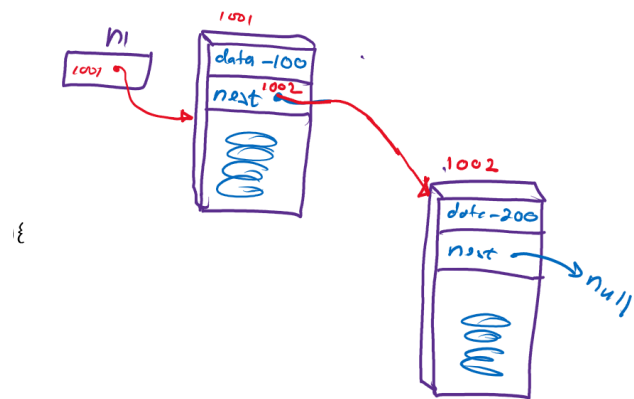
class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
    }
}
```



Q155 Step VIII

=====

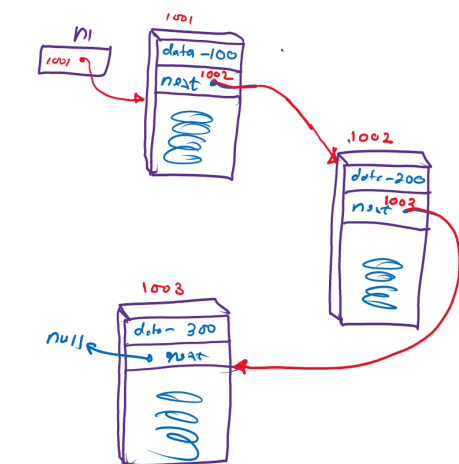
```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
    }
}
```



Q156 Step IX

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
        n1.next.next=new Node(300);
    }
}
```

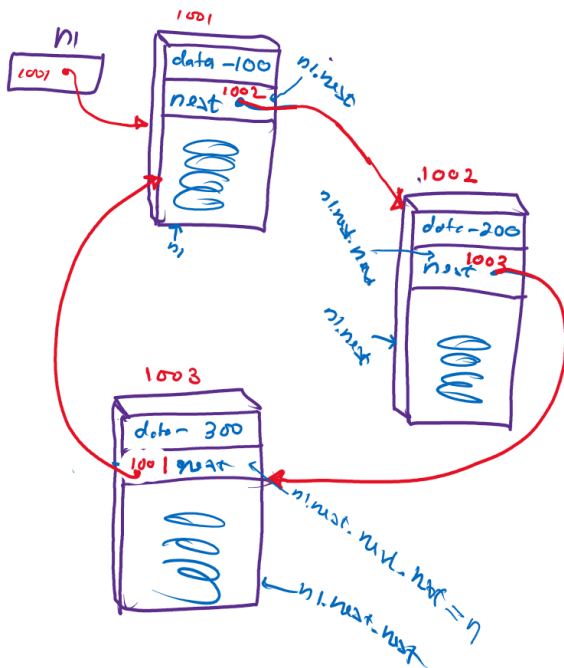


Q157 Step X

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
        n1.next.next=new Node(300);
        n1.next.next.next=n1;
    }
}
```



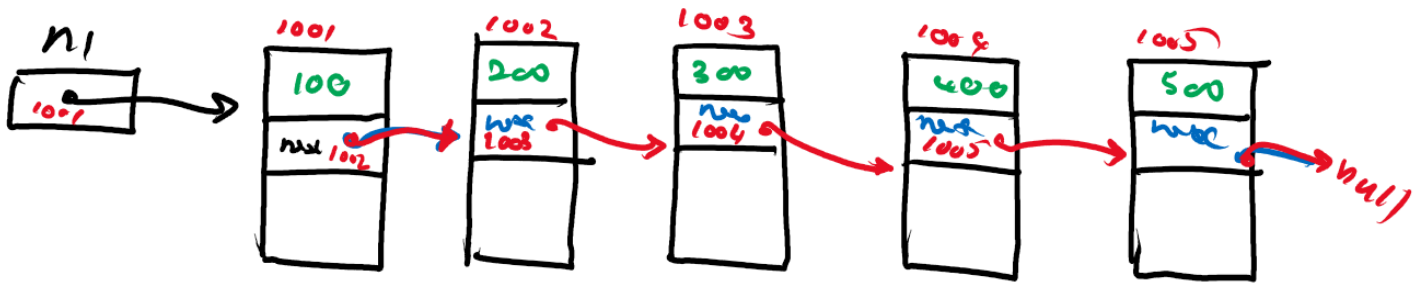
Q158 Step XI

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
        n1.next.next=new Node(300);
        n1.next.next.next=new Node(400);
        n1.next.next.next.next=new Node(500);

        //prints data- [100, 200, 300, 400, 500] using a while loop
    }
}
```



////////////////////////////////////

Q159 From Q158

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Demo{
    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
        n1.next.next=new Node(300);
        n1.next.next.next=new Node(400);
        n1.next.next.next.next=new Node(500);

        //prints data- [100, 200, 300, 400, 500] using a while loop
        System.out.print("[");

        System.out.print(n1.data+", ");
        n1=n1.next;

        System.out.print(n1.data+", ");
        n1=n1.next;

        System.out.println("\b\b]");
    }
}
```

```
////////////////////////////////////  
Q160 From Q159
```

```
=====
```

```
class Node{  
    int data;  
    Node next;  
    Node(int data){this.data=data;}  
}  
class Demo{  
    public static void main(String args[]){  
        Node n1=new Node(100);  
        n1.next=new Node(200);  
        n1.next.next=new Node(300);  
        n1.next.next.next=new Node(400);  
        n1.next.next.next.next=new Node(500);  
  
        //prints data- [100, 200, 300, 400, 500] using a while loop  
        System.out.print("[");  
        while(n1!=null){  
            System.out.print(n1.data+", ");  
            n1=n1.next;  
        }  
        System.out.println("\b\b");  
    }  
}
```

```
////////////////////////////////////
```

```
Q161 From Q160
```

```
=====
```

```
class Node{  
    int data;  
    Node next;  
    Node(int data){this.data=data;}  
}  
class Demo{  
    public static void main(String args[]){  
        Node n1=new Node(100);  
        n1.next=new Node(200);  
        n1.next.next=new Node(300);  
        n1.next.next.next=new Node(400);  
        n1.next.next.next.next=new Node(500);  
  
        //prints data- [100, 200, 300, 400, 500] using a while loop  
        System.out.print("[");  
        Node temp=n1;  
        while(temp!=null){  
            System.out.print(temp.data+", ");  
            temp=temp.next;  
        }  
        System.out.println("\b\b");  
  
        System.out.print("[");  
        temp=n1;  
        while(temp!=null){
```

////////////////////////////////////

=====

////////////////////////////////////

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Demo{
    public static void printData(Node node){
        System.out.print("[");
        while(node!=null){
            System.out.print(node.data+", ");
            node=node.next;
        }
        System.out.println("\b\b");
    }

    public static void main(String args[]){
        Node n1=new Node(100);
        n1.next=new Node(200);
        n1.next.next=new Node(300);
```

```

//prints data
printData(n1); //[100, 200, 300, 400, 500] using a while loop
printData(n1); //[100, 200, 300, 400, 500] using a while loop
}
}

```

```

////////////////////////////////////////////////////////////////////////////////////////////////
Q164 Step XIII
=====
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Demo{
    public static void printData(Node node){
        System.out.print("[");
        while(node!=null){
            System.out.print(node.data+" ");
            node=node.next;
        }
        System.out.println("\b\b]");
    }
    public static void main(String args[]){
        Node top=null;
        push(top,100);
        printData(top); //[100]

        push(top,200);
        printData(top); //[200,100]

        push(top,300);
        printData(top); //[300,200,100]

        push(top,400);
        printData(top); //[400,300,200,100]
    }
}

```

Q165 From Q164

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Demo{
    public static void printData(Node node){
        System.out.print("[");
        while(node!=null){
            System.out.print(node.data+", ");
            node=node.next;
        }
        System.out.println("\b\b]");
    }
    public static void main(String args[]){
        Node top=null;

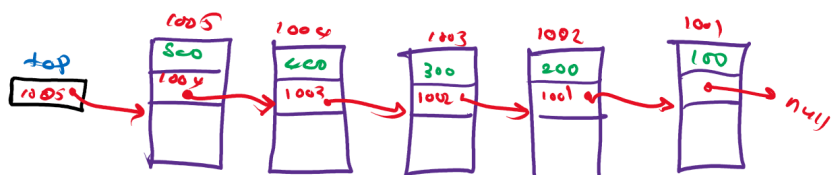
        Node n1=new Node(100);
        n1.next=top;
        top=n1;
        printData(top); //[100]

        n1=new Node(200);
        n1.next=top;
        top=n1;
        printData(top); //[200,100]

        n1=new Node(300);
        n1.next=top;
        top=n1;
        printData(top); //[300,200,100]

        n1=new Node(400);
        n1.next=top;
        top=n1;
        printData(top); //[400,300,200,100]
        n1=new Node(500);
        n1.next=top;
        top=n1;
        printData(top); //[500,400,300,200,100]

    }
}
```



////////////////////////////////////
Q166 Step XIV IRemove the top element (Node)
=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Demo{
    public static void printData(Node node){
        System.out.print("[");
        while(node!=null){
            System.out.print(node.data+", ");
            node=node.next;
        }
        System.out.println("\b\b]");
    }
    public static void main(String args[]){
        Node top=null;

        Node n1=new Node(100);
        n1.next=top;
        top=n1;
        printData(top); //[100]

        n1=new Node(200);
        n1.next=top;
        top=n1;
        printData(top); //[200,100]

        n1=new Node(300);
        n1.next=top;
        top=n1;
        printData(top); //[300,200,100]

        n1=new Node(400);
        n1.next=top;
        top=n1;
        printData(top); //[400,300,200,100]

        n1=new Node(500);
        n1.next=top;
        top=n1;
        printData(top); //[500,400,300,200,100]

        //Insert codes here to remove the top element
        //
        //
        printData(top); //[400,300,200,100]
    }
}
```

////////////////////////////////////
Q167 From Q166

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Demo{
    public static void printData(Node node){
        System.out.print("[");
        while(node!=null){
            System.out.print(node.data+", ");
            node=node.next;
        }
        System.out.println("\b\b]");
    }
    public static void main(String args[]){
        Node top=null;

        Node n1=new Node(100);
        n1.next=top;
        top=n1;
        printData(top); //[100]

        n1=new Node(200);
        n1.next=top;
        top=n1;
        printData(top); //[200,100]

        n1=new Node(300);
        n1.next=top;
        top=n1;
        printData(top); //[300,200,100]

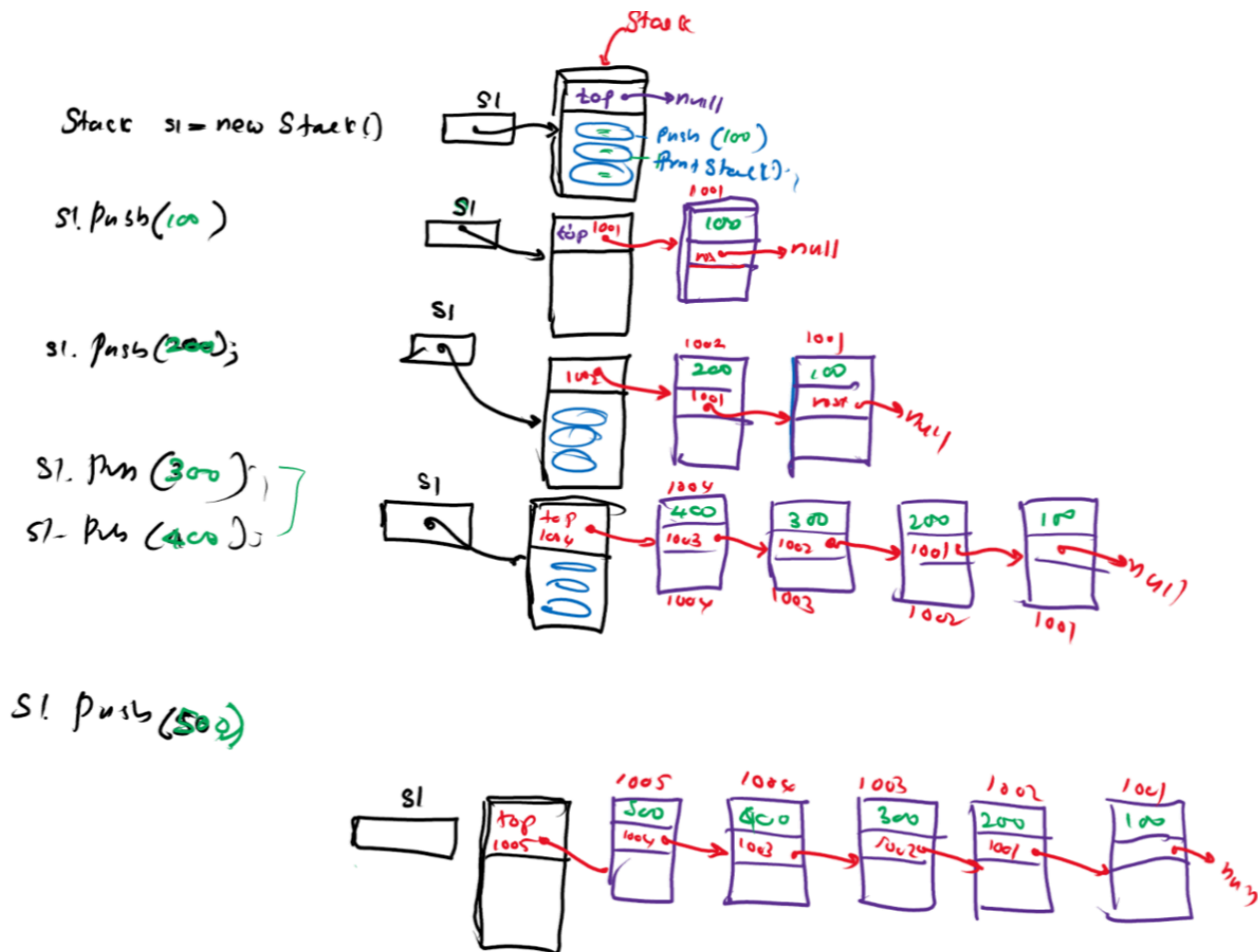
        n1=new Node(400);
        n1.next=top;
        top=n1;
        printData(top); //[400,300,200,100]

        n1=new Node(500);
        n1.next=top;
        top=n1;
        printData(top); //[500,400,300,200,100]

        //Insert codes here to remove the top element
        if(top!=null){
            top=top.next;
        }
        printData(top); //[400,300,200,100]
    }
}
```

Q168 Class "Stack"

=====



Q169 From Q168

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Stack{
    private Node top;
    public void push(int data){
        //
    }
    public void pop(){
        //
    }
    public void printStack(){
        //
    }
}
```

```
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(100);
        s1.push(200);
        s1.push(300);
        s1.push(400);
        s1.push(500);
        s1.printStack(); //[500, 400, 300, 200, 100]

        s1.pop();
        s1.printStack(); //[400, 300, 200, 100]
    }
}

//////////////////////////////////////////////////////////////////////////////////////////////////////////////////
Q170 From Q169
=====
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class Stack{
    private Node top;
    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;

    }
    public void pop(){
        //
    }
    public void printStack(){
        System.out.print("[");
        Node temp=top;
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println("\b\b]");
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(100);
        s1.push(200);
        s1.push(300);
        s1.push(400);
        s1.push(500);
        s1.printStack(); //[500, 400, 300, 200, 100]
```

////////////////////////////////////

////////////////////////////////////
Q172 public boolean isEmpty(), public int size() and public void clear()
=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Stack{
    private Node top;
    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.print("[");
        Node temp=top;
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println("\b\b");
    }
}
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack();//[empty]
        System.out.println("Size of the stack : "+s1.size()); //0
        System.out.println("Stack is empty   : "+s1.isEmpty());//true
        s1.push(100);
        s1.push(200);
        s1.push(300);
        s1.push(400);
        s1.push(500);
        s1.printStack(); //[500, 400, 300, 200, 100]
        System.out.println("Size of the stack : "+s1.size()); //5
        System.out.println("Stack is empty   : "+s1.isEmpty());//false

        s1.clear();
        s1.printStack();//[empty]
        System.out.println("Size of the stack : "+s1.size()); //0
        System.out.println("Stack is empty   : "+s1.isEmpty());//true
    }
}
```

Q173 From Q172

=====

```
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Stack{
    private Node top;
    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.print("[");
        Node temp=top;
        while(temp!=null){
            System.out.print(temp.data+" ");
            temp=temp.next;
        }
        System.out.println(isEmpty() ?"empty]":"\b\b]");
    }
    public int size(){
        Node temp=top;
        int count=0;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    public boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
}
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack();//[empty]
        System.out.println("Size of the stack : "+s1.size()); //0
        System.out.println("Stack is empty   : "+s1.isEmpty());//true
        s1.push(100);
    }
}
```

[illegible]


```

    public int size(){
        Node temp=top;
        int count=0;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    public boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(100);
        s1.push(200);
        s1.push(300);
        s1.push(400);
        s1.push(500);
        s1.printStack(); //[500, 400, 300, 200, 100]
        System.out.println("Index of 200 : "+s1.search(200)); //-1

        System.out.println("Index of 600 : "+s1.search(600)); //-1
    }
}

```

```

class Stack{
    private Node top;
    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;

    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.print("[");
        Node temp=top;
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(isEmpty() ?"empty]":"\b\b]");
    }
    public int size(){
        Node temp=top;
        int count=0;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    public boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
}

```

////////////////////////////////////

```

    public boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=top;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(100);
        s1.push(200);
        s1.push(300);
        s1.push(400);
        s1.push(500);
        s1.printStack(); //[500, 400, 300, 200, 100]

        int[] dataArray=s1.toArray();
        System.out.println(Arrays.toString(dataArray));//[500, 400, 300, 200, 100]
    }
}

```

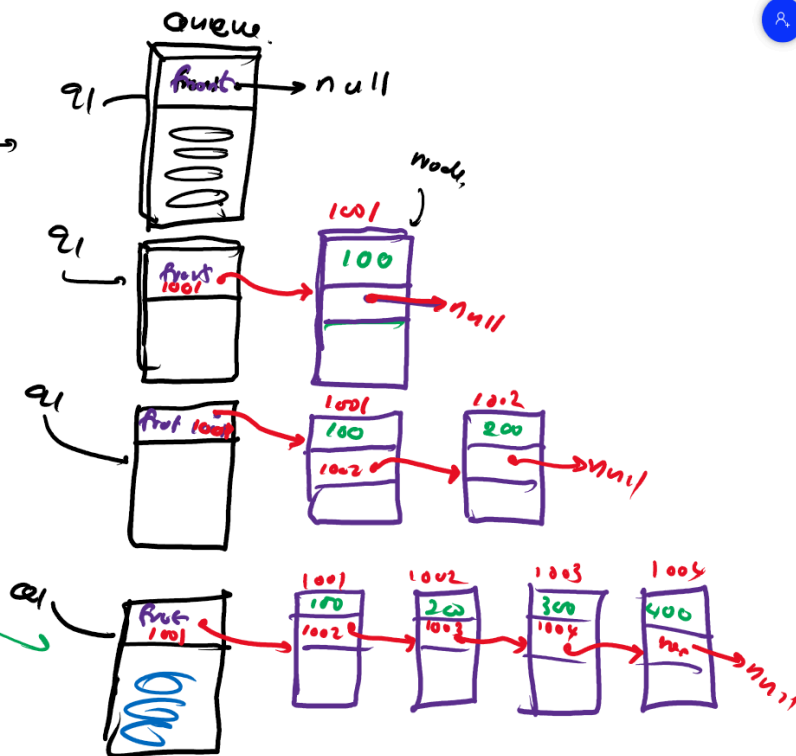


Queue q1 = new Queue();

q1.add(100);

q1.add(200);

q1.add(300);
q1.add(400);



Q178

import java.util.*;

class Node{

int data;

Node next;

Node(int data){this.data=data;}

}

class Queue{

private Node front;

public void add(int data){

Node n1=new Node(data);

}

public void remove(){

if(front!=null){

front=front.next;

}

}

public void printQueue(){

System.out.print("[");

Node temp=front;

while(temp!=null){

System.out.print(temp.data+", ");

temp=temp.next;

}

System.out.println(isEmpty() ? "empty" : "\b\b");

}

```

    public int size(){
        Node temp=front;
        int count=0;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    public boolean isEmpty(){
        return front==null;
    }
    public void clear(){
        front=null;
    }
    public int search(int data){
        Node temp=front;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=front;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        Queue q1=new Queue();
        q1.add(100);
        q1.add(200);
        q1.add(300);
        q1.add(400);
        q1.add(500);
        q1.printQueue();//[100, 200, 300, 400, 500]

        q1.remove();
        q1.printQueue();//[200, 300, 400, 500]
    }
}

```

////////////////////////////////////
Q179 From Q178

=====

```
import java.util.*;
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class Queue{
    private Node front;
    public void add(int data){
        Node n1=new Node(data);
        //find the last node
        if(isEmpty()){
            front=n1;
        }else{
            Node lastNode=front;
            while(lastNode.next!=null){
                lastNode=lastNode.next;
            }
            lastNode.next=n1;
        }
    }
    public void remove(){
        if(front!=null){
            front=front.next;
        }
    }
    public void printQueue(){
        System.out.print("[");
        Node temp=front;
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(isEmpty() ?"empty]":"\b\b]");
    }
    public int size(){
        Node temp=front;
        int count=0;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    public boolean isEmpty(){
        return front==null;
    }
    public void clear(){
        front=null;
    }
}
```



```

    public int search(int data){
        Node temp=front;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=front;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        Queue q1=new Queue();
        q1.add(100);
        q1.add(200);
        q1.add(300);
        q1.add(400);
        q1.add(500);
        q1.printQueue();//[100, 200, 300, 400, 500]

        q1.remove();
        q1.printQueue();//[200, 300, 400, 500]
    }
}

```

Class List

List (indexing, random access, insertion only)

public void add (index, data)

" " remove (index)

" int get (index) → return data

public void addFirst (data)

" " addLast (data)

" " removeFirst()

" " removeLast()

Q180

```
import java.util.*;
```

```
class Node{
```

```
    int data;
```

```
    Node next;
```

```
    Node(int data){this.data=data;}
```

```
}
```

```
class List{
```

```
    private Node first;
```

```
    public void add(int data){
```

```
        Node n1=new Node(data);
```

```
        //find the last node
```

```
        if(isEmpty()){
```

```
            first=n1;
```

```
        }else{
```

```
            Node lastNode=first;
```

```
            while(lastNode.next!=null){
```

```
                lastNode=lastNode.next;
```

```
            }
```

```
            lastNode.next=n1;
```

```
        }
```

```
    }
```

```
    public void add(int index, int data){
```

```
    }
```

```
    public void remove(int index){
```

```
        //
```

```
    }
```

```

public int get(int index){
    //
    return -1;
}
public void printList(){
    System.out.print("[");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty":"\b\b");
}
public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
public boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}
public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}
public int[] toArray(){
    int[] tempDataArray=new int[size()];
    Node temp=first;
    for (int i = 0; i < tempDataArray.length; i++)    {
        tempDataArray[i]=temp.data;
        temp=temp.next;
    }
    return tempDataArray;
}
}

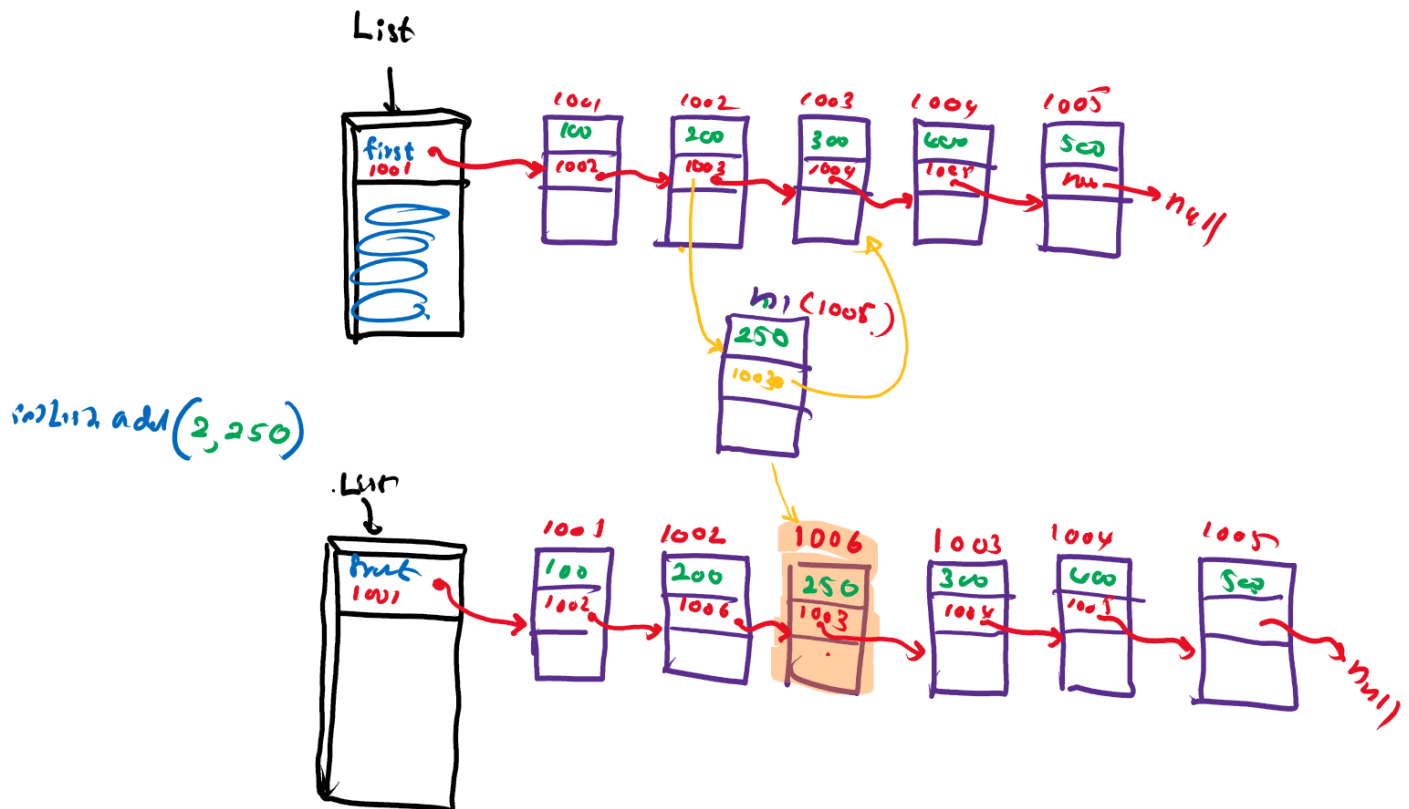
```

```

class Demo{
    public static void main(String args[]){
        List intList=new List();
        intList.add(100); //add(data)-->Insertion order[Queue]
        intList.add(200);
        intList.add(300);
        intList.add(400);
        intList.add(500);
        intList.printList();//[100, 200, 300, 400, 500]

        intList.add(2,250);//[100, 200, 250, 300, 400, 500]
    }
}

```



////////////////////////////////////

Q181 From Q180

=====

```

import java.util.*;
class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}
class List{
    private Node first;

    public void add(int data){ //Insertion order
        addLast(data); //add(size(),data)
    }
}

```

```

public void add(int index, int data){
    Node n1=new Node(data);
    if(index==0){
        n1.next=first;
        first=n1;
    }else{
        int count=0;
        Node temp=first;
        while(count<index-1){
            count++;
            temp=temp.next;
        }
        n1.next=temp.next;
        temp.next=n1;
    }
}
public void addLast(int data){
    add(size(),data);
}
public void addFirst(int data){
    add(0,data);
}
public void remove(int index){
    //
}
public int get(int index){
    //
    return -1;
}
public void printList(){
    System.out.print("[");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty]":"\b\b]");
}
public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
public boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}

```

```

    public int search(int data){
        Node temp=first;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=first;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        List intList=new List();
        intList.add(100); //add(data)-->Insertion order[Queue]
        intList.add(200);
        intList.add(300);
        intList.add(400);
        intList.add(500);
        intList.printList();//[100, 200, 300, 400, 500]

        intList.add(2,250);
        intList.printList();//[100, 200, 300, 400, 500]

        intList.addFirst(50);
        intList.addLast(600);
        intList.printList();//[50, 100, 200, 300, 400, 500, 600]
    }
}

```

Q182

```
import java.util.*;
```

```
class Node{
```

```
    int data;
```

```
    Node next;
```

```
    Node(int data){this.data=data;}
```

```
}
```

```
class List{
```

```
    private Node first;
```

```
    public void add(int data){ //Insertion order
```

```
        addLast(data); //add(size(),data)
```

```
}
```

```
    public void add(int index, int data){
```

```
        Node n1=new Node(data);
```

```
        if(index==0){
```

```
            n1.next=first;
```

```
            first=n1;
```

```
        }else{
```

```
            int count=0;
```

```
            Node temp=first;
```

```
            while(count<index-1){
```

```
                count++;
```

```
                temp=temp.next;
```

```
            }
```

```
            n1.next=temp.next;
```

```
            temp.next=n1;
```

```
        }
```

```
    }
```

```
    public void addLast(int data){
```

```
        add(size(),data);
```

```
}
```

```
    public void addFirst(int data){
```

```
        add(0,data);
```

```
}
```

```
    public void removeLast(){
```

```
        remove(0);
```

```
}
```

```
    public void removeFirst(){
```

```
        remove(size()-1);
```

```
}
```

```
    public void remove(int index){
```

```
        if(!isEmpty() && index>=0 && index<size()){
```

```
            if(index==0){
```

```
                first=first.next;
```

```
            }else{
```

```
                int count=0;
```

```
                Node temp=first;
```

```
                while(count<index-1){
```

```
                    count++;
```

```
                    temp=temp.next;
```

```
                }
```

```

        temp.next=temp.next.next;
    }
}

public int get(int index){
    //
    return -1;
}

public void printList(){
    System.out.print("[");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty":"\b\b");
}

public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}

public boolean isEmpty(){
    return first==null;
}

public void clear(){
    first=null;
}

public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public int[] toArray(){
    int[] tempDataArray=new int[size()];
    Node temp=first;
    for (int i = 0; i < tempDataArray.length; i++)    {
        tempDataArray[i]=temp.data;
        temp=temp.next;
    }
    return tempDataArray;
}

```



```

        temp=temp.next;
    }
    n1.next=temp.next;
    temp.next=n1;
}
}
}
public void addLast(int data){
    add(size(),data);
}
public void addFirst(int data){
    add(0,data);
}
public void removeLast(){
    remove(size()-1);
}
public void removeFirst(){
    remove(0);
}
public void remove(int index){
    if(!isEmpty() && index>=0 && index<size()){
        if(index==0){
            first=first.next;
        }else{
            int count=0;
            Node temp=first;
            while(count<index-1){
                count++;
                temp=temp.next;
            }
            temp.next=temp.next.next;
        }
    }
}
}
public int get(int index){
    //
    return -1;
}
public void printList(){
    System.out.print("[");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty]":"\b\b]");
}
public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
}

```

```

        }
        return count;
    }
    public boolean isEmpty(){
        return first==null;
    }
    public void clear(){
        first=null;
    }
    public int search(int data){
        Node temp=first;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=first;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        List intList=new List();
        intList.add(100);
        intList.printList(); //[100]

        intList.clear();
        intList.add(0,100);
        intList.printList(); //[100]

        intList.clear();
        intList.addFirst(100);
        intList.printList(); //[100]

        intList.clear();
        intList.addLast(100);
        intList.printList(); //[100]

        intList.remove(0);
        intList.printList(); //[empty]

        intList.add(100);
    }
}

```

```
intList.printList(); //[100]
intList.removeFirst();
intList.printList(); //[empty]

intList.add(100);
intList.printList(); //[100]
intList.removeLast();
intList.printList(); //[empty]
}
}
```

Q184 public int get(index)

=====

```
import java.util.*;

class Node{
    int data;
    Node next;
    Node(int data){this.data=data;}
}

class List{
    private Node first;

    public void add(int data){ //Insertion order
        addLast(data); //add(size(),data)
    }

    public void add(int index, int data){
        if(index>=0 && index<=size()){
            Node n1=new Node(data);
            if(index==0){
                n1.next=first;
                first=n1;
            }else{
                int count=0;
                Node temp=first;
                while(count<index-1){
                    count++;
                    temp=temp.next;
                }
                n1.next=temp.next;
                temp.next=n1;
            }
        }
    }

    public void addLast(int data){
        add(size(),data);
    }

    public void addFirst(int data){
        add(0,data);
    }

    public void removeLast(){
        remove(size()-1);
    }
}
```

```

public void removeFirst(){
    remove(0);
}
public void remove(int index){
    if(!isEmpty() && index>=0 && index<size()){
        if(index==0){
            first=first.next;
        }else{
            int count=0;
            Node temp=first;
            while(count<index-1){
                count++;
                temp=temp.next;
            }
            temp.next=temp.next.next;
        }
    }
}
public int get(int index){
    if(index>=0 && index<size()){
        int count=0;
        Node temp=first;
        while(count<index-1){
            count++;
            temp=temp.next;
        }
        return temp.data;
    }
    return -1;
}
public void printList(){
    System.out.print("[");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty]":"\b\b]");
}
public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
public boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}

```

```

    }
    public int search(int data){
        Node temp=first;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int[] toArray(){
        int[] tempDataArray=new int[size()];
        Node temp=first;
        for (int i = 0; i < tempDataArray.length; i++)    {
            tempDataArray[i]=temp.data;
            temp=temp.next;
        }
        return tempDataArray;
    }
}

class Demo{
    public static void main(String args[]){
        List intList=new List();
        intList.add(100);
        intList.add(200);
        intList.add(300);
        intList.add(400);
        intList.add(500);
        intList.add(600);
        intList.printList(); //[100,200,300,400,500,600]
        for (int i = 0; i < intList.size(); i++){
            int data=intList.get(i);
            System.out.println(data);
        }
    }
}

```



```

        first=n1;
    }else{
        int count=0;
        Node temp=first;
        while(count<index-1){
            count++;
            temp=temp.next;
        }
        n1.next=temp.next;
        temp.next=n1;
    }
    return true;
}
return false;
}
public boolean addLast(Student student){
    return add(size(),student);
}
public boolean addFirst(Student student){
    return add(0,student);
}
public void removeLast(){
    remove(size()-1);
}
public void removeFirst(){
    remove(0);
}
public void remove(int index){
    if(!isEmpty() && index>=0 && index<size()){
        if(index==0){
            first=first.next;
        }else{
            int count=0;
            Node temp=first;
            while(count<index-1){
                count++;
                temp=temp.next;
            }
            temp.next=temp.next.next;
        }
    }
}
public Student get(int index){
    if(index>=0 && index<size()){
        int count=0;
        Node temp=first;
        while(count<index-1){
            count++;
            temp=temp.next;
        }
        return temp.student;
    }
    return null;
}

```



```

}
public void printList(){
    System.out.print("{}");
    Node temp=first;
    while(temp!=null){
        System.out.print(temp.student+" ");
        temp=temp.next;
    }
    System.out.println(isEmpty() ?"empty":"\b\b");
}
public int size(){
    Node temp=first;
    int count=0;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
public boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}
public int search(Student student){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.student.equals(student)){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}
public Student[] toArray(){
    Student[] studentArray=new Student[size()];
    Node temp=first;
    for (int i = 0; i < studentArray.length; i++)    {
        studentArray[i]=temp.student;
        temp=temp.next;
    }
    return studentArray;
}
//-----Inner classes-----
public Student search(String id){
    Node temp=first;
    while(temp!=null){
        if(temp.student.getId().equalsIgnoreCase(id)){
            return temp.student;
        }
    }
}

```

```

    }
    return null;
}
class Node{
    private Student student;
    private Node next;
    Node(Student student){this.student=student;}
}
}

```

//-----AddStudenForm.java-----

```

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*;
class AddStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btAdd;

    private StudentList studentList;
    AddStudentForm(StudentList studentList){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.studentList=studentList;
        JLabel titleLabel=new JLabel("Add Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btAdd=new JButton("Add Student");
        btCancel=new JButton("Cancel");
        btAdd.setFont(new Font("",1,20));
        btCancel.setFont(new Font("",1,20));

        southPanel.add(btAdd);
        btAdd.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String id=txtStudentId.getText();
                String name=txtName.getText();
                int prfMarks=Integer.parseInt(txtPrfMarks.getText());
                int dbmsMarks=Integer.parseInt(txtDbmsMarks.getText());
                Student s1=new Student(id,name,prfMarks,dbmsMarks);
                studentList.add(s1);
            }
        });

        southPanel.add(btCancel);
    }
}

```

```
add("South",southPanel);  
//-----
```

```
JLabel lblStudentId=new JLabel("Student ID");  
JLabel lblName=new JLabel("Name");  
JLabel lblPrfMarks=new JLabel("Prf Marks");  
JLabel lblDbmsMarks=new JLabel("Dbms Marks");  
lblStudentId.setFont(new Font("",1,20));  
lblName.setFont(new Font("",1,20));  
lblPrfMarks.setFont(new Font("",1,20));  
lblDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel labelPanel=new JPanel(new GridLayout(4,1));  
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
labelPanel.add(lblStudentId);  
labelPanel.add(lblName);  
labelPanel.add(lblPrfMarks);  
labelPanel.add(lblDbmsMarks);  
add("West",labelPanel);
```

```
//-----
```

```
txtStudentId=new JTextField(5);  
txtStudentId.setFont(new Font("",1,20));  
txtName=new JTextField(10);  
txtName.setFont(new Font("",1,20));  
txtPrfMarks=new JTextField(4);  
txtPrfMarks.setFont(new Font("",1,20));  
txtDbmsMarks=new JTextField(4);  
txtDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel textPanel=new JPanel(new GridLayout(4,1));
```

```
JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));  
idTextPanal.add(txtStudentId);  
textPanel.add(idTextPanel);
```

```
JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
nameTextPanal.add(txtName);  
textPanel.add(nameTextPanal);
```

```
JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
prfMarksTextPanal.add(txtPrfMarks);  
textPanel.add(prfMarksTextPanal);
```

```
JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));  
dbmsMarksTextPanal.add(txtDbmsMarks);  
textPanel.add(dbmsMarksTextPanal);  
add("Center",textPanel);
```

```
}
```

```
}
```

//-----SearchStudentForm.java-----

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class SearchStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btSearch;
    private StudentList studentList;
    SearchStudentForm(StudentList studentList){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        this.studentList=studentList;

        JLabel titleLabel=new JLabel("Search Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btSearch=new JButton("Search Student");
        btCancel=new JButton("Cancel");
        btSearch.setFont(new Font("",1,20));
        btSearch.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String id=txtStudentId.getText();
                Student s1=studentList.search(id);
                if(s1!=null){
                    txtName.setText(s1.getName());
                    txtPrfMarks.setText(""+s1.getPrfMarks());
                    txtDbmsMarks.setText(s1.getDbmsMarks()+"");
                }
            }
        });
        btCancel.setFont(new Font("",1,20));

        southPanel.add(btSearch);
        southPanel.add(btCancel);
        add("South",southPanel);
        //-----

        JLabel lblStudentId=new JLabel("Student ID");
        JLabel lblName=new JLabel("Name");
        JLabel lblPrfMarks=new JLabel("Prf Marks");
        JLabel lblDbmsMarks=new JLabel("Dbms Marks");
        lblStudentId.setFont(new Font("",1,20));
        lblName.setFont(new Font("",1,20));
```

```
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);
```

```
//-----
```

```
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));
```

```
JPanel textPanel=new JPanel(new GridLayout(4,1));
```

```
JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanal.add(txtStudentId);
textPanel.add(idTextPanel);
```

```
JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanal.add(txtName);
textPanel.add(nameTextPanal);
```

```
JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanal.add(txtPrfMarks);
textPanel.add(prfMarksTextPanal);
```

```
JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
dbmsMarksTextPanal.add(txtDbmsMarks);
textPanel.add(dbmsMarksTextPanal);
add("Center",textPanel);
```

```
}
```

```
}
```

```
//-----MainForm.java-----
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
class MainForm extends JFrame{
```

```
    private JButton btAddStudent;
```

```
    private JButton btSearchStudent;
```

```
    private JButton btViewStudent;
```

```
    private JButton btExit;
```

```
    private StudentList studentList;
```

```
    MainForm(){
```

```

setSize(500,300);
setLocationRelativeTo(null);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
studentList=new StudentList();
JLabel titleLabel=new JLabel("Student' Marks Management System");
titleLabel.setFont(new Font("",1,27));
titleLabel.setHorizontalAlignment(JLabel.CENTER);
add("North",titleLabel);
//-----

JPanel buttonPanel=new JPanel(new GridLayout(3,1));
btAddStudent=new JButton("Add Student Form");
btAddStudent.setFont(new Font("",1,20));
btAddStudent.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent evt){
        new AddStudentForm(studentList).setVisible(true);
    }
});
buttonPanel.add(btAddStudent);

btViewStudent=new JButton("View Student Form");
btViewStudent.setFont(new Font("",1,20));
buttonPanel.add(btViewStudent);

btSearchStudent=new JButton("Search Student Form");
btSearchStudent.setFont(new Font("",1,20));
buttonPanel.add(btSearchStudent);
btSearchStudent.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent evt){
        new SearchStudentForm(studentList).setVisible(true);
    }
});
btExit=new JButton("Exit");
btExit.setFont(new Font("",1,20));
buttonPanel.add(btExit);
add("Center",buttonPanel);
}
public static void main(String args[]){
    new MainForm().setVisible(true);
}
}

```

File reading and writing

Q186 FileWriter

```
import java.util.*;
import java.io.*;
class Demo{
    public static void main(String args[])throws IOException{
        String name="Niroth Lakmal";
        FileWriter fw=new FileWriter("Student.txt");
        fw.write(name);
        fw.flush(); //or fw.close()
    }
}
```

Q187 Appendable FileWriter

```
import java.util.*;
import java.io.*;
class Demo{
    public static void main(String args[])throws IOException{
        String name="Niroth Lakmal";
        FileWriter fw=new FileWriter("Student.txt",true);
        fw.write(name);
        fw.write("\n");
        fw.flush();
    }
}
```

Q188 File Reading

```
import java.util.*;
import java.io.*;
class Demo{
    public static void main(String args[])throws IOException{
        FileReader fr=new FileReader("Student.txt");
        BufferedReader br=new BufferedReader(fr);

        String line=br.readLine();
        while(line!=null){
            System.out.println(line);
            line=br.readLine();
        }
    }
}
```

```
////////////////////////////////////
Q189 File Reading using class "Scanner"
=====
```

```
import java.util.*;
import java.io.*;
class Demo{
    public static void main(String args[])throws IOException{
        Scanner input=new Scanner(new File("Student.txt"));

        String line=input.nextLine();
        while(input.hasNext()){
            System.out.println(line);
            line=input.nextLine();
        }
    }
}
```

```
//-----Student.txt-----
```

```
S0001,Niroth,56,67
```

```
S0002,Nimal,58,67
```

```
S0003,Amal,55,67
```

```
S0004,Bimal,96,57
```

```
S0005,Lakmal,96,37
```

```
////////////////////////////////////
Q190
```

```
import java.util.*;
import java.io.*;
class Demo{
    public static void main(String args[])throws IOException{
        Scanner input=new Scanner(new File("Student.txt"));
        StudentList stList=new StudentList();

        String line=input.nextLine();
        while(input.hasNext()){
            String[] rowData=line.split(",");
            String id=rowData[0];
            String name=rowData[1];
            int prf=Integer.parseInt(rowData[2]);
            int dbms=Integer.parseInt(rowData[3]);
            Student student=new Student(id,name,prf,dbms);
            stList.add(student);
            line=input.nextLine();
        }

        stList.printList();
    }
}
```



```
//-----SearchStudentForm.java-----
import javax.swing.*;
import java.util.*;
import java.io.*;
import java.awt.*;
import java.awt.event.*;

class SearchStudentForm extends JFrame{
    private JTextField txtStudentId;
    private JTextField txtName;
    private JTextField txtPrfMarks;
    private JTextField txtDbmsMarks;

    private JButton btCancel;
    private JButton btSearch;

    SearchStudentForm(){
        setSize(400,300);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel titleLabel=new JLabel("Search Student Form");
        titleLabel.setFont(new Font("",1,27));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        add("North",titleLabel);
        //-----

        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btSearch=new JButton("Search Student");
        btCancel=new JButton("Cancel");
        btSearch.setFont(new Font("",1,20));
        btSearch.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                try{
                    Scanner input=new Scanner(new File("Student.txt"));
                    String id=txtStudentId.getText();
                    String[] rowData=null;
                    while(input.hasNext()){
                        String line=input.nextLine();
                        rowData=line.split(",");
                        if(rowData[0].equalsIgnoreCase(id)){
                            break;
                        }
                    }
                    txtName.setText(rowData[1]);
                    txtPrfMarks.setText(rowData[2]);
                    txtDbmsMarks.setText(rowData[3]);
                }catch(IOException ex){}
            }
        });
        btCancel.setFont(new Font("",1,20));
    }
}
```

```

southPanel.add(btSearch);
southPanel.add(btCancel);
add("South",southPanel);
//-----

JLabel lblStudentId=new JLabel("Student ID");
JLabel lblName=new JLabel("Name");
JLabel lblPrfMarks=new JLabel("Prf Marks");
JLabel lblDbmsMarks=new JLabel("Dbms Marks");
lblStudentId.setFont(new Font("",1,20));
lblName.setFont(new Font("",1,20));
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));

JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanal.add(txtStudentId);
textPanel.add(idTextPanel);

JPanel nameTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanal.add(txtName);
textPanel.add(nameTextPanal);

JPanel prfMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanal.add(txtPrfMarks);
textPanel.add(prfMarksTextPanal);

JPanel dbmsMarksTextPanal=new JPanel(new FlowLayout(FlowLayout.LEFT));
dbmsMarksTextPanal.add(txtDbmsMarks);
textPanel.add(dbmsMarksTextPanal);
add("Center",textPanel);

```

```

}

```

```

}

```

```
////////////////////////////////////  
Q192 UpdateStudentForm
```

```
=====
```

```
import javax.swing.*;  
import java.util.*;  
import java.io.*;  
import java.awt.*;  
import java.awt.event.*;  
class UpdateStudentForm extends JFrame{  
    private JTextField txtStudentId;  
    private JTextField txtName;  
    private JTextField txtPrfMarks;  
    private JTextField txtDbmsMarks;  
  
    private JButton btCancel;  
    private JButton btUpdate;  
  
    UpdateStudentForm(){  
        setSize(400,300);  
        setLocationRelativeTo(null);  
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
  
        JLabel titleLabel=new JLabel("Search Student Form");  
        titleLabel.setFont(new Font("",1,27));  
        titleLabel.setHorizontalAlignment(JLabel.CENTER);  
        add("North",titleLabel);  
        //-----  
  
        JPanel southPanel=new JPanel(new FlowLayout(FlowLayout.RIGHT));  
        btUpdate=new JButton("Update Student");  
        btCancel=new JButton("Cancel");  
        btUpdate.setFont(new Font("",1,20));  
        btUpdate.addActionListener(new ActionListener(){  
            public void actionPerformed(ActionEvent evt){  
                try{  
                    File stFile=new File("Student.txt");  
                    File tempFile=new File("TempStudent.txt");  
                    FileWriter fw=new FileWriter(tempFile);  
  
                    Scanner input=new Scanner(stFile);  
                    String id=txtStudentId.getText();  
                    String[] rowData=null;  
                    while(input.hasNext()){  
                        String line=input.nextLine();  
                        rowData=line.split(",");  
                        if(rowData[0].equalsIgnoreCase(id)){  
                            String  
newRow=id+","+txtName.getText()+","+txtPrfMarks.getText()+","+txtDbmsMarks.getText();  
                            fw.write(newRow+"\n");  
                            fw.flush();  
                        }else{  
                            fw.write(line+"\n");  
                            fw.flush();  
                        }  
                    }  
                }  
            }  
        });  
    }  
}
```

```

        }
    }
    fw.close();
    input.close();
    //-----
    fw=new FileWriter("Student.txt");
    input=new Scanner(new File("TempStudent.txt"));
    while(input.hasNext()){
        String line=input.nextLine();
        fw.write(line+"\n");
    }
    fw.close();
    input.close();
    tempFile.delete();
} catch(IOException ex){

}

});
btCancel.setFont(new Font("",1,20));

southPanel.add(btUpdate);
southPanel.add(btCancel);
add("South",southPanel);
//-----

JLabel lblStudentId=new JLabel("Student ID");
JLabel lblName=new JLabel("Name");
JLabel lblPrfMarks=new JLabel("Prf Marks");
JLabel lblDbmsMarks=new JLabel("Dbms Marks");
lblStudentId.setFont(new Font("",1,20));
lblName.setFont(new Font("",1,20));
lblPrfMarks.setFont(new Font("",1,20));
lblDbmsMarks.setFont(new Font("",1,20));

JPanel labelPanel=new JPanel(new GridLayout(4,1));
JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
labelPanel.add(lblStudentId);
labelPanel.add(lblName);
labelPanel.add(lblPrfMarks);
labelPanel.add(lblDbmsMarks);
add("West",labelPanel);

//-----
txtStudentId=new JTextField(5);
txtStudentId.setFont(new Font("",1,20));
txtStudentId.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent evt){
        try{
            Scanner input=new Scanner(new File("Student.txt"));
            String id=txtStudentId.getText();
            String[] rowData=null;
            while(input.hasNext()){
                String line=input.nextLine();

```

```

        rowData=line.split(",");
        if(rowData[0].equalsIgnoreCase(id)){
            break;
        }
    }
    txtName.setText(rowData[1]);
    txtPrfMarks.setText(rowData[2]);
    txtDbmsMarks.setText(rowData[3]);
} catch(IOException ex){}

}

});
txtName=new JTextField(10);
txtName.setFont(new Font("",1,20));
txtPrfMarks=new JTextField(4);
txtPrfMarks.setFont(new Font("",1,20));
txtDbmsMarks=new JTextField(4);
txtDbmsMarks.setFont(new Font("",1,20));

JPanel textPanel=new JPanel(new GridLayout(4,1));

JPanel idTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
idTextPanel.add(txtStudentId);
textPanel.add(idTextPanel);

JPanel nameTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
nameTextPanel.add(txtName);
textPanel.add(nameTextPanel);

JPanel prfMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
prfMarksTextPanel.add(txtPrfMarks);
textPanel.add(prfMarksTextPanel);

JPanel dbmsMarksTextPanel=new JPanel(new FlowLayout(FlowLayout.LEFT));
dbmsMarksTextPanel.add(txtDbmsMarks);
textPanel.add(dbmsMarksTextPanel);
add("Center",textPanel);
}
public static void main(String args[]){
    new UpdateStudentForm().setVisible(true);
}
}

//-----MainForm.java-----
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MainForm extends JFrame{
    private JButton btUpdateStudent;
    private JButton btDeleteStudent;
    private JButton btAddStudent;
    private JButton btSearchStudent;
    private JButton btViewStudent;
    private JButton btExit;

```

```

MainForm(){
    setSize(500,300);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    JLabel titleLabel=new JLabel("Student' Marks Management System");
    titleLabel.setFont(new Font("",1,27));
    titleLabel.setHorizontalAlignment(JLabel.CENTER);
    add("North",titleLabel);
    //-----

    JPanel buttonPanel=new JPanel(new GridLayout(6,1));
    btAddStudent=new JButton("Add Student Form");
    btAddStudent.setFont(new Font("",1,20));
    btAddStudent.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent evt){
            new AddStudentForm().setVisible(true);
        }
    });
    buttonPanel.add(btAddStudent);

    btViewStudent=new JButton("View Student Form");
    btViewStudent.setFont(new Font("",1,20));
    buttonPanel.add(btViewStudent);

    btSearchStudent=new JButton("Search Student Form");
    btSearchStudent.setFont(new Font("",1,20));
    buttonPanel.add(btSearchStudent);
    btSearchStudent.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent evt){
            new SearchStudentForm().setVisible(true);
        }
    });

    btUpdateStudent=new JButton("Update Student Form");
    btUpdateStudent.setFont(new Font("",1,20));
    buttonPanel.add(btUpdateStudent);
    btUpdateStudent.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent evt){
            new UpdateStudentForm().setVisible(true);
        }
    });

    btDeleteStudent=new JButton("Delete Student Form");
    btDeleteStudent.setFont(new Font("",1,20));
    buttonPanel.add(btDeleteStudent);
    btDeleteStudent.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent evt){
            //
        }
    });

    btExit=new JButton("Exit");

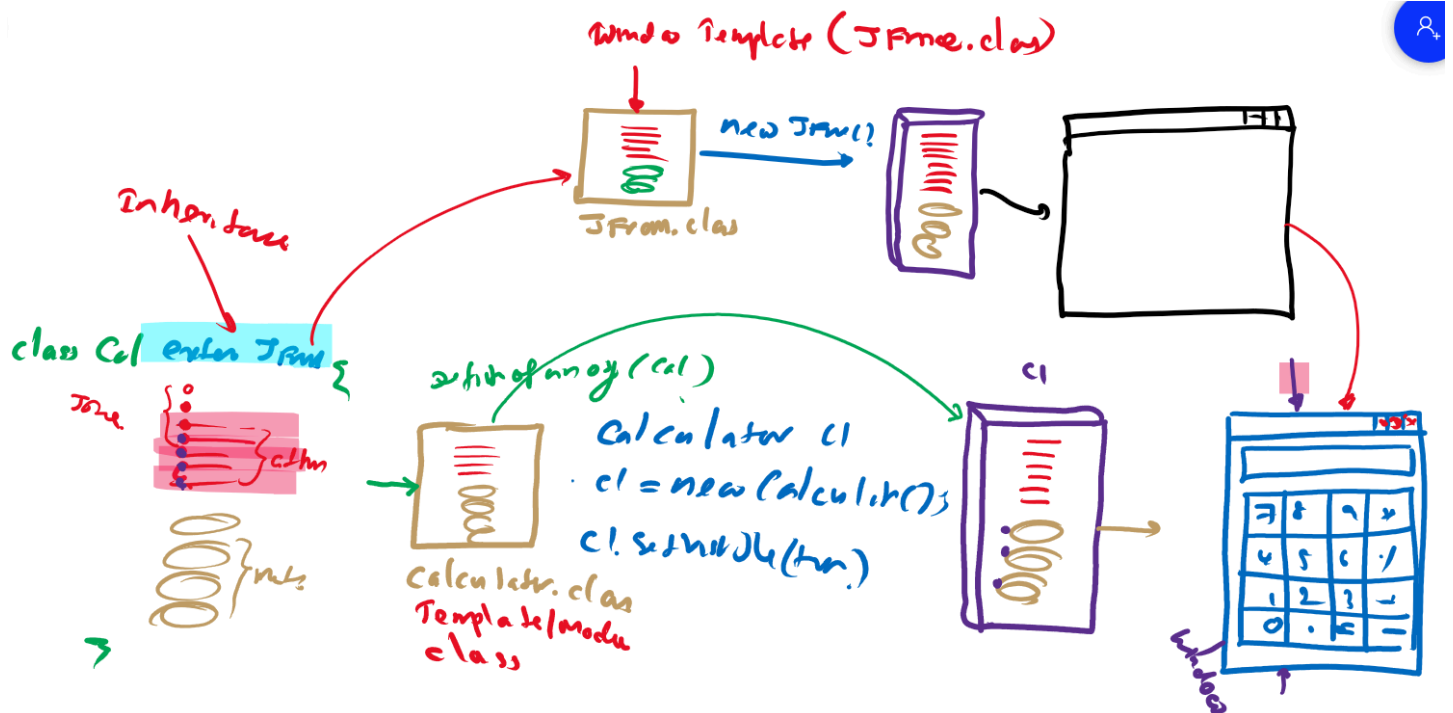
```



```
class Demo{
    public static void main(String args[]){
        ArrayList <Student>stList=new ArrayList<>();
        stList.add(new Student("S0001","Nimal",65,67));
        stList.add(new Student("S0002","Amal",85,70));
        stList.add(new Student("S0003","Bimal",35,30));
        stList.add(new Student("S0004","Ramal",55,70));
        stList.add(new Student("S0005","Anil",95,90));
        System.out.println(stList);

        for (int i = 0; i < stList.size(); i++){
            Student s1=stList.get(i);
            System.out.println(s1);
        }

        stList.remove(new Student("S0003","Bimal",35,30));
        System.out.println(stList);
    }
}
```



Q195

```
import java.util.*;
class Calculator{

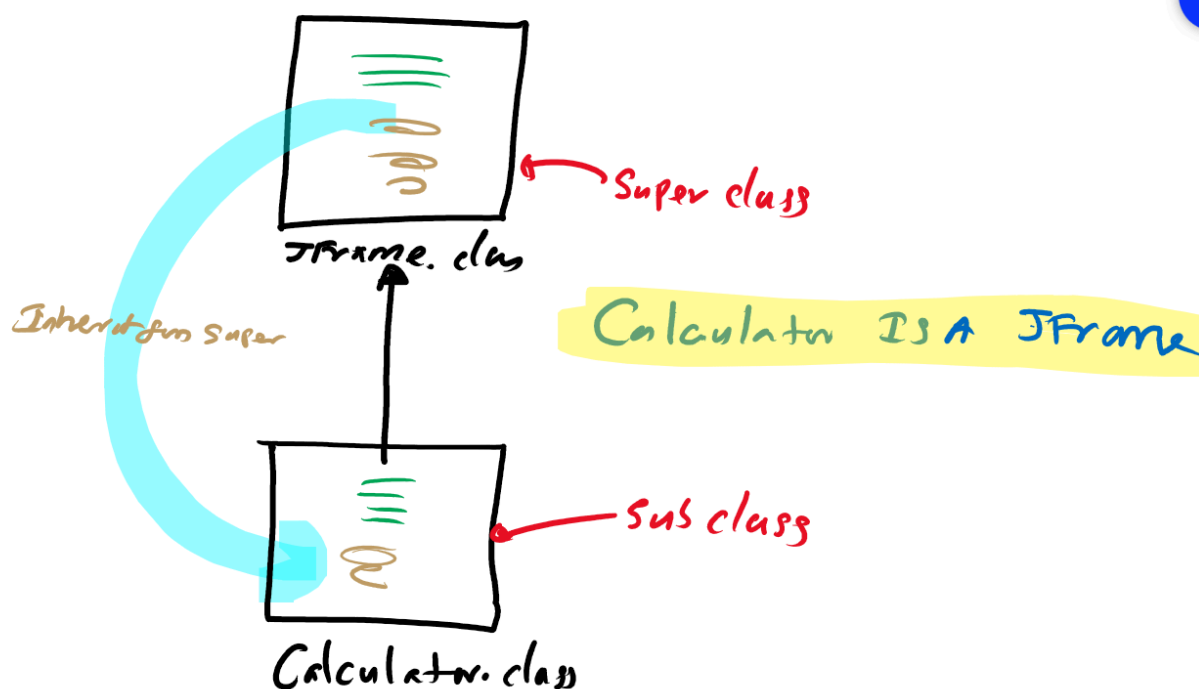
}
class Demo{
    public static void main(String args[]){
        new Calculator().setVisible(true);
    }
}
```

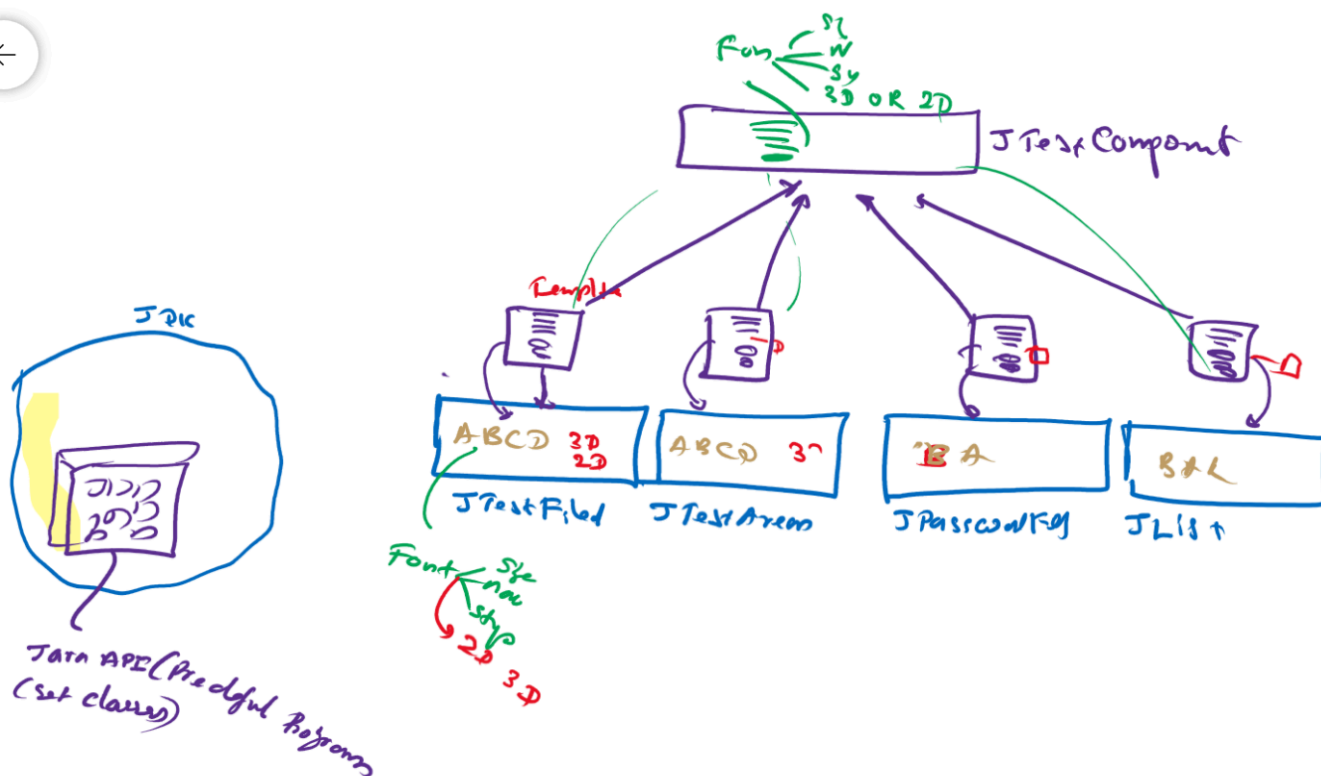
Q196 Reusability

=====

```
import java.util.*;
import javax.swing.*;
class Calculator extends JFrame{

}
class Demo{
    public static void main(String args[]){
        Calculator c1=new Calculator();
        c1.setSize(300,300);
        c1.setTitle("Calculator");
        c1.setDefaultCloseOperation(Calculator.EXIT_ON_CLOSE);
        c1.setVisible(true);
    }
}
//class Calculator extends JFrame{-->Inheritance
//Calculator-->Subclass
//JFrame -->Superclass
```





Q198 Polymorphism

Q199 Case I

```
class A{
class B extends A{ // "extends" --> Inheritance
```

```
}
//A -> superclass
//B -> subclass
```

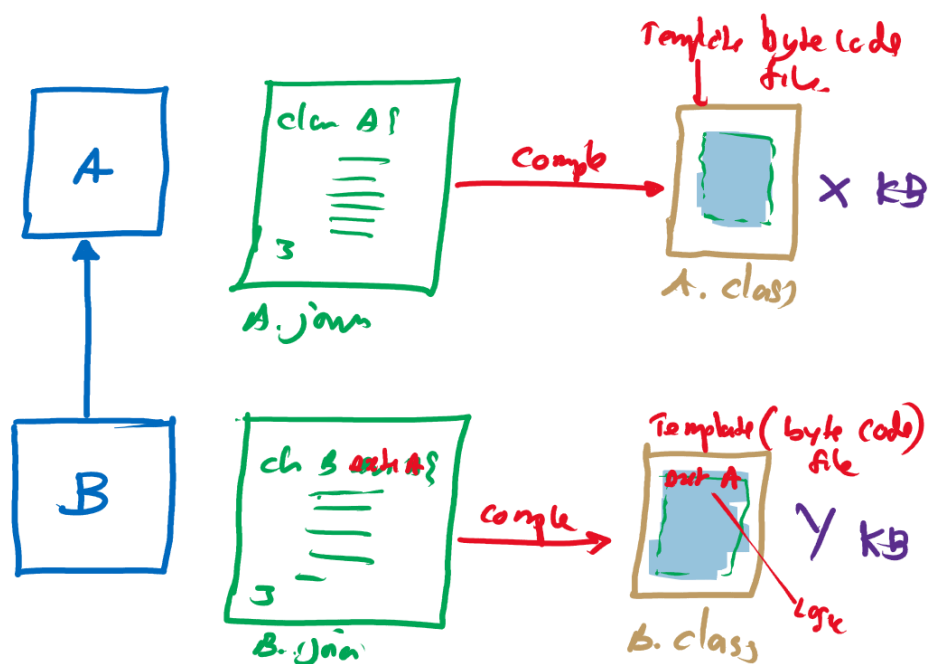
Q200 Case II

```
class A{
    int a;
    public void printA(){
        System.out.println("print a : "+a);
    }
}

class B extends A{
    int b;
    public void printB(){
        System.out.println("print b : "+b);
    }
    public void printAB(){
        System.out.println("print a : "+a); // Legal ?
        System.out.println("print b : "+b);
    }
}
```

```
public void callPrints(){  
    printA();//Legal  
    printB();  
}  
}  
/////////////////////////////////////////////////////////////////  
Q201 Exercise  
=====  
import javax.swing.*;  
class Calculator extends JFrame{  
    Calculator(){  
        setSize(300,300); //Legal ?  
        setTitle("Calculator"); //Legal ?  
        setDefaultCloseOperation(EXIT_ON_CLOSE); //Legal ?  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        Calculator c1=new Calculator();  
        c1.setVisible(true);  
    }  
}  
/////////////////////////////////////////////////////////////////  
Q202 Case III  
=====  
class A{  
    int a;  
    public void printA(){  
        System.out.println("print a : "+a);  
    }  
}  
class B extends A{  
    int b;  
    public void printB(){  
        System.out.println("print b : "+b);  
    }  
    public void printAB(){  
        System.out.println("print a : "+a); //Legal ?  
        System.out.println("print b : "+b);  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        B b1=null;  
        b1.a=100; //Legal ?  
        b1.b=200;  
        b1.printA(); //Legal ?  
        b1.printB();  
    }  
}
```

=====



- (1) $x > y$
- (2) $x >= y$
- (3) $y > x$
- (4) $y >= x$
- (5) $x == y$
- (6) $x != y$
- (7) None of these

=====

```

class A{
    int a;
    static{
        System.out.println("static block A");
    }
    public void printA(){
        System.out.println("print a : "+a);
    }
}

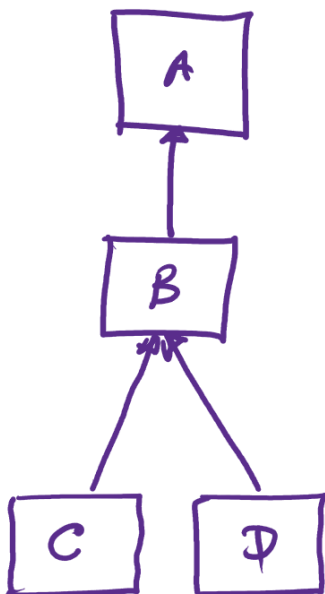
class B extends A{
    int b;
    static{
        System.out.println("static block B");
    }
    public void printB(){
        System.out.println("print b : "+b);
    }
}

class Demo{
    public static void main(String args[]){
        new B();
    }
}
    
```

Q205 Exercise

=====

```
class A{
    int a;
    static{
        System.out.println("static block A");
    }
}
class B extends A{
    int b;
    static{
        System.out.println("static block B");
    }
}
class C extends B{
    int c;
    static{
        System.out.println("static block C");
    }
}
class D extends B{
    int d;
    static{
        System.out.println("static block D");
    }
}
class Demo{
    public static void main(String args[]){
        new D();
    }
}
```



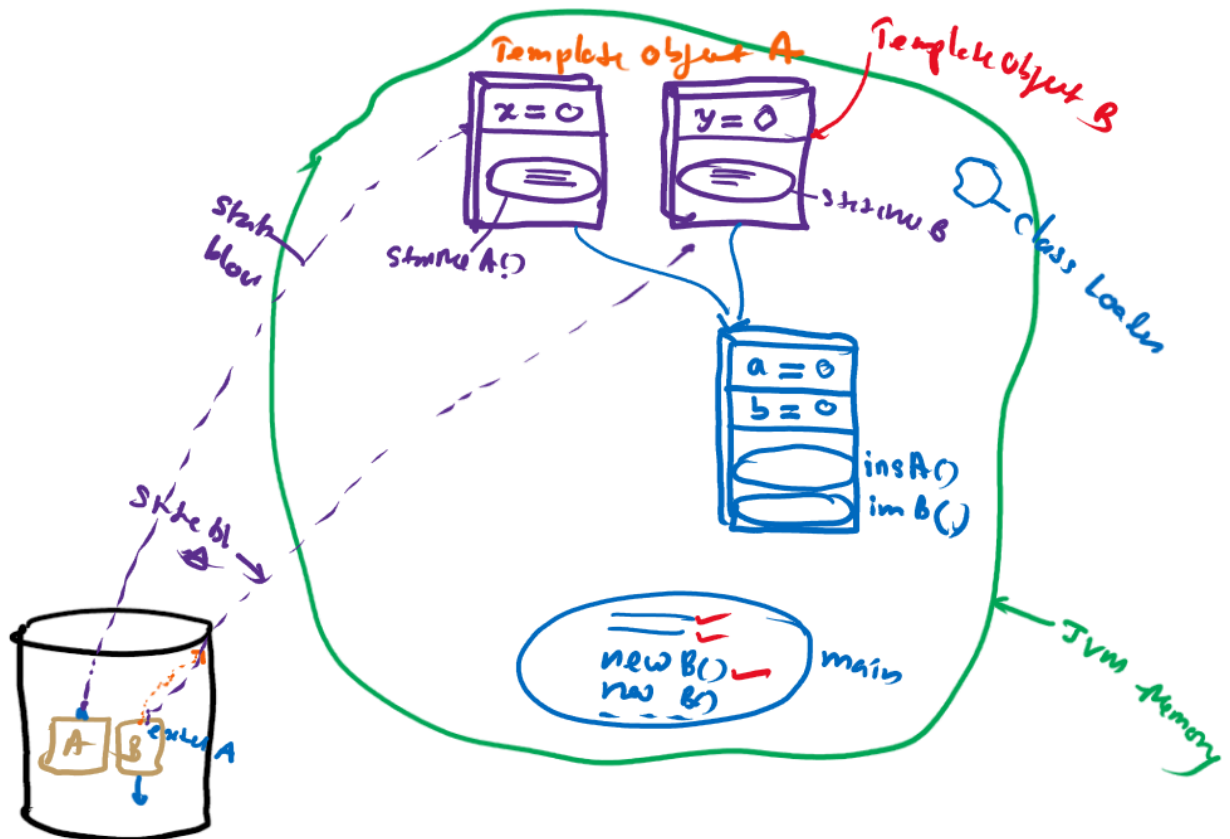
=====

```

class A{
    int a;
    static int x;
    public void instanceMethodA(){
        System.out.println("instance method A");
    }
    public static void staticMethodA(){
        System.out.println("static method A");
    }
}

class B extends A{
    int b;
    static int y;
    public void instanceMethodB(){
        System.out.println("instance method B");
    }
    public static void staticMethodB(){
        System.out.println("static method B");
    }
}

class Demo{
    public static void main(String args[]){
        B b1=new B();
    }
}
    
```



////////////////////////////////////
Q207Case VI (Calling super class constructors)
=====

```
class A{
    int a;
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
    A(int i, int j){
        System.out.println("A(int,int)");
    }
}
class B extends A{
    int b;
    B(){
        System.out.println("B()");
    }
    B(int i){
        System.out.println("B(int)");
    }
    B(int i, int j){
        System.out.println("B(int,int)");
    }
}
class Demo{
    public static void main(String args[]){
        B b1=new B();
        System.out.println("-----");
        B b2=new B(100);
        System.out.println("-----");
        B b3=new B(100,200);
    }
}
```

////////////////////////////////////
Q208

```
class A{
    int a;
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
    A(int i, int j){
        System.out.println("A(int,int)");
    }
}
```

```
class Demo{
    public static void main(String args[]){
        B b1=new B();
        System.out.println("-----");
        B b2=new B(100);
        System.out.println("-----");
        B b3=new B(100,200);
    }
}
```

Q209

}

////////////////////////////////////

=====

////////////////////////////////////

```
class A{
    int a=1;
    static int x=2;
    A(){
        System.out.println("A()");
    }
    A(int i){
        System.out.println("A(int)");
    }
}
```

```
//A. super();
//B. super(10);
```

////////////////////////////////////

=====

////////////////////////////////////

=====

}

////////////////////////////////////
Q214 Case VIII (Method Overriding)
=====

```
class A{
    int a;
    public void print(){
        System.out.println("print of class A ");
    }
}
class B extends A{
    int b;
    public void print(){ //Method overriding
        System.out.println("print of class B ");
    }
}
class Demo{
    public static void main(String args[]){
        B b1=new B();
        b1.print();
    }
}
```

////////////////////////////////////
Q215 Case X (Method overriding vs overloading)
=====

```
class A{
    int a;
    public void print(){
        System.out.println("print() of class A ");
    }
}
class B extends A{
    int b;
    public void print(){ //Method overriding
        System.out.println("print() of class B ");
    }
    public void print(int a){ //Method overloading
        System.out.println("print(int) of class B ");
    }
}
class Demo{
    public static void main(String args[]){
        B b1=new B();
        b1.print();
        b1.print(100);
    }
}
```

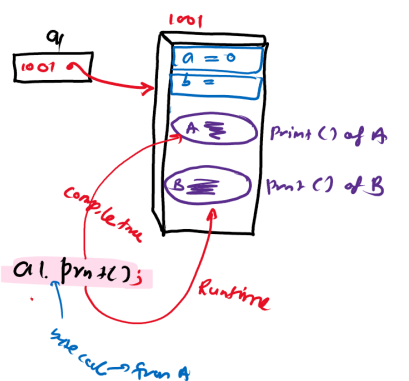
Q216 Case X (Dynamic Method dispatch)

```
class A{
    int a;
}
class B extends A{
    int b;
    public void print(){
        System.out.println("print of B ");
    }
}
class Demo{
    public static void main(String args[]){
        A a1=new B();
        a1.print();
    }
}
```

Q217

```
class A{
    int a;
    public void print(){
        System.out.println("print of A ");
    }
}
class B extends A{
    int b;
    public void print(){
        System.out.println("print of B ");
    }
}
class Demo{
    public static void main(String args[]){
        A a1=new A();
        a1.print(); //print of A

        a1=new B();
        a1.print(); //print of B [Dynamic method dispatch]
    }
}
```



```
class A{
    int a;
    public void print(){
        System.out.println("print of A ");
    }
}

class B extends A{
    int b;
    public void print(){
        System.out.println("print of B ");
    }
}

class C extends B{
    int c;
    public void print(){
        System.out.println("print of C ");
    }
}
```

```

class D extends B{
    int d;
    public void print(){
        System.out.println("print of D ");
    }
}

class Demo{
    public static void main(String args[]){
        A a1=new A();
        A a2=new B();
        A a3=new C();
        A a4=new D();

        B b1=new B();
        B b2=new C();
        B b3=new D();

        C c1=new C();
        D d1=new D();

        a1.print();    //No
        a2.print();    //yes
        a3.print();    //yes
        a4.print();    //yes
        b1.print();    //No
        b2.print();    //yes
        b3.print();    //yes
        c1.print();    //No
        d1.print();    //No
    }
}

```

Polymorphism

Polymorphism (single interface, many forms)

Compile time

- * Compiler
- * Method overloading
- * at compile time

```

int x = Math.abs(-100);
long y = Math.abs(-1000);
float f = Math.abs(1.5f);
double d = Math.abs(1.275);

```

Single interface,
many forms

Runtime

- * JVM
- * Method overriding
- * at runtime

```

Vehicle v1;
v1 = new Car();
v1.Park(); → Car.Park()
v1 = new Bus();
v1.Park(); → Bus.Park()
v1 = new Van();
v1.Park(); → Van.Park()

```

Single interface
v1.Park()

Q220 Compile time polymorphism

=====

```
class Math{
    public static int abs(int num){
        return num<0 ? -num:num;
    }
    public static long abs(long num){
        return num<0 ? -num:num;
    }
    public static float abs(float num){
        return num<0 ? -num:num;
    }
    public static double abs(double num){
        return num<0 ? -num:num;
    }
}
class Demo{
    public static void main(String args[]){
        int x=-1213;
        long y=-454511;
        float f=-12.45f;
        double d=-12.12222d;
        x=Math.abs(x); //abs-->single integer
        y=Math.abs(y);
        f=Math.abs(f);
        d=Math.abs(d);
    }
}
```

Q221 Runtime Polymorphism

=====

```
class Vehicle{
    public void park(){

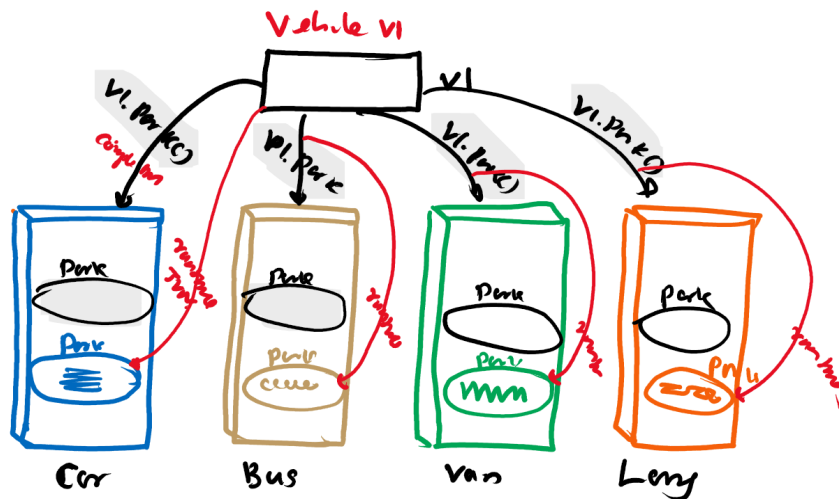
    }
}
class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking..");
    }
}
class Bus extends Vehicle{
    public void park(){
        System.out.println("Bus Parking..");
    }
}
class Van extends Vehicle{
    public void park(){
        System.out.println("Van Parking..");
    }
}
```

```

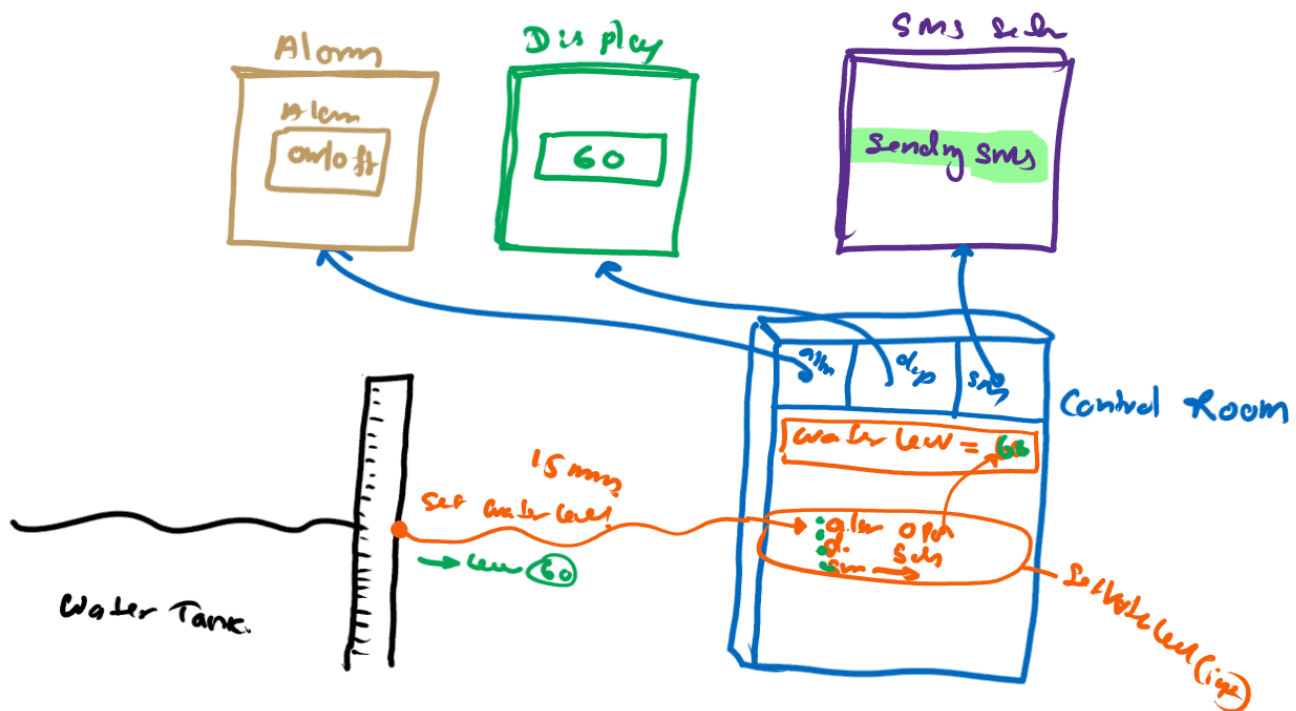
class Lorry extends Vehicle{
    public void park(){
        System.out.println("Lorry Parking..");
    }
}

class Demo{
    public static void main(String args[]){
        Vehicle[] vehicles={new Car(),new Van(),new Bus(),new Lorry()};
        for(Vehicle v1 : vehicles){
            v1.park(); //single interface
        }
    }
}

```



Q222



Polymorphism enhances software extensibility

Q223 From Q222

=====

```
import java.util.*;

class Alarm{
    public void operateAlarm(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}

class Display{
    public void display(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}

class SMSSender{
    public void send(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}

class ControlRoom{
    private Alarm alarm;
    private Display display;
    private SMSSender smsSender;
    private int waterLevel;

    public void addAlarm(Alarm alarm){
        this.alarm=alarm;
    }
    public void addDisplay(Display display){
        this.display=display;
    }
    public void addSMSSender(SMSSender smsSender){
        this.smsSender=smsSender;
    }
    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        alarm.operateAlarm(waterLevel);
        display.display(waterLevel);
        smsSender.send(waterLevel);
    }
}

class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom=new ControlRoom();
        controlRoom.addAlarm(new Alarm());
        controlRoom.addDisplay(new Display());
        controlRoom.addSMSSender(new SMSSender());
    }
}
```

```
Random r=new Random();
while(true){
    int waterLevel=r.nextInt(101); //0 to 100
    controlRoom.setWaterLevel(waterLevel);
    try{Thread.sleep(1000);}catch(Exception ex){}
}
}
```

Q224 Adding a Splitter (From Q223)

```
class Splitter{
    public void split(int waterLevel){
        System.out.println(waterLevel>=75 ? "Splitter ON":"Splitter OFF");
    }
}
```

Q225 From Q224

```
import java.util.*;

class Splitter{
    public void split(int waterLevel){
        System.out.println(waterLevel>=75 ? "Splitter ON":"Splitter OFF");
    }
}

class Alarm{
    public void operateAlarm(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}

class Display{
    public void display(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}

class SMSSender{
    public void send(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}

class ControlRoom{
    private Alarm alarm;
    private Display display;
    private SMSSender smsSender;
    private Splitter splitter;

    private int waterLevel;

    public void addSplitter(Splitter splitter){
        this.splitter=splitter;
    }
}
```


////////////////////////////////////
Q227 Incomplete (From Q226)

=====

```
import java.util.*;
class WaterLevelObserver{
    public void update(int waterLevel){

    }
}
class Alarm extends WaterLevelObserver{
    public void operateAlarm(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}
class Display extends WaterLevelObserver{
    public void display(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}
class SMSSender extends WaterLevelObserver{
    public void send(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}
class ControlRoom{
    private WaterLevelObserver[] observerArray=new WaterLevelObserver[0];

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        extendsArray();
        observerArray[observerArray.length-1]=ob;
    }
    private void extendsArray(){
        WaterLevelObserver[] tempObserverArray=new WaterLevelObserver[observerArray.length+1];
        for (int i = 0; i < observerArray.length; i++){
            tempObserverArray[i]=observerArray[i];
        }
        observerArray=tempObserverArray;
    }

    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(int i=0; i<observerArray.length; i++){
            observerArray[i].update(waterLevel);
        }
    }
}
```

```
import java.util.*;
class WaterLevelObserver{
    public void update(int waterLevel){

    }
}
class Alarm extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}
class Display extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}
class SMSSender extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}
class ControlRoom{
    private WaterLevelObserver[] observerArray=new WaterLevelObserver[0];

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        extendsArray();
        observerArray[observerArray.length-1]=ob;
    }
}
```



```

class Display extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}

class SMSSender extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}

class ControlRoom{
    private WaterLevelObserver[] observerArray=new WaterLevelObserver[0];

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        extendsArray();
        observerArray[observerArray.length-1]=ob;
    }
    private void extendsArray(){
        WaterLevelObserver[] tempObserverArray=new WaterLevelObserver[observerArray.length+1];
        for (int i = 0; i < observerArray.length; i++){
            tempObserverArray[i]=observerArray[i];
        }
        observerArray=tempObserverArray;
    }

    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(int i=0; i<observerArray.length; i++){
            observerArray[i].update(waterLevel);
        }
    }
}

class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom=new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Display());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new Splitter());
        Random r=new Random();
        while(true){
            int waterLevel=r.nextInt(101); //0 to 100
            controlRoom.setWaterLevel(waterLevel);
            try{Thread.sleep(1000);}catch(Exception ex){}
            System.out.println();
        }
    }
}

```

////////////////////////////////////
Q230 From Q229 Using class "ArrayList"

=====

```
import java.util.*;
class WaterLevelObserver{
    public void update(int waterLevel){

    }
}
class Splitter extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=75 ? "Splitter ON":"Splitter OFF");
    }
}
class Alarm extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}
class Display extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}
class SMSSender extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}
class ControlRoom{
    private ArrayList <WaterLevelObserver>observerList=new ArrayList<>();

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        observerList.add(ob);
    }
    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(WaterLevelObserver ob : observerList){
            ob.update(waterLevel);
        }
    }
}
class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom=new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Display());
        controlRoom.addWaterLevelObserver(new SMSSender());
    }
}
```


////////////////////////////////////

////////////////////////////////////

////////////////////////////////////

}

Q233

```
class A /*extends Object*/{}
class Customer extends A{

}

class Demo{
    public static void main(String args[]){
        Customer c1=new Customer();
        c1.toString(); //Legal
        c1.hashCode(); //Legal
        c1.equals(c1); //Legal
        c1.getClass(); //Legal
    }
}
```

Q234

```
class A{}
class Customer{

}

class Demo{
    public static void main(String args[]){
        Object obj;
        obj=new A();//Legal
        obj=new Customer();//Legal
        obj="abc";//Legal
        obj=new int[6]; //Legal
    }
}
```

Q235 public Class getClass()

```
=====
class A{}
class Customer{

}

class Demo{
    public static void main(String args[]){
        Customer c1=new Customer();
        Class templateObj=c1.getClass();
        System.out.println(templateObj.getName());
    }
}
```

////////////////////////////////////
Q236 public int hashCode()

=====

```
class A{}
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
}
class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1111,"Nimal");
        Customer c2=new Customer(2222,"Bimal");
        System.out.println("c1 : "+c1.hashCode());
        System.out.println("c2 : "+c2.hashCode());
    }
}
```

////////////////////////////////////
Q237 public String toString()

=====

```
class A{}
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
}
class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1111,"Nimal");
        Customer c2=new Customer(2222,"Bimal");
        System.out.println("c1 : "+c1.hashCode());
        System.out.println("c2 : "+c2.hashCode());
        System.out.println("convert hash code to hexadecimal c1 : "+(Integer.toHexString(c1.hashCode())));
        System.out.println("convert hash code to hexadecimal c2 : "+(Integer.toHexString(c2.hashCode())));

        System.out.println("c1 : "+c1);
        System.out.println("c2 : "+c2);

        System.out.println("c1.toString() : "+c1.toString());
        System.out.println("c2.toString() : "+c1.toString());
    }
}
```

////////////////////////////////////
Q238

```
class A{
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    /*
    public String toString(){
        String className=getClass().getName();
        int hashCode=hashCode();
        return className+"@"+Integer.toHexString(hashCode);
    }
    */
}
class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1111,"Nimal");
        int x=100;
        System.out.println(x); //100
        String s1="abc";
        System.out.println(s1); //abc-->s1.toString()

        System.out.println(c1); //call c1.toString()
    }
}
```

////////////////////////////////////
Q239

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return "["+code+"-"+name+"]";
    }
}
class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1111,"Nimal");
        System.out.println(c1); //[1111-Nimal]
    }
}
```

Q240 Integer Stack

=====

```
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println("\b\b");
    }
    class Node{
        int data;
        Node next;
        Node(int data){this.data=data;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack intStack=new Stack();
        intStack.push(10);
        intStack.push(20);
        intStack.push(30);
        intStack.push(40);
        intStack.push(50);
        intStack.printStack(); //[50,40,30,20,10]
        System.out.println(intStack); //[50,40,30,20,10]
    }
}
```

////////////////////////////////////
Q241 From Q240

=====

```
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.data+", ";
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    class Node{
        int data;
        Node next;
        Node(int data){this.data=data;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack intStack=new Stack();
        intStack.push(10);
        intStack.push(20);
        intStack.push(30);
        intStack.push(40);
        intStack.push(50);
        intStack.printStack(); //[50,40,30,20,10]

        System.out.println(intStack); //[50,40,30,20,10]
    }
}
```

////////////////////////////////////
Q242 Exercise

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
}

class Stack{
    private Node top;

    public void push(Customer customer){
        Node n1=new Node(customer);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.customer+", ";
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    class Node{
        Customer customer;
        Node next;
        Node(Customer customer){this.customer=customer;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        System.out.println(customerStack);//{1003-Amal, 1002-Bimal, 1001-Nimal}
    }
}
```

////////////////////////////////////
Q243 From Q242

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
}

class Stack{
    private Node top;
    public void push(Customer customer){
        Node n1=new Node(customer);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.customer+", "; //temp.customer.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    class Node{
        Customer customer;
        Node next;
        Node(Customer customer){this.customer=customer;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        System.out.println(customerStack);//{1003-Amal, 1002-Bimal, 1001-Nimal}
    }
}
```


////////////////////////////////////
Q244

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
}
class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1001,"Nimal");
        //String s1=c1; //Illegal
        String s1=c1+""; //legal==>s1=c1.toString()+""
    }
}
```

////////////////////////////////////
Q245 Exercise

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
}
class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
}
class Stack{
    private Node top;
    public void push(Customer customer){
        Node n1=new Node(customer);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
}
```



```

        Item(String itemCode, double price){
            this.itemCode=itemCode;
            this.price=price;
        }
        public String toString(){
            return itemCode+"-"+price;
        }
    }
}
class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+", "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    class Node{
        Object obj;
        Node next;
        Node(Object obj){this.obj=obj;}
    }
}
class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        System.out.println(customerStack);//{1003-Amal, 1002-Bimal, 1001-Nimal}

        Stack itemStack=new Stack();
        itemStack.push(new Item("P001",50.0));
        itemStack.push(new Item("P002",10.0));
        itemStack.push(new Item("P003",60.0));
        System.out.println(itemStack);//{P003-60.0, P002-10.0, P001-50}
    }
}

```

Q247 Exercise

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
}

class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+", "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
}

class Node{
    Object obj;
    Node next;
    Node(Object obj){this.obj=obj;}
```


////////////////////////////////////
Q249

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        //return this.code==obj.code;
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}

class Demo{
    public static void main(String args[]){
        Customer c1=new Customer(1001,"Nimal");
        Customer c2=new Customer(1001,"Nimal");
        Customer c3=new Customer(1002,"Bimal");
        System.out.println(c1==c2); //false
        System.out.println(c1==c3); //false
        System.out.println(c1.equals(c2)); //false
        System.out.println(c1.equals(c3)); //false
    }
}
```

////////////////////////////////////
Q250 From Q247

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
}
```

```

    }
}
class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(Object obj){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public void printStack(){
        System.out.println(toString());
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+", "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    class Node{
        Object obj;
        Node next;
        Node(Object obj){this.obj=obj;}
    }
}
class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        System.out.println(customerStack);//{1003-Amal, 1002-Bimal, 1001-Nimal}
        System.out.println();
    }
}

```

```
Stack itemStack=new Stack();
itemStack.push(new Item("P001",50.0));
itemStack.push(new Item("P002",10.0));
itemStack.push(new Item("P003",60.0));
System.out.println(itemStack);//{P003-60.0, P002-10.0, P001-50}
System.out.println("Index of P002-10.0 : "+itemStack.search(new Item("P002",10.0))); //1
}
}
```

Q251 From Q250

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}
```

```
}  
class Item{  
    private String itemCode;  
    private double price;  
    Item(String itemCode, double price){  
        this.itemCode=itemCode;  
        this.price=price;  
    }  
    public String toString(){  
        return itemCode+"-"+price;  
    }  
    public boolean equals(Object obj){  
        Item item=(Item)obj;  
        return item.itemCode.equals(this.itemCode);  
    }  
}
```

```
class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
```



```

        if(top!=null){
            top=top.next;
        }
    }

    public int search(Object obj){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public void printStack(){
        System.out.println(toString());
    }

    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+", "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }

    class Node{
        Object obj;
        Node next;
        Node(Object obj){this.obj=obj;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        System.out.println(customerStack);//{1003-Amal, 1002-Bimal, 1001-Nimal}
        System.out.println("Index of 1002-Bimal : "+customerStack.search(new Customer(1002,"Bimal")));//1
        System.out.println();

        Stack itemStack=new Stack();
        itemStack.push(new Item("P001",50.0));
        itemStack.push(new Item("P002",10.0));
        itemStack.push(new Item("P003",60.0));
        System.out.println(itemStack);//{P003-60.0, P002-10.0, P001-50}
        System.out.println("Index of P002-10.0 : "+itemStack.search(new Item("P002",10.0)));//1
    }
}

```

////////////////////////////////////
Q252 From Q251 lusse1

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
    public boolean equals(Object obj){
        Item item=(Item)obj;
        return item.itemCode.equals(this.itemCode);
    }
}

class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(Object obj){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
        }
    }
}
```

```

        index++;
        temp=temp.next;
    }
    return -1;
}

public void printStack(){
    System.out.println(toString());
}

public String toString(){
    String list="[";
    Node temp=top;
    while(temp!=null){
        list+=temp.obj+", "; //temp.obj.toString()
        temp=temp.next;
    }
    return top==null ? "[empty]":list+"\b\b";
}

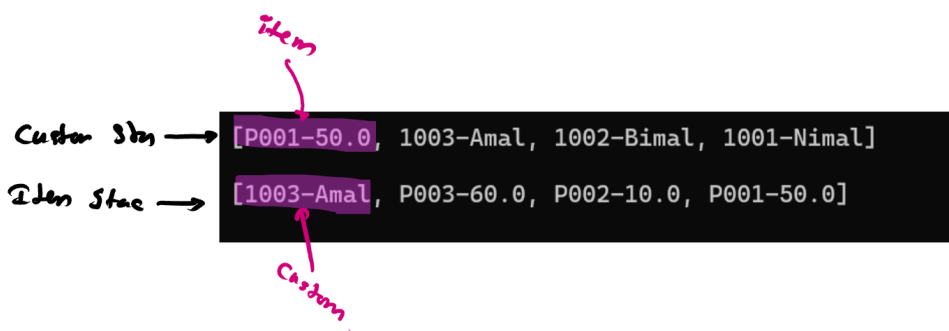
class Node{
    Object obj;
    Node next;
    Node(Object obj){this.obj=obj;}
}

}

class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        customerStack.push(new Item("P001",50.0));
        System.out.println(customerStack);//{P001-50.0, 1003-Amal, 1002-Bimal, 1001-Nimal}
        System.out.println();

        Stack itemStack=new Stack();
        itemStack.push(new Item("P001",50.0));
        itemStack.push(new Item("P002",10.0));
        itemStack.push(new Item("P003",60.0));
        itemStack.push(new Customer(1003,"Amal"));
        System.out.println(itemStack);//{1003-Amal, P003-60.0, P002-10.0, P001-50}
    }
}

```



////////////////////////////////////
Q253 luuse 2

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
    public boolean equals(Object obj){
        Item item=(Item)obj;
        return item.itemCode.equals(this.itemCode);
    }
}

class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public Object peek(){
        if(top!=null){
            return top.obj;
        }
        return null;
    }
    public int search(Object obj){
```

```

        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public void printStack(){
        System.out.println(toString());
    }

    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+" "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]"+list+"\\b\\b]";
    }

    class Node{
        Object obj;
        Node next;
        Node(Object obj){this.obj=obj;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack customerStack=new Stack();
        customerStack.push(new Customer(1001,"Nimal"));
        customerStack.push(new Customer(1002,"Bimal"));
        customerStack.push(new Customer(1003,"Amal"));
        customerStack.push(new Item("P001",50.0));
        System.out.println(customerStack);//{P001-50.0, 1003-Amal, 1002-Bimal, 1001-Nimal}
        //Customer topCustomer=customerStack.peek(); //Illegal-->need to cast
        Customer topCustomer=(Customer)customerStack.peek(); //legal-->Runtime error
        System.out.println();
    }
}

```

Q254 Issue 3

=====

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
}
```

```

    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
    public boolean equals(Object obj){
        Item item=(Item)obj;
        return item.itemCode.equals(this.itemCode);
    }
}

class Stack{
    private Node top;
    public void push(Object obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public Object peek(){
        if(top!=null){
            return top.obj;
        }
        return null;
    }
    public int search(Object obj){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
            index++;
            temp=temp.next;
        }
    }
}

```

////////////////////////////////////

////////////////////////////////////

```
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
```

```

        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }

}

class Item{
    private String itemCode;
    private double price;
    Item(String itemCode, double price){
        this.itemCode=itemCode;
        this.price=price;
    }
    public String toString(){
        return itemCode+"-"+price;
    }
    public boolean equals(Object obj){
        Item item=(Item)obj;
        return item.itemCode.equals(this.itemCode);
    }
}

class Stack<T>{
    private Node top;
    public void push(T obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public T peek(){
        if(top!=null){
            return top.obj;
        }
        return null;
    }
    public int search(Object obj){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.obj.equals(obj)){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public void printStack(){
        System.out.println(toString());
    }
}

```


////////////////////////////////////

Class ArrayList

////////////////////////////////////

Q256

```
package demo;
```

```
import java.util.ArrayList;
```

```
class Customer {
```

```
int code;
```

```
String name;
```

```
Customer(int code, String name) {
```

```
this.code = code;
```

```
this.name = name;
```

}

```
public String toString() {
```

```

        return code + "-" + name;
    }

    public boolean equals(Object obj) {
        Customer c1 = (Customer) obj;
        return this.code == c1.code;
    }
}

public class Demo {
    public static void main(String args[]) {
        ArrayList <String> strList=new ArrayList<>();
        strList.add("Niroth");

        ArrayList <Integer>intList=new ArrayList<>();
        intList.add(200); //autoboxing
        intList.add(100); //autoboxing
    }
}

```

////////////////////////////////////

=====

```
import java.util.*;
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}
class Demo{
    public static void main(String args[]){
        ArrayList <Customer>customerList=new ArrayList<>();
        customerList.add(new Customer(1001,"Nimal"));
        customerList.add(new Customer(1002,"Bimal"));
        customerList.add(new Customer(1003,"Amal"));

        for(Customer c1 : customerList){
            System.out.println(c1);
        }
    }
}
```

Q258

```
import java.util.*;
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}
class Demo{
    public static void main(String args[]){
        ArrayList customerList=new ArrayList(); //ArrayList <Object>customerList=new ArrayList<>();
        customerList.add(new Customer(1001,"Nimal")); //add(Object)
        customerList.add(new Customer(1002,"Bimal"));
        customerList.add(new Customer(1003,"Amal")); //add(Object)

        /*for(Customer c1 : customerList){ //Illegal
            System.out.println(c1);
        }*/
        for(Object c1 : customerList){ //Illegal
            System.out.println(c1);
        }
    }
}
```

Q259

```
import java.util.*;
class Customer{
    int code;
    String name;
    Customer(int code, String name){
        this.code=code;
        this.name=name;
    }
    public String toString(){
        return code+"-"+name;
    }
    public boolean equals(Object obj){
        Customer c1=(Customer)obj;
        return this.code==c1.code;
    }
}
```

////////////////////////////////////

=====


```

class Stack<T extends Vehicle>{ //Bounded type Generic class
    private Node top;
    public void push(T obj){
        Node n1=new Node(obj);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public String toString(){
        String list="[";
        Node temp=top;
        while(temp!=null){
            list+=temp.obj+", "; //temp.obj.toString()
            temp=temp.next;
        }
        return top==null ? "[empty]":list+"\b\b";
    }
    public void callPark(){
        Node temp=top;
        while(temp!=null){
            temp.obj.park();
            temp=temp.next;
        }
    }
    class Node{
        T obj;
        Node next;
        Node(T obj){this.obj=obj;}
    }
}

class Demo{
    public static void main(String args[]){
        Stack <Vehicle>vehicleStack=new Stack<>();
        vehicleStack.push(new Car());
        vehicleStack.push(new Car());
        vehicleStack.push(new Bus());
        vehicleStack.push(new Van());
        vehicleStack.callPark();

        //Stack <String>stStack=new Stack<>(); //Illegal
    }
}

```

Q262 Generic methods

=====

```
class Vehicle{
    public void park(){
        //
    }
}
class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking...");
    }
}
class Bus extends Vehicle{
    public void park(){
        System.out.println("Bus Parking...");
    }
}
class Van extends Vehicle{
    public void park(){
        System.out.println("Van Parking...");
    }
}
class Demo{
    /*public static void callPark(Car car){
        car.park();
    }
    public static void callPark(Bus bus){
        bus.park();
    }
    public static void callPark(Van van){
        van.park();
    }
    */
    public static<T extends Vehicle> void callPark(T vehicle){ //Bounded type generic method
        vehicle.park();
    }
    public static void main(String args[]){
        Demo.callPark(new Car());
        Demo.callPark(new Bus());
        Demo.callPark(new Van());
    }
}
```

Overriding Rules

Q263 Rule No.1 (Static methods) -Case 1

```
class Vehicle{
    static public void park(){
        //
    }
}
class Car extends Vehicle{
    public void park(){ //Method Overriding (Illegal)
        System.out.println("Car Parking...");
    }
}
```

Q264 Case II

```
class Vehicle{
    public void park(){
        //
    }
}
class Car extends Vehicle{
    static public void park(){ //Method Overriding (Illegal)
        System.out.println("Car Parking...");
    }
}
```

Q265 Case III

```
class Vehicle{
    static public void park(){
        System.out.println("Vehicle Parking...");
    }
}
class Car extends Vehicle{
    static public void park(){ //Legal
        System.out.println("Car Parking...");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1=new Car();
        v1.park();
    }
}
```


////////////////////////////////////
Q266 Rule No. 2 (private methods)

=====

```
class Vehicle{
    private void park(){
        System.out.println("Vehicle Parking...");
    }
    public void callPark(){
        park();
    }
}
class Car extends Vehicle{
    public void park(){ //Legal, as a new method
        System.out.println("Car Parking...");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1=new Car();
        v1.callPark();
    }
}
```

////////////////////////////////////
Q267 Rule No.3 final methods

=====

```
class Vehicle{
    final public void park(){
        System.out.println("Vehicle Parking...");
    }
}
class Car extends Vehicle{
    public void park(){ //Illegal
        System.out.println("Car Parking...");
    }
}
}
```

////////////////////////////////////
Q268 final classes

=====

```
final class Vehicle{
    public void park(){
        System.out.println("Vehicle Parking...");
    }
}
class Car extends Vehicle{ //Illegal
    public void park(){
        System.out.println("Car Parking...");
    }
}
}
```

Access modifiers vs access levels (Visibility control)

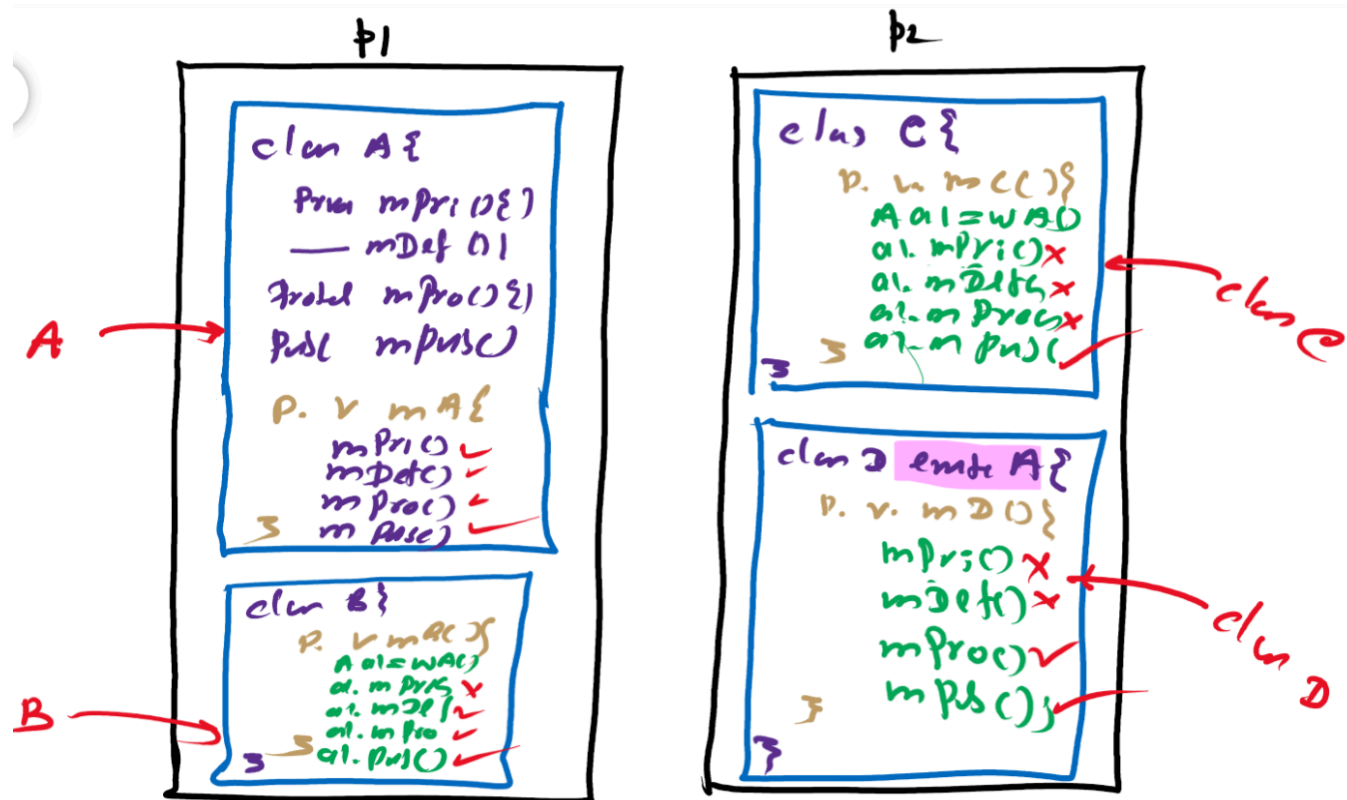
access modifier (3)

private
<default>
protected
public

access levels (4)

same class
same package
same package, different package, in a
any package, sub class

Q269



Q270 Rule No. 4 (weaker access privileges)

```
class Vehicle{
    protected void park(){
        System.out.println("Vehicle Parking...");
    }
}

class Car extends Vehicle{
    /*Insert code here*/ void park(){
        System.out.println("Car Parking...");
    }
}
```

////////////////////////////////////

=====

////////////////////////////////////

```
class Super{}
class Sub extends Super{}
class Vehicle{
```

```
class Demo{
    public static void main(String args[]){
        Vehicle v1=new Car();
        Super b=v1.park();
    }
}
```

Abstractions (Abstract classes and abstract methods)

Q273 From Q221

=====

```
abstract class Vehicle{
    abstract public void park();
}
class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking..");
    }
}
class Bus extends Vehicle{
    public void park(){
        System.out.println("Bus Parking..");
    }
}
class Van extends Vehicle{
    public void park(){
        System.out.println("Van Parking..");
    }
}
class Lorry extends Vehicle{
    public void park(){
        System.out.println("Lorry Parking..");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle[] vehicles={new Car(),new Van(),new Bus(),new Lorry()};
        for(Vehicle v1 : vehicles){
            v1.park(); //single interface
        }
    }
}
```

Q274 From Q230

=====

```
import java.util.*;
abstract class WaterLevelObserver{
    abstract public void update(int waterLevel);
}
class Splitter extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=75 ? "Splitter ON":"Splitter OFF");
    }
}
class Alarm extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}
```

```

}
class Display extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}
class SMSSender extends WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}
class ControlRoom{
    private ArrayList <WaterLevelObserver>observerList=new ArrayList<>();

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        observerList.add(ob);
    }
    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(WaterLevelObserver ob : observerList){
            ob.update(waterLevel);
        }
    }
}
class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom=new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Display());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new Splitter());
        Random r=new Random();
        while(true){
            int waterLevel=r.nextInt(101); //0 to 100
            controlRoom.setWaterLevel(waterLevel);
            try{Thread.sleep(1000);}catch(Exception ex){}
            System.out.println();
        }
    }
}

```

=====

////////////////////////////////////

=====

////////////////////////////////////

=====

////////////////////////////////////

=====

////////////////////////////////////

=====

////////////////////////////////////

=====

```
abstract class Vehicle{
    int year;
    abstract String model; //Illegal
    abstract public void start();
}
```


////////////////////////////////////
Q284 Case X

=====

```
abstract class Vehicle{
    abstract public void park();
}
class Car extends Vehicle{
    public void park(){
        System.out.println("Car Parking...");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1=new Car();
        v1.park(); //Legal
    }
}
```

////////////////////////////////////
Q285 Case XI

=====

```
abstract class Vehicle{
    static abstract public void park();
    final abstract public void start();
    abstract private void stop();
}
```

////////////////////////////////////
Q286 Case XII

=====

```
class Super{
    public void park(){

    }
}
abstract class Vehicle extends Super{ //Legal
    abstract public void park();
}
```


Java Interfaces

Q287

```
/*abstract class Vehicle{
    abstract public void park();
    abstract public void start();
    abstract public void stop();
}*/
interface Vehicle{
    public void park();
    public void start();
    public void stop();
}
```

Q288 Case I

=====

```
interface Vehicle{

}
class Car extends Vehicle{} //Illegal
class Van implements Vehicle{}
```

Q289 Case II

=====

```
interface Vehicle{
    public void park();
    public void start();
}
class Car implements Vehicle{}
class Van implements Vehicle{
    public void park(){

    }
}
class Bus implements Vehicle{
    public void park(){

    }
    public void start(){

    }
}
abstract class Lorry implements Vehicle{

}
```

```
////////////////////////////////////
Q290 Case III
```

```
=====
```

```
interface Vehicle{
    public void park(); //Implicitly abstract
    abstract public void start();
}
```

```
////////////////////////////////////
Q291 Case IV
```

```
=====
```

```
interface Vehicle{
    public void park(){ //Illegal, implicitly abstract

    }
    public void start();
}
```

```
////////////////////////////////////
Q292 Case V
```

```
=====
```

```
interface Vehicle{
    void park(); //implicitly public
    void start();
}
class Car implements Vehicle{
    public void start(){ //Legal

    }
    void park(){ //Illegal-->weaker access privileges

    }
}
```

```
////////////////////////////////////
Q293 Case VI
```

```
=====
```

```
interface Vehicle{
    //int year; // Illegal, public final static
    int year=2026; //Legal
    void park();
}
class Demo{
    public static void main(String args[]){
        System.out.println(Vehicle.year); //Legal, static
        Vehicle.year=1999; //Illegal
    }
}
```

Q294 Case VII

=====

```
interface Vehicle{
    int year=2026;

    Vehicle(){ //Illegal

    }
    void park();
}
```

Q295 Case VIII

=====

```
interface Vehicle{
    void park();
}
class Demo{
    public static void main(String args[]){
        Vehicle v1; //Legal
        v1=new Vehicle(); //Illegal
    }
}
```

Q296 Case IX

=====

```
interface Vehicle{
    void park();
}
class Car implements Vehicle{
    public void park(){
        System.out.println("Car Parking..");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1;
        v1=new Car(); //Legal
        v1.park();
    }
}
```

////////////////////////////////////
Q297 From Q273

=====

```
interface Vehicle{
    public void park();
}
class Car implements Vehicle{
    public void park(){
        System.out.println("Car Parking..");
    }
}
class Bus implements Vehicle{
    public void park(){
        System.out.println("Bus Parking..");
    }
}
class Van implements Vehicle{
    public void park(){
        System.out.println("Van Parking..");
    }
}
class Lorry implements Vehicle{
    public void park(){
        System.out.println("Lorry Parking..");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle[] vehicles={new Car(),new Van(),new Bus(),new Lorry()};
        for(Vehicle v1 : vehicles){
            v1.park(); //single interface
        }
    }
}
```

////////////////////////////////////
Q298 From Q274

=====

```
import java.util.*;
interface WaterLevelObserver{
    public void update(int waterLevel);
}
class Splitter implements WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=75 ? "Splitter ON":"Splitter OFF");
    }
}
class Alarm implements WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");
    }
}
```

```

class Display implements WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("WaterLevel : "+waterLevel);
    }
}

class SMSSender implements WaterLevelObserver{
    public void update(int waterLevel){
        System.out.println("Sending water level : "+waterLevel);
    }
}

class ControlRoom{
    private ArrayList <WaterLevelObserver>observerList=new ArrayList<>();

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        observerList.add(ob);
    }

    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(WaterLevelObserver ob : observerList){
            ob.update(waterLevel);
        }
    }
}

class Demo{
    public static void main(String args[]){
        ControlRoom controlRoom=new ControlRoom();
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Alarm());
        controlRoom.addWaterLevelObserver(new Display());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new SMSSender());
        controlRoom.addWaterLevelObserver(new Splitter());
        Random r=new Random();
        while(true){
            int waterLevel=r.nextInt(101); //0 to 100
            controlRoom.setWaterLevel(waterLevel);
            try{Thread.sleep(1000);}catch(Exception ex){}
            System.out.println();
        }
    }
}

```

```
////////////////////////////////////  
Q299 Case X
```

```
=====
```

```
interface Vehicle{  
    static void park(); //Illegal  
    private void start(); //Illegal  
    final void stop(); //Illegal  
}
```

```
////////////////////////////////////  
Q300 Exercise
```

```
=====
```

```
import javax.swing.*;  
abstract class WaterLevelObserver{  
    abstract public void update(int waterLevel);  
}  
class Alarm extends JFrame, WaterLevelObserver{ //Illegal, No multiple inheritance in java  
    public void update(int waterLevel){  
        System.out.println(waterLevel>=50 ? "Alarm ON":"Alarm OFF");  
    }  
}
```

```
////////////////////////////////////  
Q301 Case XI
```

```
=====
```

```
import java.util.*;  
import java.awt.*;  
import javax.swing.*;  
interface WaterLevelObserver{  
    public void update(int waterLevel);  
}  
class AlarmWindow extends JFrame implements WaterLevelObserver{  
    private JLabel lblAlarm;  
  
    AlarmWindow(){  
        setSize(300,300);  
        setTitle("Alarm window");  
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);  
        setLocationRelativeTo(null);  
        setLayout(new FlowLayout());  
  
        lblAlarm=new JLabel();  
        lblAlarm.setFont(new Font("",1,30));  
        add(lblAlarm);  
        setVisible(true);  
    }  
    public void update(int waterLevel){  
        lblAlarm.setText(waterLevel>=50 ? "Alarm ON":"Alarm OFF");  
    }  
}
```

```

class ControlRoom{
    private ArrayList <WaterLevelObserver>observerList=new ArrayList<>();

    private int waterLevel;

    public void addWaterLevelObserver(WaterLevelObserver ob){
        observerList.add(ob);
    }
    public void setWaterLevel(int waterLevel){
        if(this.waterLevel!=waterLevel){
            this.waterLevel=waterLevel;
        }
        for(WaterLevelObserver ob : observerList){
            ob.update(waterLevel);
        }
    }
}

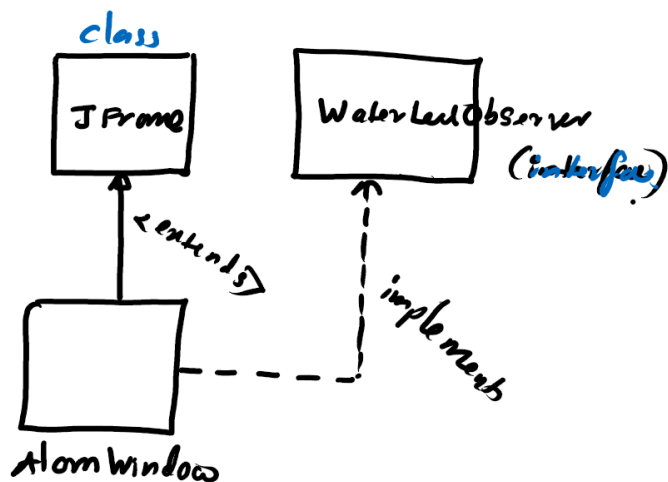
```

```

class Demo{
    public static void main(String args[]){
        ControlRoom cn=new ControlRoom();
        cn.addWaterLevelObserver(new AlarmWindow());
        cn.addWaterLevelObserver(new AlarmWindow());
        cn.addWaterLevelObserver(new AlarmWindow());

        Random r=new Random();
        while(true){
            int waterLevel=r.nextInt(100);
            cn.setWaterLevel(waterLevel);
            try{Thread.sleep(2000);}catch(Exception xe){}
        }
    }
}

```



class AlarmWindow extends JFrame implements WaterLevelObserver {

```
////////////////////////////////////
Q302 Case XII
```

```
=====
```

```
interface P{}
interface Q{}
class A{}
class B extends A implements P,Q{} //Legal
```

```
////////////////////////////////////
Q303 Case XIII
```

```
=====
```

```
interface P{}
interface Q{}
interface R extends P,Q{} //Legal
```

```
////////////////////////////////////
Q304 Case XIV
```

```
=====
```

```
class A{}
interface P extends A{} //Illegal
interface Q implements A{} //Illegal
```

```
////////////////////////////////////
Q305 Case XV
```

```
=====
```

```
interface Vehicle{
    default public void park(){ //Legal, since JDK1.8
        System.out.println("default implementation..");
    }
    public void start();//abstract
}
class Car implements Vehicle{
    public void start(){
        System.out.println("Car Parking...");
    }
}
class Demo{
    public static void main(String args[]){
        Vehicle v1=new Car();
        v1.park();
        v1.start();
    }
}
```

```
////////////////////////////////////
Q306 Case XVI
```

```
=====
```

```
interface Vehicle{
    static public void park(){ //Legal, since JDK1.8
        System.out.println("static implementation..");
    }
}
```


////////////////////////////////////

////////////////////////////////////

////////////////////////////////////

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JButton btnExit;

    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        btnExit=new JButton("Exit");
        btnExit.setFont(new Font("",1,30));
        btnExit.add(new ButtonClick());
        add(btnExit);
    }
}
```

```
class ButtonClick{
    public void click(){
        System.out.println("Exited...");
        System.exit(0);
    }
}

class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

Q309 ActionListener vs ActionEvent

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JButton btnExit;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        btnExit=new JButton("Exit");
        btnExit.setFont(new Font("",1,30));
        btnExit.addActionListener(new ButtonClick());
        add(btnExit);
    }
}

class ButtonClick implements ActionListener{
    public void actionPerformed(ActionEvent evt){
        System.out.println("Exited...");
        System.exit(0);
    }
}

class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

Q310 MouseListener vs MouseEvent

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JButton btnExit;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        btnExit=new JButton("Exit");
        btnExit.setFont(new Font("",1,30));
        btnExit.addMouseListener(new ButtonClick());
        add(btnExit);
    }
}

class ButtonClick implements MouseListener{
    public void mouseExited(MouseEvent evt){
        System.out.println("Mouse Exited...");
    }
    public void mouseEntered(MouseEvent evt){
        System.out.println("Mouse Entered...");
    }
    public void mouseReleased(MouseEvent evt){
        System.out.println("Mouse relesed...");
    }
    public void mousePressed(MouseEvent evt){
        System.out.println("Mouse Pressed...");
    }
    public void mouseClicked(MouseEvent evt){
        System.out.println("Mouse Clicked...");
    }
}

class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

Q311 KeyListener vs KeyEvent

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JTextField txtTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        txtTitle.addKeyListener(new TextFiledListener());
        add(txtTitle);
    }
}

class TextFiledListener implements KeyListener{
    public void keyReleased(KeyEvent evt){
        System.out.println("keyReleased");
    }
    public void keyPressed(KeyEvent evt){
        System.out.println("keyPressed");
    }
    public void keyTyped(KeyEvent evt){
        System.out.println("keyTyped");
    }
}

class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

Q312 Exercise

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        add(btnTitle);
    }
}

class ButtonActionListener implements ActionListener{
    public void actionPerformed(ActionEvent evt){
        String title=txtTitle.getText();
        setTitle(title);
    }
}

class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

Q313 Option 1 for Q312 (NullPointerException)

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
```

```

        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        btnTitle.addActionListener(new ButtonActionListener());
        add(btnTitle);
    }
}
class ButtonActionListener implements ActionListener{
    MyFrame myFrame;

    public void actionPerformed(ActionEvent evt){
        String title=myFrame.txtTitle.getText();
        myFrame.setTitle(title);
    }
}
class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}

```

//////////////////////////////////////
 Q314 From Q313

```

=====
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        btnTitle.addActionListener(new ButtonActionListener(this));
        add(btnTitle);
    }
}
class ButtonActionListener implements ActionListener{
    MyFrame myFrame;

    ButtonActionListener(MyFrame myFrame){
        this.myFrame=myFrame;
    }
}

```


Q316 Option 3 Using Regular Inner classes

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        btnTitle.addActionListener(new ButtonActionListener());
        add(btnTitle);
    }
    //-----Regular inner class-----
    class ButtonActionListener implements ActionListener{
        public void actionPerformed(ActionEvent evt){
            String title=txtTitle.getText();
            setTitle(title);
        }
    }
}
class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```


Q317 Regular Inner class (Member inner)

```
class Outer{
    int aOuter=100;
    public void mOuter(){

    }
    class Inner{
        int aInner=200;
        public void mInner(){
            System.out.println("a inner : "+aInner);
            System.out.println("a outer : "+aOuter); //Legal
        }
    }
}

class Demo{
    public static void main(String args[]){
        Outer.Inner iob;
        Outer ob=new Outer();
        iob=ob.new Inner();
        iob.mInner();
    }
}
```

Q318 Option 4 (Local Inner classes)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        //-----Local inner class-----
        class ButtonActionListener implements ActionListener{
            public void actionPerformed(ActionEvent evt){
                String title=txtTitle.getText();
                setTitle(title);
            }
        }
    }
}
```

```
class Demo{
    public static void main(String args[]){
        //Vehicle v1=new Vehicle();
        Vehicle v2=new Vehicle(){
            int year;
            public void park(){
                System.out.println("Car Parking...");
            }
            public void start(){
            }
        };
    }
}
```

////////////////////////////////////
Q321 Option 5 for Q312

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        //-----Anonymous inner class-----
        ActionListener ob=new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String title=txtTitle.getText();
                setTitle(title);
            }
        };
        btnTitle.addActionListener(ob);
        add(btnTitle);
    }
}
class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

////////////////////////////////////
Q322 From Q321

=====

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MyFrame extends JFrame{
    JTextField txtTitle;
    JButton btnTitle;
    MyFrame(){
        setSize(300,300);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new FlowLayout());

        txtTitle=new JTextField(8);
        txtTitle.setFont(new Font("",1,30));
        add(txtTitle);

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        //-----Anonymous inner class-----
        btnTitle.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                String title=txtTitle.getText();
                setTitle(title);
            }
        });
        add(btnTitle);
    }
}
class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}
```

////////////////////////////////////
Q323 Exercise

=====

```
class Vehicle{
    public void park(){

    }
}
class Car extends Vehicle{
    int year;
    public void park(){
        System.out.println("Car Parking...");
    }
    public void start(){

    }
}
```

```
}  
class Demo{  
    public static void main(String args[]){  
        Vehicle v1=new Vehicle(); //Legal  
        Vehicle v2=new Vehicle(){  
            int year;  
            public void park(){  
                System.out.println("Car Parking...");  
            }  
            public void start(){  
            }  
        };  
        v2.park();  
        //v2.start();  
    }  
}
```

```
abstract class Vehicle{
    abstract public void park();
}

class Demo{
    public static void main(String args[]){
        Vehicle v1=new Vehicle(); //Illegal

        Vehicle v2=new Vehicle(){
            public void park(){
                System.out.println("Car Parking...");
            }
            public void start(){
            }
        };
        v2.park();
    }
}
```

```
abstract class Vehicle{

}

class Demo{
    public static void main(String args[]){
        Vehicle v1=new Vehicle(); //Illegal
        Vehicle v2=new Vehicle(){}; //Legal->Anonymous inner
    }
}
```

```
////////////////////////////////////  
Q326
```

```
interface Vehicle{  
  
}  
class Demo{  
    public static void main(String args[]){  
        Vehicle v1=new Vehicle(); //Illegal  
        Vehicle v2=new Vehicle(){}; //Legal  
    }  
}
```

```
////////////////////////////////////  
Lambda Expression (Since JDK1.8)  
////////////////////////////////////
```

```
Q327
```

```
interface Vehicle{  
    public void park();  
}  
class Demo{  
    public static void main(String args[]){  
        Vehicle v1=new Vehicle(){  
            public void park(){  
                System.out.println("Anonymous inner..");  
            }  
        };  
        Vehicle v2=()->{  
            System.out.println("Lambda Expression..");  
        };  
        v1.park();  
        v2.park();  
    }  
}
```

```
////////////////////////////////////  
Q328 Option 6 for Q312 (Using Lambda Expression)
```

```
=====
```

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.*;  
class MyFrame extends JFrame{  
    JTextField txtTitle;  
    JButton btnTitle;  
    MyFrame(){  
        setSize(300,300);  
        setDefaultCloseOperation(EXIT_ON_CLOSE);  
        setLocationRelativeTo(null);  
        setLayout(new FlowLayout());  
  
        txtTitle=new JTextField(8);  
        txtTitle.setFont(new Font("",1,30));  
        add(txtTitle);  
    }  
}
```

```

        btnTitle=new JButton("Set Title");
        btnTitle.setFont(new Font("",1,30));
        //-----Lambda Expression-----
        btnTitle.addActionListener((ActionEvent evt)->{
            String title=txtTitle.getText();
            setTitle(title);
        });
        add(btnTitle);
    }
}
class Demo{
    public static void main(String args[]){
        new MyFrame().setVisible(true);
    }
}

```

//////////////////////////////////////
 Q329 Case 1

```

=====
interface Vehicle{
    public void park();
}
class Demo{
    public static void main(String args[]){
        Vehicle v2=()->{
            System.out.println("Lambda Expression..");
        }//Illegal; ==> ";"
        v1.park();
    }
}

```

//////////////////////////////////////
 Q330 Case II

```

=====
interface Vehicle{
    public void park();
}
class Demo{
    public static void main(String args[]){
        Vehicle car=()->System.out.println("Car parking..");
        Vehicle van=()->System.out.println("Van parking..");
        Vehicle bus=()->System.out.println("Bus parking..");
        car.park();
        van.park();
        bus.park();
    }
}

```

Q331 Case III

=====

```
interface Vehicle{
    public void park();
}
class Demo{
    public static void main(String args[]){
        Vehicle car=()->{
            System.out.println("Car parking..");
            System.out.println("Car parking..");
        };
        car.park();
    }
}
```

Q332 Case IV

=====

```
interface Vehicle{
    public void park(int location);
}
class Demo{
    public static void main(String args[]){
        Vehicle car1=(int location)->{
            System.out.println("Car parking.." +location);
        };
        car1.park(1001);

        Vehicle car2=(x)->{
            System.out.println("Car parking.." +x);
        };
        car2.park(1002);
    }
}
```

Q333 Case V

=====

```
interface Vehicle{
    public boolean park();
}
class Demo{
    public static void main(String args[]){
        Vehicle car1=()->true; //Legal
        Vehicle car2=()->return true; //Illegal

        Vehicle car3=(x)->{
            return true; //Legal
        };
    }
}
```


////////////////////////////////////
Q334 Case VI

=====

```
interface Vehicle{
    public void park();
    public void start();
}

class Demo{
    public static void main(String args[]){
        Vehicle car1=()->{ //Illegal, Vehicle is not a functional interface

        };

    }
}
```

////////////////////////////////////
Q335 Case VII (Default methods)

=====

```
interface Vehicle{
    public void park();
    default public void start(){
        System.out.println("default implementation of start....");
    }
}

class Demo{
    public static void main(String args[]){
        Vehicle car1=()->{ //legal, Vehicle is a functional interface
            System.out.println("Park....");

        };
        car1.park();
        car1.start();
    }
}
```

////////////////////////////////////
Q336 Case VIII

=====

```
abstract class Vehicle{
    abstract public void park();
}

class Demo{
    public static void main(String args[]){
        Vehicle car1=()->{ //Illegal,

        };
    }
}
```

Enumerations (Since JDK1.5)

Q337

```
class Colour{
    public static final String RED="RED";
    public static final String BLUE="BLUE";
    public static final String BLACK="BLACK";
    public static final String GREEN="GREEN";
}

class Demo{
    public static void main(String args[]){
        String c1=Colour.BLACK;
        switch(c1){
            case "BLACK":
            case "BLUE":

        }
    }
}
```

Q338 From Q337

=====

```
/*class Colour{
    public static final String RED="RED";
    public static final String BLUE="BLUE";
    public static final String BLACK="BLACK";
    public static final String GREEN="GREEN";
}*/

enum Colour{
    RED,BLUE,BLACK,GREEN; //Objects
}

class Demo{
    public static void main(String args[]){
        Colour c1; //Reference Variable type enum Colour
        c1=Colour.BLACK; //BLACK->enumeration Object type Colour
    }
}
```

Q339

```
enum Colour{
    RED,BLUE,BLACK,GREEN; //Objects
}

class Demo{
    public static void main(String args[]){
        Colour c1=new Colour(); //Illegal
    }
}
```

```
////////////////////////////////////  
Q340
```

```
enum Colour{  
    RED,BLUE,BLACK,GREEN; //Objects  
}  
class Demo{  
    public static void main(String args[]){  
        Colour[] colours=Colour.values();  
        for(Colour c1 :colours){  
            System.out.println(c1); //c1.toString  
        }  
    }  
}
```

```
////////////////////////////////////  
Q341
```

```
enum Colour{  
    RED,BLUE,BLACK,GREEN; //Objects  
  
    private int code;  
    public void setCode(int code){  
        this.code=code;  
    }  
    public int getCode(){  
        return code;  
    }  
}  
class Demo{  
    public static void main(String args[]){  
        Colour c1=Colour.BLACK;  
        System.out.println("Colour code : "+c1.getCode()); //0  
        c1.setCode(1001);  
        System.out.println("Colour code : "+c1.getCode()); //1001  
    }  
}
```

```
////////////////////////////////////  
Q342
```

```
enum Colour{  
    RED(1001),BLUE(1002),BLACK(1003),GREEN; //Objects  
  
    Colour(){  
        System.out.println("Colour()");  
    }  
    Colour(int code){  
        this.code=code;  
        System.out.println("Colour(int)");  
    }  
    private int code;  
    public void setCode(int code){  
        this.code=code;  
    }  
}
```

////////////////////////////////////

////////////////////////////////////

////////////////////////////////////

}

////////////////////////////////////
Q345

```
class MyClass{
    long[] ar=new long[100000];
    public void finalize(){
        System.out.println("Deleted...");
    }
}
class Demo{
    public static void main(String args[]){
        System.out.println("Start main");
        MyClass[] cr=new MyClass[100000];

        for(int i=0; i<100000; i++){
            new MyClass();
            System.out.println("i : "+i);
        }
        System.out.println("End main");
    }
}
```

////////////////////////////////////
Q346

```
class MyClass{
    long[] ar=new long[100000];
    public void finalize(){
        System.out.println("Deleted...");
    }
}
class Demo{
    public static void main(String args[]){
        System.out.println("Start main");
        MyClass[] cr=new MyClass[100000];

        for(int i=0; i<100000; i++){
            cr[i]=new MyClass();
            System.out.println("i : "+i);
        }
        System.out.println("End main");
    }
}
```

Q347 Case I

=====

```
class MyClass{
    int code;
    MyClass(int code){this.code=code;}
    long[] ar=new long[100000];
    public void finalize(){
        System.out.println("Deleted..." +code);
    }
}
class Demo{
    public static void main(String args[]){
        new MyClass(1001);
        new MyClass(1002);
        new MyClass(1003);
        new MyClass(1004);
    }
}
```

Q348 Case II

=====

```
class MyClass{
    int code;
    MyClass(int code){this.code=code;}
    long[] ar=new long[100000];
    public void finalize(){
        System.out.println("Deleted..." +code);
    }
}
class Demo{
    public static void main(String args[]){
        new MyClass(1001);
        new MyClass(1002);
        new MyClass(1003);
        new MyClass(1004);
        System.gc(); //
        try{Thread.sleep(100);}catch(Exception ex){}
    }
}
```

Q349 Case III (Null referencing)

=====

```
class MyClass{
    int code;
    MyClass(int code){this.code=code;}

    public void finalize(){
        System.out.println("Deleted..." +code);
    }
}
class Demo{
    public static void main(String args[]){
        MyClass c1=new MyClass(1001);
        //c1.
        //c1.
        c1=null;
        System.gc(); //
        try{Thread.sleep(100);}catch(Exception ex){}
        System.out.println("End main....");
    }
}
```

Q350 Case IV Re-assigning

=====

```
class MyClass{
    int code;
    MyClass(int code){this.code=code;}

    public void finalize(){
        System.out.println("Deleted..." +code);
    }
}
class Demo{
    public static void main(String args[]){
        MyClass c1=new MyClass(1001);
        //c1.
        //c1.
        //c1.
        c1=new MyClass(2001);
        System.gc(); //
        try{Thread.sleep(100);}catch(Exception ex){}
        System.out.println("End main....");
    }
}
```

////////////////////////////////////